Irksome User Manual

Pablo Brubeck,
Patrick E. Farrell,
Robert C. Kirby,
Scott P. MacLachlan

First edition: 2024

CONTENTS

Irksome is a Python library that adds a temporal discretization layer on top of finite element discretizations provided by Firedrake. We provide a symbolic representation of time derivatives in UFL, allowing users to write weak forms of semidiscrete PDE. Irksome maps this and a Butcher tableau encoding a Runge-Kutta method into a fully discrete variational problem for the stage values. Irksome then leverages existing advanced solver technology in Firedrake and PETSc to allow for efficient computation of the Runge-Kutta stages. Convenience classes package the underlying lower-level manipulations and present users with a friendly high-level interface time stepping.

So, instead of manually coding UFL for backward Euler for the heat equation:

```
F = inner((unew - uold) / dt, v) * dx + inner(grad(unew), grad(v)) * dx
```

and rewriting this if you want a different time-stepping method, Irksome lets you write UFL for a semidiscrete form:

```
F = inner(Dt(u), v) * dx + inner(grad(u), grad(v)) * dx
```

and maps this and a Butcher tableau for some Runge-Kutta method to UFL for a fully-discrete method. Hence, switching between RK methods is plug-and-play.

Irksome provides convenience classes to package the transformation of forms and boundary conditions and provide a method to advance by a time step. The underlying variational problem for the (possibly implicit!) Runge-Kutta stages composes fully with advanced Firedrake/PETSc solver technology, so you can use block preconditioners, multigrid with patch smoothers, and more – and in parallel, too!

CHAPTER

ONE

ACKNOWLEDGEMENTS

The Irksome team is deeply grateful to David Ham for setting up our initial documentation and Jack Betteridge for initializing the CI. Ivan Yashchuk set up the GitHub Actions. Lawrence Mitchell provided early assistance on some UFL manipulation and documentation.

We also acknowledge support from NSF 2410408.

CHAPTER

TWO

GETTING STARTED

Irksome requires Firedrake. Instructions for installing Firedrake can be found here.

If you have used pip to install the release version of Firedrake, you can:

```
$ pip install IRKsome
```

in your virtual environment. If you are running the developer/main branch of Firedrake, you can also get our development branch by:

```
$ pip install --src . --editable git+https://github.com/firedrakeproject/
↪Irksome.git#egg=Irksome
```

or, equivalently:

```
$ git clone https://github.com/firedrakeproject/Irksome.git
$ pip install --editable ./Irksome
```

CHAPTER

THREE

TUTORIALS

After your installation works, please check out our demos.

The best place to start are with some simple heat and wave equations:

3.1 Solving the Heat Equation with Irksome

Let's start with the simple heat equation on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ:

$$\begin{aligned} u_t - \Delta u &= f \\ u &= 0 \quad \text{on } \Gamma \end{aligned}$$

for some known function f. At each time t, the solution to this equation will be some function $u \in V$, for a suitable function space V.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We know have the variational problem of finding $u : [0, T] \to V$ such that

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

This demo implements an example used by Solin with a particular choice of f given below

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import GaussLegendre, Dt, TimeStepper
```

We will create the Butcher tableau for the lowest-order Gauss-Legendre Runge-Kutta method, which is more commonly known as the implicit midpoint rule:

```
butcher_tableau = GaussLegendre(1)
ns = butcher_tableau.num_stages
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 100
x0 = 0.0
x1 = 10.0
```

(continues on next page)

(continued from previous page)

```
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)
V = FunctionSpace(msh, "CG", 1)
```

We define variables to store the time step and current time value:

```
dt = Constant(10.0 / N)
t = Constant(0.0)
```

This defines the right-hand side using the method of manufactured solutions:

```
x, y = SpatialCoordinate(msh)
S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step:

```
u = Function(V)
u.interpolate(uexact)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome:

```
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx
bc = DirichletBC(V, 0, "on_boundary")
```

Later demos will show how to use Firedrake's sophisticated interface to PETSc for efficient block solvers, but for now, we will solve the system with a direct method.

```
luparams = {"mat_type": "aij",
            "ksp_type": "preonly",
            "pc_type": "lu"}
```

Most of Irksome's magic happens in the `TimeStepper`. It transforms our semidiscrete form F into a fully discrete form for the stage unknowns and sets up a variational problem to solve for the stages at each time step.

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      solver_parameters=luparams)
```

This logic is pretty self-explanatory. We use the `TimeStepper`'s `advance` method, which solves the variational problem to compute the Runge-Kutta stage values and then updates the solution.:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - t)
    stepper.advance()
    print(float(t))
    t.assign(t + dt)
```

Finally, we print out the relative L^2 error:

```
print()
print(norm(u-uexact)/norm(uexact))
```

So far we have neglected the fact that the problem is linear, and that the Jacobian is time-independent. We can instead formulate the semidiscrete form *F* in terms of a `TrialFunction`, and specify the option `constant_jacobian`, so that the factorization is not redone at each time step.

```
dt.assign(10.0 / N)
t.assign(0)

u = Function(V)
u.interpolate(uexact)

v = TestFunction(V)
w = TrialFunction(V)
F = inner(Dt(w), v)*dx + inner(grad(w), grad(v))*dx - inner(rhs, v)*dx

linear_stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                             constant_jacobian=True,
                             solver_parameters=luparams)
```

We can flag the Jacobian for an update by calling `invalidate_jacobian`, in case `dt` is changed

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        linear_stepper.invalidate_jacobian()
        dt.assign(1.0 - t)
    linear_stepper.advance()
    print(float(t))
    t.assign(t + dt)
```

Finally, we see that we obtain same relative L^2 error from before:

```
print()
print(norm(u-uexact)/norm(uexact))
```

3.2 Solving the Mixed form of the Heat Equation

This shows a different variational formulation of the heat equation. As before, we put $\Omega = [0, 10] \times [0, 10]$, with boundary Γ: but now we write the heat equation in mixed form:

$$\sigma + \nabla u = 0$$
$$u_t + \nabla \cdot \sigma = f$$

which gives rise to the weak form

$$(\sigma, v) - (u, \nabla \cdot v) = 0$$
$$(u_t, w) + (\nabla \cdot \sigma, w) = (f, w)$$

Here, σ is a vector-valued variable and will be discretized in a suitable $H(\mathrm{div})$ -conforming space. The variable u actually only requires L^2 regularity and will be discretized with discontinuous piecewise polynomials.

Note that this gives us a differential-algebraic system at the fully discrete level as there is no time derivative on σ.

Standard imports, although we're using a different RK scheme this time:

```
from firedrake import *
from irksome import LobattoIIIC, Dt, MeshConstant, TimeStepper

butcher_tableau = LobattoIIIC(2)
```

Build the mesh and approximating spaces:

```
N = 32
x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0
msh = RectangleMesh(N, N, x1, y1)

V = FunctionSpace(msh, "RT", 2)
W = FunctionSpace(msh, "DG", 1)
Z = V * W
```

Create time and time-step variables:

```
MC = MeshConstant(msh)
dt = MC.Constant(10.0 / N)
t = MC.Constant(0.0)
```

As in the first heat demo, we build the RHS via the method of manufactured solutions:

```
x, y = SpatialCoordinate(msh)

S = Constant(2.0)
C = Constant(1000.0)
```

(continues on next page)

(continued from previous page)

```
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5

uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
sigexact = -grad(uexact)

rhs = Dt(uexact) + div(sigexact)
```

Set up the initial condition:

```
sigu = project(as_vector([0, 0, uexact]), Z)
sigma, u = split(sigu)
```

And define the variational form:

```
v, w = TestFunctions(Z)

F = (inner(Dt(u), w) * dx + inner(div(sigma), w) * dx - inner(rhs, w) * dx
     + inner(sigma, v) * dx - inner(u, div(v)) * dx)
```

As before, we use a sparse direct method:

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "preonly",
          "pc_type": "lu"}
```

We set the time stepper as before, except there are no strongly-enforced boundary conditiuons for the mixed method:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, sigu,
                      solver_parameters=params)
```

And we advance the solution in time:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    print(float(t))
    t.assign(float(t) + float(dt))
```

Finally, we check the accuracy of the solution:

```
sigma, u = sigu.subfunctions
print("U error      : ", errornorm(uexact, u) / norm(uexact))
print("Sig error    : ", errornorm(sigexact, sigma) / norm(sigexact))
print("Div Sig error: ",
      errornorm(sigexact, sigma, norm_type='Hdiv')
      / norm(sigexact, norm_type='Hdiv'))
```

3.3 Solving the Mixed Wave Equation: Energy conservation

Let Ω be the unit square with boundary Γ. We write the wave equation as a first-order system of PDE:

$$u_t + \nabla p = 0$$
$$p_t + \nabla \cdot u = 0$$

together with homogeneous Dirichlet boundary conditions

$$p = 0 \quad \text{on } \Gamma$$

In this form, at each time, u is a vector-valued function in the Soboleve space $H(\mathrm{div})$ and *p* is a scalar-valued function. If we select appropriate test functions v and w, then we can arrive at the weak form

$$(u_t, v) - (p, \nabla \cdot v) = 0$$
$$(p_t, w) + (\nabla \cdot u, w) = 0$$

Note that in mixed formulations, the Dirichlet boundary condition is weakly enforced via integration by parts rather than strongly in the definition of the approximating space.

In this example, we will use the next-to-lowest order Raviart-Thomas elements for the velocity variable u and discontinuous piecewise linear polynomials for the scalar variable p.

Here is some typical Firedrake boilerplate and the construction of a simple mesh and the approximating spaces:

```
from firedrake import *
from irksome import GaussLegendre, Dt, MeshConstant, TimeStepper

N = 10

msh = UnitSquareMesh(N, N)
V = FunctionSpace(msh, "RT", 2)
W = FunctionSpace(msh, "DG", 1)
Z = V*W
```

Now we can build the initial condition, which has zero velocity and a sinusoidal displacement:

```
x, y = SpatialCoordinate(msh)
up0 = project(as_vector([0, 0, sin(pi*x)*sin(pi*y)]), Z)
u0, p0 = split(up0)
```

We build the variational form in UFL:

```
v, w = TestFunctions(Z)
F = inner(Dt(u0), v)*dx + inner(div(u0), w) * dx + inner(Dt(p0), w)*dx -␣
↪inner(p0, div(v)) * dx
```

Energy conservation is an important principle of the wave equation, and we can test how well the spatial discretization conserves energy by creating a UFL expression and evaluating it at each time step:

```
E = 0.5 * (inner(u0, u0)*dx + inner(p0, p0)*dx)
```

The time and time step variables:

```
MC = MeshConstant(msh)
t = MC.Constant(0.0)
dt = MC.Constant(1.0/N)
```

The two-stage Gauss-Legendre method is, like all instances of that family, A-stable and symplectic. This gives us a fourth order method in time, although our spatial accuracy is of lower order. Feel free to experiment!:

```
butcher_tableau = GaussLegendre(2)
```

Like the heat equation demo, we are just using a direct method to solve the system at each time step:

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "preonly",
          "pc_type": "lu"}

stepper = TimeStepper(F, butcher_tableau, t, dt, up0,
                      solver_parameters=params)
```

And, as with the heat equation, our time-stepping logic is quite simple. At easch time step, we print out the energy in the system:

```
print("Time    Energy")
print("==============")

while (float(t) < 1.0):
    if float(t) + float(dt) > 1.0:
        dt.assign(1.0 - float(t))

    stepper.advance()

    t.assign(float(t) + float(dt))
    print("{0:1.1e} {1:5e}".format(float(t), assemble(E)))
```

If all is well with the world, the energy will be nearly identical (up to roundoff error) at each time step because the GL methods are symplectic and applied to a linear Hamiltonian system. As an exercise, the reader should edit this code to use other RK methods. In particular, *LobattoIIIC* and *BackwardEuler* or other Radau methods as well as the symplectic DIRK *QinZhang*.

To see a nonlinear problem in action, we have a simple lid-driven cavity example, too:

3.4 Solving the Navier-Stokes Equations with Irksome

Let's consider the lid-driven cavity problem on $\Omega = [0,1] \times [0,1]$, with boundary $\Gamma_T \cup \Gamma$, where Γ_T is the top of the domain, $0 \leq x \leq 1, y = 1$:

$$\begin{aligned} u_t + u \cdot \nabla u - \frac{1}{Re}\Delta u + \nabla p &= 0 \\ \nabla \cdot u &= 0 \\ u &= (0,0) \quad \text{on } \Gamma \\ u &= (1,0) \quad \text{on } \Gamma_T \end{aligned}$$

At each time t, the solution to this equation will be some functions $(u,p) \in V \times W$, for a suitable function spaces V, W.

We transform this into weak form by multiplying arbitrary test functions $v \in V$ and $w \in W$ and integrating over Ω. This gives the variational problem of finding $u : [0,T] \to V$ and $p : [0,T] \to W$ such that

$$\begin{aligned} (u_t, v) + (u \cdot \nabla u, v) + \frac{1}{Re}(\nabla u, \nabla v) - (p, \nabla \cdot v) &= 0 \\ (\nabla \cdot u, w) &= 0 \end{aligned}$$

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import RadauIIA, Dt, MeshConstant, TimeStepper
```

We will create the Butcher tableau for the two-stage RadauIIA Runge-Kutta method:

```
butcher_tableau = RadauIIA(2)
ns = butcher_tableau.num_stages
```

Now we define the mesh and Taylor-Hood approximating space in standard Firedrake fashion:

```
N = 32
msh = UnitSquareMesh(N, N)
V = VectorFunctionSpace(msh, "CG", 2)
W = FunctionSpace(msh, "CG", 1)
Z = V*W
```

We define variables to store the time step and current time value, as well as the Reynolds number:

```
MC = MeshConstant(msh)
dt = MC.Constant(1.0 / N)
t = MC.Constant(0.0)
Re = MC.Constant(10.0)
```

We define the solution over the product space, which will get overwritten at each time step:

```
up = Function(Z)
u, p = split(up)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome:

```
v, w = TestFunctions(Z)
F = (inner(Dt(u), v) * dx + inner(dot(grad(u), u), v) * dx
     + 1/Re * inner(grad(u), grad(v)) * dx - inner(p, div(v)) * dx
     + inner(div(u), w) * dx)

bcs = [DirichletBC(Z.sub(0), as_vector([0, 0]), (1, 2, 3)),
       DirichletBC(Z.sub(0), as_vector([1, 0]), (4,))]
```

Later demos will show how to use Firedrake's sophisticated interface to PETSc for efficient solvers, but for now, we will solve the system with a direct method (Note: the matrix type needs to be explicitly set to aij if sparse direct factorization were used with a multi-stage method, as we wind up with a mixed problem that Firedrake will assemble into a PETSc MatNest otherwise):

```
luparams = {"mat_type": "aij",
            "snes_type": "newtonls",
            "snes_monitor": None,
            "ksp_type": "preonly",
            "pc_type": "lu",
            "pc_factor_mat_solver_type": "mumps",
            "snes_linesearch_type": "l2",
            "snes_force_iteration": 1,
            "snes_rtol": 1e-8,
            "snes_atol": 1e-8,
            }
```

Since this problem is ill-posed, we specify a nullspace vector to remove the possible constant "shift" in the pressure:

```
nsp = MixedVectorSpaceBasis(Z, [Z.sub(0), VectorSpaceBasis(constant=True,␣
↪comm=msh.comm)])
```

Most of Irksome's magic happens in the `TimeStepper`. It transforms our semidiscrete form F into a fully discrete form for the stage unknowns and sets up a variational problem to solve for the stages at each time step.:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, up, bcs=bcs,
                      solver_parameters=luparams, nullspace=nsp)
```

This logic is pretty self-explanatory. We use the `TimeStepper`'s `advance` method, which solves the variational problem to compute the Runge-Kutta stage values and then updates the solution.:

```
for _ in range(N):
    print(f"Stepping from time {float(t)}")
    stepper.advance()
    t.assign(float(t) + float(dt))
```

Finally, we can visualize results of the simulation using Firedrake's plotting capabilities:

```
import matplotlib.pyplot as plt
from firedrake.pyplot import streamplot
u_, p_ = up.subfunctions
fig, axes = plt.subplots()
streamplot(u_, resolution=0.02, axes=axes)
axes.set_aspect("equal")
fig.savefig("demo_nse_streamlines.png")
```

Since those demos invariably rely on the non-scalable LU factorization, we have several demos showing how to work with Firedrake solver options to deploy more efficient methods:

3.5 Block diagonal preconditioners for the heat equation

This demo applies the method suggested in:

Mardal, Nilssen, Staff, "Order-optimal preconditioners for implicit Runge-Kutta schemes applied to parabolic PDEs", SISC 29(1): 361–375 (2007),

to our ongoing heat equation demonstration problem on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ, giving rise to the weak form

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

A multi-stage RK method applied to the heat equation gives a block-structured system. The on-diagonal blocks are quite similar to what one obtains from a backward Euler discretization of the equation.

With a 2-stage method, we have

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$$

And the suggestion (analyzed rigorously) of Mardal, Nilssen, and Staff is to use a block diagonal preconditioner:

$$P = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix}$$

This allows one to leverage an existing methodology for a low order method like backward Euler for the diagonal blocks. In our case, we will simply use an algebraic multigrid scheme, although one could certainly use geometric multigrid or some other technique.

Common set-up for the problem:

```
from firedrake import *  # noqa: F403
from irksome import LobattoIIIC, TimeStepper, Dt, MeshConstant

butcher_tableau = LobattoIIIC(3)

N = 64

x0 = 0.0
```

(continues on next page)

(continued from previous page)

```
x1 = 10.0
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)

MC = MeshConstant(msh)
dt = MC.Constant(10. / N)
t = MC.Constant(0.0)

V = FunctionSpace(msh, "CG", 1)
x, y = SpatialCoordinate(msh)

S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5

uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))
u = Function(V)
u.interpolate(uexact)
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx

bc = DirichletBC(V, 0, "on_boundary")
```

Now, we define the solver parameters. PETSc-speak for taking the block diagonal is an "additive fieldsplit":

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "gmres",
          "ksp_monitor": None,
          "pc_type": "fieldsplit",    # block preconditioner
          "pc_fieldsplit_type": "additive"  # block diagaonal
          }
```

We also have to configure the (approximate) inverse of for each diagonal block. We'll just apply a sweek of gamg (PETSC's algebraic multigrid):

```
per_field = {"ksp_type": "preonly",
             "pc_type": "gamg"}

for s in range(butcher_tableau.num_stages):
    params["fieldsplit_%s" % (s,)] = per_field
```

Note that we have used the same technique for each RK stage, which is probably typical. However, it is not necessary at all.

To test this preconditioning strategy, we'll create a time stepping object which will set up the

variational problem for us:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      solver_parameters=params)
```

But, since we're just testing the efficacy of the preconditioner, we'll solve the inside variational problem one time:

```
stepper.solver.solve()
```

3.6 Solving the heat equation with monolithic multigrid

This reprise of the heat equation demo uses a monolithic multigrid algorithm suggested by Patrick Farrell to perform time advancement.

We consider the heat equation on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ: giving rise to the weak form

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

This demo implements an example used by Solin with a particular choice of f given below

We perform similar imports and setup as before:

```
from firedrake import *
from irksome import GaussLegendre, Dt, MeshConstant, TimeStepper
butcher_tableau = GaussLegendre(2)
```

However, we need to set up a mesh hierarchy to enable geometric multigrid within Firedrake:

```
N = 128
x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0

from math import log
coarseN = 8  # size of coarse grid
nrefs = log(N/coarseN, 2)
assert nrefs == int(nrefs)
nrefs = int(nrefs)
base = RectangleMesh(coarseN, coarseN, x1, y1)
mh = MeshHierarchy(base, nrefs)
msh = mh[-1]
```

From here, setting up the function space, manufactured solution, etc, are just as for the regular heat equation demo:

```
V = FunctionSpace(msh, "CG", 1)

MC = MeshConstant(msh)
dt = MC.Constant(10.0 / N)
```

(continues on next page)

(continued from previous page)

```
t = MC.Constant(0.0)

x, y = SpatialCoordinate(msh)
S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))

u = Function(V)
u.interpolate(uexact)
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx
bc = DirichletBC(V, 0, "on_boundary")
```

And now for the solver parameters. Note that we are solving a block-wise system with all stages coupled together. This performs a monolithic multigrid with pointwise block Jacobi preconditioning:

```
mgparams = {"mat_type": "aij",
            "snes_type": "ksponly",
            "ksp_type": "gmres",
            "ksp_monitor_true_residual": None,
            "pc_type": "mg",
            "mg_levels": {
                "ksp_type": "chebyshev",
                "ksp_max_it": 1,
                "ksp_convergence_test": "skip",
                "pc_type": "python",
                "pc_python_type": "firedrake.PatchPC",
                "patch": {
                    "pc_patch": {
                        "save_operators": True,
                        "partition_of_unity": False,
                        "construct_type": "star",
                        "construct_dim": 0,
                        "sub_mat_type": "seqdense",
                        "dense_inverse": True,
                        "precompute_element_tensors": None},
                    "sub": {
                        "ksp_type": "preonly",
                        "pc_type": "lu"}}},
            "mg_coarse": {
                "pc_type": "lu",
                "pc_factor_mat_solver_type": "mumps"}
            }
```

These solver parameters work just fine in the `TimeStepper`:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      solver_parameters=mgparams)
```

And we can advance the solution in time in typical fashion:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    print(float(t), flush=True)
    t.assign(float(t) + float(dt))
```

After the solve, we can retrieve some statistics about the solver:

```
steps, nonlinear_its, linear_its = stepper.solver_stats()

print("Total number of timesteps was %d" % (steps))
print("Average number of nonlinear iterations per timestep was %.2f" %␣
↪(nonlinear_its/steps))
print("Average number of linear iterations per timestep was %.2f" % (linear_
↪its/steps))
```

Finally, we print out the relative L^2 error:

```
print()
print(norm(u-uexact)/norm(uexact))
```

3.7 Solving the heat equation with monolithic multigrid and Discontinuous Galerkin-in-Time

This reprise of the heat equation demo uses a monolithic multigrid algorithm suggested by Patrick Farrell to perform time advancement, but now applies it to the Discontinuous Galerkin-in-Time discretization.

We consider the heat equation on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ: giving rise to the weak form

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

This demo implements an example used by Solin with a particular choice of f given below

We perform similar imports and setup as before:

```
from firedrake import *
from irksome import TimeStepper, Dt, DiscontinuousGalerkinScheme
```

However, we need to set up a mesh hierarchy to enable geometric multigrid within Firedrake:

```
N = 128
x0 = 0.0
x1 = 10.0
```

(continues on next page)

(continued from previous page)

```
y0 = 0.0
y1 = 10.0

from math import log
coarseN = 8  # size of coarse grid
nrefs = log(N/coarseN, 2)
assert nrefs == int(nrefs)
nrefs = int(nrefs)
base = RectangleMesh(coarseN, coarseN, x1, y1)
mh = MeshHierarchy(base, nrefs)
msh = mh[-1]
```

From here, setting up the function space, manufactured solution, etc, are just as for the regular heat equation demo:

```
V = FunctionSpace(msh, "CG", 1)

dt = Constant(10.0 / N)
t = Constant(0.0)

x, y = SpatialCoordinate(msh)
S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))

u = Function(V)
u.interpolate(uexact)
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx
bc = DirichletBC(V, 0, "on_boundary")
```

And now for the solver parameters. Note that we are solving a block-wise system with all stages coupled together. This performs a monolithic multigrid with pointwise block Jacobi preconditioning:

```
mgparams = {"mat_type": "aij",
            "snes_type": "ksponly",
            "ksp_type": "gmres",
            "ksp_monitor_true_residual": None,
            "pc_type": "mg",
            "mg_levels": {
                "ksp_type": "chebyshev",
                "ksp_max_it": 1,
                "ksp_convergence_test": "skip",
                "pc_type": "python",
                "pc_python_type": "firedrake.PatchPC",
                "patch": {
```

(continues on next page)

(continued from previous page)

```
                    "pc_patch": {
                        "save_operators": True,
                        "partition_of_unity": False,
                        "construct_type": "star",
                        "construct_dim": 0,
                        "sub_mat_type": "seqdense",
                        "dense_inverse": True,
                        "precompute_element_tensors": None},
                    "sub": {
                        "ksp_type": "preonly",
                        "pc_type": "lu"}}},
            "mg_coarse": {
                "pc_type": "lu",
                "pc_factor_mat_solver_type": "mumps"}
            }
```

These solver parameters work just fine using a *DiscontinuousGalerkinScheme*:

```
scheme = DiscontinuousGalerkinScheme(2, quadrature_degree="auto")
stepper = TimeStepper(F, scheme, t, dt, u, bcs=bc, solver_parameters=mgparams)
```

And we can advance the solution in time in typical fashion:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    print(float(t), flush=True)
    t.assign(float(t) + float(dt))
```

After the solve, we can retrieve some statistics about the solver:

```
steps, nonlinear_its, linear_its = stepper.solver_stats()

print("Total number of timesteps was %d" % (steps))
print("Average number of nonlinear iterations per timestep was %.2f" %␣
↪(nonlinear_its/steps))
print("Average number of linear iterations per timestep was %.2f" % (linear_
↪its/steps))
```

Finally, we print out the relative L^2 error:

```
print()
print(norm(u-uexact)/norm(uexact))
```

3.8 Rana/Howle/et al preconditioning

This demo applies a method suggested in:

Rana, Howle, Long, Meek, Milestone "A new block preconditioner for implicit Runge-Kutta methods for parabolic PDE problems," SISC 2021

to our ongoing heat equation demonstration problem on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ, giving rise to the weak form

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

A multi-stage RK method applied to the heat equation gives a block-structured system. The on-diagonal blocks are quite similar to what one obtains from a backward Euler discretization of the equation.

With a 2-stage method, we have

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$$

The suggestion in the paper is to approximate the Butcher matrix A with with a triangular approximation. For example, if A = LDU, then one could approximate A with LD or DU. This gives rise to a block triangular preconditioner

$$P = \begin{bmatrix} \tilde{A}_{11} & 0 \\ \tilde{A}_{21} & \tilde{A}_{22} \end{bmatrix}$$

This allows one to leverage an existing methodology for a low order method like backward Euler for the diagonal blocks. Empirical results suggest that this method is strongly stage-independent.

Common set-up for the problem:

```
from firedrake import *  # noqa: F403
from irksome import LobattoIIIC, TimeStepper, Dt, MeshConstant

butcher_tableau = LobattoIIIC(3)

N = 16

x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)

MC = MeshConstant(msh)
dt = MC.Constant(10. / N)
t = MC.Constant(0.0)

V = FunctionSpace(msh, "CG", 1)
x, y = SpatialCoordinate(msh)

S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
```

(continues on next page)

(continued from previous page)

```
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))
u = Function(V)
u.interpolate(uexact)

v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v) * dx

bc = DirichletBC(V, 0, "on_boundary")
```

Now, we define the solver parameters. We get at the Rana method through an Irksome-provided Python preconditioner. This method inherits from `firedrake.AuxiliaryOperatorPC` (it provides the Jacobian of the variational form with the approximate Butcher tableu substituted) and so provides the user with an 'aux' preconditioner to configure. Since the Rana technique gives us a block triangular matrix, a multiplicative field split exactly applies the preconditioner if its diagonal blocks are exactly inverted:

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "gmres",
          "ksp_monitor": None,
          "pc_type": "python",
          "pc_python_type": "irksome.RanaLD",
          "aux": {
              "pc_type": "fieldsplit",
              "pc_fieldsplit_type": "multiplicative"
          }}
```

But they don't have to be. We'll approximate the inverse of each diagonal block with a sweep of gamg (PETSc's multigrid):

```
per_field = {"ksp_type": "preonly",
             "pc_type": "gamg"}

for s in range(butcher_tableau.num_stages):
    params["fieldsplit_%s" % (s,)] = per_field
```

Note that we have used the same technique for each RK stage, which is probably typical. However, it is not necessary at all.

To test this preconditioning strategy, we'll create a time stepping object which will set up the variational problem for us. (Important note: The stepper puts some special information into a PETSc context for the variational problem it configures. This is vital for the Rana-type preconditioners to function. If you want to use the preconditioner outside of the `TimeStepper` then you will have some extra setup to do):

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      solver_parameters=params)
```

But, since we're just testing the efficacy of the preconditioner, we'll solve the inside variational problem one time:

```
stepper.solver.solve()
```

3.9 Accessing DIRK methods

Many practitioners favor diagonally implicit methods over fully implicit ones since the stages can be computed sequentially rather than concurrently. We support a range of DIRK methods and provide a convenient high-level interface that is very similar to other RK schemes. This demo is intended to show how to access DIRK methods seamlessly in Irksome.

This example uses the Qin Zhang symplectic DIRK to attack the mixed form of the wave equation. Let Ω be the unit square with boundary Γ. We write the wave equation as a first-order system of PDE:

$$u_t + \nabla p = 0$$
$$p_t + \nabla \cdot u = 0$$

together with homogeneous Dirichlet boundary conditions

$$p = 0 \quad \text{on } \Gamma$$

In this form, at each time, u is a vector-valued function in the Sobolev space $H(\mathrm{div})$ and *p* is a scalar-valued function. If we select appropriate test functions v and w, then we can arrive at the weak form (see the mixed wave demos for more information):

$$(u_t, v) - (p, \nabla \cdot v) = 0$$
$$(p_t, w) + (\nabla \cdot u, w) = 0$$

As in that case, we will use the next-to-lowest order Raviart-Thomas space for u and discontinuous piecewise linear elements for p.

As an example, we will use the two-stage A-stable and symplectic DIRK of Qin and Zhang, given by Butcher tableau:

$$\begin{array}{c|cc} 1/4 & 1/4 & 0 \\ 3/4 & 1/2 & 1/4 \\ \hline & 1/2 & 1/2 \end{array}$$

Imports from Firedrake and Irksome:

```
from firedrake import *
from irksome import QinZhang, Dt, MeshConstant, TimeStepper
```

We configure the discretization:

```
N = 10
msh = UnitSquareMesh(N, N)

MC = MeshConstant(msh)
t = MC.Constant(0.0)
dt = MC.Constant(1.0/N)

V = FunctionSpace(msh, "RT", 2)
```

(continues on next page)

(continued from previous page)

```
W = FunctionSpace(msh, "DG", 1)
Z = V*W

v, w = TestFunctions(Z)

butcher_tableau = QinZhang()
```

And set up the initial condition and variational problem:

```
x, y = SpatialCoordinate(msh)
up0 = project(as_vector([0, 0, sin(pi*x)*sin(pi*y)]), Z)

u0, p0 = split(up0)

F = inner(Dt(u0), v)*dx + inner(div(u0), w) * dx + inner(Dt(p0), w)*dx -␣
↪inner(p0, div(v)) * dx
```

We will keep track of the energy to determine whether we're sufficiently accurate in solving the linear system:

```
E = 0.5 * (inner(u0, u0)*dx + inner(p0, p0)*dx)
```

When stepping with a DIRK, we only solve for one stage at a time. Although we could configure PETSc to try some iterative solver, here we will just use a direct method:

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "preonly",
          "pc_type": "lu"}
```

Now, we just set the stage type to be "dirk" in Irksome and we're ready to advance in time:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, up0,
                      stage_type="dirk",
                      solver_parameters=params)

print("Time    Energy")
print("==============")
while (float(t) < 1.0):
    if float(t) + float(dt) > 1.0:
        dt.assign(1.0 - float(t))

    stepper.advance()
    print("{0:1.1e} {1:5e}".format(float(t), assemble(E)))

    t.assign(float(t) + float(dt))
```

If all is right in the universe, you should see that the energy remains constant.

We now have support for DIRKs:

3.10 Solving the Heat Equation with a DIRK in Irksome

Let's start with the simple heat equation on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ:

$$\begin{aligned} u_t - \Delta u &= f \\ u &= 0 \quad \text{on } \Gamma \end{aligned}$$

for some known function f. At each time t, the solution to this equation will be some function $u \in V$, for a suitable function space V.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We know have the variational problem of finding $u : [0, T] \to V$ such that

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

This demo implements an example used by Solin with a particular choice of f given below

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import Alexander, Dt, MeshConstant, TimeStepper
```

We will create the Butcher tableau for a three-stage L-stable DIRK due to Alexander (SINUM 14(6): 1006-1021, 1977).:

```
butcher_tableau = Alexander()
ns = butcher_tableau.num_stages
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 100
x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)
V = FunctionSpace(msh, "CG", 1)
```

We define variables to store the time step and current time value:

```
MC = MeshConstant(msh)
dt = MC.Constant(10.0 / N)
t = MC.Constant(0.0)
```

This defines the right-hand side using the method of manufactured solutions:

```
x, y = SpatialCoordinate(msh)
S = Constant(2.0)
C = Constant(1000.0)
```

(continues on next page)

(continued from previous page)

```
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step:

```
u = Function(V)
u.interpolate(uexact)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome:

```
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx
bc = DirichletBC(V, 0, "on_boundary")
```

We'll just solve each stage with gamg + gmres:

```
params = {"mat_type": "aij",
          "ksp_type": "gmres",
          "pc_type": "gamg"}
```

Most of Irksome's magic happens in the `TimeStepper`. It transforms our semidiscrete form *F* into a fully discrete form for the stage unknowns and sets up a variational problem to solve for the stages at each time step.:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      solver_parameters=params,
                      stage_type="dirk")
```

This logic is pretty self-explanatory. We use the `TimeStepper`'s `advance` method, which solves the variational problem to compute the Runge-Kutta stage values and then updates the solution.:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    print(float(t))
    t.assign(float(t) + float(dt))
```

Now, check the relative L^2 error:

```
print()
print(norm(u-uexact)/norm(uexact))
```

And report the solver statistics:

```
num_steps, _, num_lin_its = stepper.solver_stats()

print(f"{num_steps} steps taken")
print(f"{num_lin_its} linear iterations taken")
print(f"That's {num_lin_its / num_steps} per time step")
print(f"And {num_lin_its / num_steps / 3} average per stage")
```

and for Galerkin-in-Time:

3.11 Solving the Mixed Wave Equation: Energy conservation, Multigrid, Galerkin-in-Time

Let Ω be the unit square with boundary Γ. We write the wave equation as a first-order system of PDE:

$$u_t + \nabla p = 0$$
$$p_t + \nabla \cdot u = 0$$

together with homogeneous Dirichlet boundary conditions

$$p = 0 \quad \text{on } \Gamma$$

In this form, at each time, u is a vector-valued function in the Soboleve space $H(\mathrm{div})$ and *p* is a scalar-valued function. If we select appropriate test functions v and w, then we can arrive at the weak form

$$(u_t, v) - (p, \nabla \cdot v) = 0$$
$$(p_t, w) + (\nabla \cdot u, w) = 0$$

Note that in mixed formulations, the Dirichlet boundary condition is weakly enforced via integration by parts rather than strongly in the definition of the approximating space.

In this example, we will use the next-to-lowest order Raviart-Thomas elements for the velocity variable u and discontinuous piecewise linear polynomials for the scalar variable p.

Here is some typical Firedrake boilerplate and the construction of a simple mesh and the approximating spaces. We are going to use a multigrid preconditioner for each timestep, so we create a MeshHierarchy as well:

```
from firedrake import *
from irksome import Dt, MeshConstant, TimeStepper,␣
↪ContinuousPetrovGalerkinScheme

N = 10

base = UnitSquareMesh(N, N)
mh = MeshHierarchy(base, 2)
msh = mh[-1]
V = FunctionSpace(msh, "RT", 2)
W = FunctionSpace(msh, "DG", 1)
Z = V*W
```

Now we can build the initial condition, which has zero velocity and a sinusoidal displacement:

```
x, y = SpatialCoordinate(msh)
up0 = project(as_vector([0, 0, sin(pi*x)*sin(pi*y)]), Z)
u0, p0 = split(up0)
```

We build the variational form in UFL:

```
v, w = TestFunctions(Z)
F = inner(Dt(u0), v)*dx + inner(div(u0), w) * dx + inner(Dt(p0), w)*dx -␣
↪inner(p0, div(v)) * dx
```

Energy conservation is an important principle of the wave equation, and we can test how well the spatial discretization conserves energy by creating a UFL expression and evaluating it at each time step:

```
E = 0.5 * (inner(u0, u0)*dx + inner(p0, p0)*dx)
```

The time and time step variables:

```
MC = MeshConstant(msh)
t = MC.Constant(0.0)
dt = MC.Constant(1.0/N)
```

Here, we experiment with a multigrid preconditioner for the CG(2)-in-time discretization:

```
mgparams = {"mat_type": "aij",
            "snes_type": "ksponly",
            "ksp_type": "fgmres",
            "ksp_rtol": 1e-8,
            "pc_type": "mg",
            "mg_levels": {
                "ksp_type": "chebyshev",
                "ksp_max_it": 2,
                "ksp_convergence_test": "skip",
                "pc_type": "python",
                "pc_python_type": "firedrake.PatchPC",
                "patch": {
                    "pc_patch": {
                        "save_operators": True,
                        "partition_of_unity": False,
                        "construct_type": "star",
                        "construct_dim": 0,
                        "sub_mat_type": "seqdense",
                        "dense_inverse": True,
                        "precompute_element_tensors": None},
                    "sub": {
                        "ksp_type": "preonly",
                        "pc_type": "lu"}}},
            "mg_coarse": {
                "pc_type": "lu",
                "pc_factor_mat_solver_type": "mumps"}
            }
```

(continues on next page)

(continued from previous page)

```
scheme = ContinuousPetrovGalerkinScheme(2)
stepper = TimeStepper(F, scheme, t, dt, up0, solver_parameters=mgparams)
```

And, as with the heat equation, our time-stepping logic is quite simple. At easch time step, we print out the energy in the system:

```
print("Time    Energy")
print("==============")

while (float(t) < 1.0):
    if float(t) + float(dt) > 1.0:
        dt.assign(1.0 - float(t))

    stepper.advance()

    t.assign(float(t) + float(dt))
    print("{0:1.1e} {1:5e}".format(float(t), assemble(E)))
```

If all is well with the world, the energy will be nearly identical (up to roundoff error) at each time step because the Galerkin-in-time methods are symplectic and applied to a linear Hamiltonian system.

We can also confirm that the multigrid preconditioner is effective, by computing the average number of linear iterations per time-step:

```
(steps, nl_its, linear_its) = stepper.solver_stats()
print(f"The average number of multigrid iterations per time-step is {linear_
↪its/steps}.")
```

and for explicit schemes:

3.12 Solving the Mixed Wave Equation: Energy conservation and explicit RK

Let Ω be the unit square with boundary Γ. We write the wave equation as a first-order system of PDE:

$$u_t + \nabla p = 0$$
$$p_t + \nabla \cdot u = 0$$

together with homogeneous Dirichlet boundary conditions

$$p = 0 \quad \text{on } \Gamma$$

In this form, at each time, u is a vector-valued function in the Sobolev space $H(\mathrm{div})$ and *p* is a scalar-valued function. If we select appropriate test functions v and w, then we can arrive at the weak form

$$(u_t, v) - (p, \nabla \cdot v) = 0$$
$$(p_t, w) + (\nabla \cdot u, w) = 0$$

Note that in mixed formulations, the Dirichlet boundary condition is weakly enforced via integration by parts rather than strongly in the definition of the approximating space.

In this example, we will use the next-to-lowest order Raviart-Thomas elements for the velocity variable u and discontinuous piecewise linear polynomials for the scalar variable p.

Here is some typical Firedrake boilerplate and the construction of a simple mesh and the approximating spaces:

```
from firedrake import *
from irksome import Dt, MeshConstant, TimeStepper, PEPRK

N = 10

msh = UnitSquareMesh(N, N)
V = FunctionSpace(msh, "RT", 2)
W = FunctionSpace(msh, "DG", 1)
Z = V*W
```

Now we can build the initial condition, which has zero velocity and a sinusoidal displacement:

```
x, y = SpatialCoordinate(msh)
up0 = project(as_vector([0, 0, sin(pi*x)*sin(pi*y)]), Z)
u0, p0 = split(up0)
```

We build the variational form in UFL:

```
v, w = TestFunctions(Z)
F = inner(Dt(u0), v)*dx + inner(div(u0), w) * dx + inner(Dt(p0), w)*dx -␣
↪inner(p0, div(v)) * dx
```

Energy conservation is an important principle of the wave equation, and we can test how well the spatial discretization conserves energy by creating a UFL expression and evaluating it at each time step:

```
E = 0.5 * (inner(u0, u0)*dx + inner(p0, p0)*dx)
```

The time and time step variables:

```
MC = MeshConstant(msh)
t = MC.Constant(0.0)
dt = MC.Constant(0.2/N)
```

The PEP RK methods of de Leon, Ketcheson, and Ranoch offer explicit RK methods that preserve the energy up to a given order in step size. They have more stages than classical explicit methods but have much better energy conservation.:

```
butcher_tableau = PEPRK(4, 2, 5)
```

We'll use a simple iterative method to invert the mass matrix for each stage:

```
params = {"snes_type": "ksponly",
          "ksp_type": "cg",
```

(continues on next page)

(continued from previous page)

```
          "pc_type": "icc"}

stepper = TimeStepper(F, butcher_tableau, t, dt, up0,
                      stage_type="explicit",
                      solver_parameters=params)
```

And, as with the heat equation, our time-stepping logic is quite simple. At each time step, we print out the energy in the system:

```
print("Time    Energy")
print("==============")

while (float(t) < 1.0):
    if float(t) + float(dt) > 1.0:
        dt.assign(1.0 - float(t))

    stepper.advance()

    t.assign(float(t) + float(dt))
    print("{0:1.1e} {1:5e}".format(float(t), assemble(E)))
```

If all is well with the world, the energy will be nearly identical (up to roundoff error) at each time step because the PEP methods conserve energy to quite high order. The reader can compare this to the mixed wave demo using Gauss-Legendre methods (which exactly conserve energy up to roundoff and solver tolerances.

and for Nystrom methods for second-order-in-time equations:

3.13 Solving the Wave Equation with Irksome

Let's start with the simple wave equation on $\Omega = [0,1] \times [0,1]$, with boundary Γ:

$$\begin{aligned} u_{tt} - \Delta u &= 0 \\ u &= 0 \quad \text{on } \Gamma \end{aligned}$$

At each time t, the solution to this equation will be some function $u \in V$, for a suitable function space V.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We know have the variational problem of finding $u : [0,T] \to V$ such that

$$(u_{tt}, v) + (\nabla u, \nabla v) = 0$$

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import GaussLegendre, Dt, MeshConstant,␣
↪StageDerivativeNystromTimeStepper
```

We will create the Butcher tableau for a Gauss-Legendre Runge-Kutta method, which generalizes the implicit midpoint rule:

```
ns = 2
butcher_tableau = GaussLegendre(ns)
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 32

msh = UnitSquareMesh(N, N)
V = FunctionSpace(msh, "CG", 2)
```

We define variables to store the time step and current time value:

```
dt = Constant(2.0 / N)
t = Constant(0.0)
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step. For a second-order problem, we need an initial condition for the solution and its time derivative (in this case, taken to be zero):

```
x, y = SpatialCoordinate(msh)
uinit = sin(pi * x) * cos(pi * y)
u = Function(V)
u.interpolate(uinit)
ut = Function(V)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome. Here, the optional second argument indicates the number of derivatives, (defaulting to 1):

```
v = TestFunction(V)
F = inner(Dt(u, 2), v)*dx + inner(grad(u), grad(v))*dx
bc = DirichletBC(V, 0, "on_boundary")
```

Let's use simple solver options:

```
luparams = {"mat_type": "aij",
            "ksp_type": "preonly",
            "pc_type": "lu"}
```

Most of Irksome's magic happens in the *StageDerivativeNystromTimeStepper*. It takes our semidiscrete form *F* and the tableau and produces the variational form for computing the stage unknowns. Then, it sets up a variational problem to be solved for the stages at each time step.:

```
stepper = StageDerivativeNystromTimeStepper(
    F, butcher_tableau, t, dt, u, ut, bcs=bc,
    solver_parameters=luparams)
```

The system energy is an important quantity for the wave equation, and it should be conserved with our choice of time-stepping method:

```
E = 0.5 * (inner(ut, ut) * dx + inner(grad(u), grad(u)) * dx)
print(f"Initial energy: {assemble(E)}")
```

Then, we can loop over time steps, much like with 1st order systems:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    t.assign(float(t) + float(dt))
    print(f"Time: {float(t)}, Energy: {assemble(E)}")
```

3.14 Solving the Wave Equation with Irksome and an explicit Nystrom method

Let's start with the simple wave equation on $\Omega = [0,1] \times [0,1]$, with boundary Γ:

$$\begin{aligned} u_{tt} - \Delta u &= 0 \\ u &= 0 \quad \text{on } \Gamma \end{aligned}$$

At each time t, the solution to this equation will be some function $u \in V$, for a suitable function space V.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We know have the variational problem of finding $u : [0,T] \to V$ such that

$$(u_{tt}, v) + (\nabla u, \nabla v) = 0$$

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import Dt, MeshConstant, StageDerivativeNystromTimeStepper,␣
↪ClassicNystrom4Tableau
```

Here, we will use the "classic" Nystrom method, a 4-stage explicit time-stepper:

```
nystrom_tableau = ClassicNystrom4Tableau()
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 32

msh = UnitSquareMesh(N, N)
V = FunctionSpace(msh, "CG", 2)
```

We define variables to store the time step and current time value, noting that an explicit scheme requires a small timestep for stability:

```
dt = Constant(0.2 / N)
t = Constant(0.0)
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step. For a second-order problem, we need an initial condition for the solution and its time derivative (in this case, taken to be zero):

```
x, y = SpatialCoordinate(msh)
uinit = sin(pi * x) * cos(pi * y)
u = Function(V)
u.interpolate(uinit)
ut = Function(V)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome. Here, the optional second argument indicates the number of derivatives, (defaulting to 1):

```
v = TestFunction(V)
F = inner(Dt(u, 2), v)*dx + inner(grad(u), grad(v))*dx
bc = DirichletBC(V, 0, "on_boundary")
```

We're using an explicit scheme, so we only need to solve mass matrices in the update system. So, some good solver parameters here are to use a fieldsplit (so we only invert the diagonal blocks of the stage-coupled system) with incomplete Cholesky as a preconditioner for the mass matrices on the diagonal blocks:

```
mass_params = {"snes_type": "ksponly",
               "mat_type": "aij",
               "ksp_type": "preonly",
               "pc_type": "fieldsplit",
               "pc_fieldsplit_type": "multiplicative"}

per_field = {"ksp_type": "cg",
             "pc_type": "icc"}

for s in range(nystrom_tableau.num_stages):
    mass_params["fieldsplit_%s" % (s,)] = per_field
```

Most of Irksome's magic happens in the *StageDerivativeNystromTimeStepper*. It takes our semidiscrete form *F* and the tableau and produces the variational form for computing the stage unknowns. Then, it sets up a variational problem to be solved for the stages at each time step. Here, we use *dDAE* style boundary conditions, which impose boundary conditions on the stages to match those of u_t on the boundary, consistent with the underlying partitioned scheme:

```
stepper = StageDerivativeNystromTimeStepper(
    F, nystrom_tableau, t, dt, u, ut, bcs=bc, bc_type="dDAE",
    solver_parameters=mass_params)
```

The system energy is an important quantity for the wave equation. It won't be conserved with our choice of time-stepping method, but serves as a good diagnostic:

```
E = 0.5 * (inner(ut, ut) * dx + inner(grad(u), grad(u)) * dx)
print(f"Initial energy: {assemble(E)}")
```

Then, we can loop over time steps, much like with 1st order systems:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    t.assign(float(t) + float(dt))
    print(f"Time: {float(t)}, Energy: {assemble(E)}")
```

3.15 Solving the wave equation with monolithic multigrid

This reprise of the wave equation demo uses a monolithic multigrid algorithm to perform time advancement.

We consider the wave equation on $\Omega = [0,1] \times [0,1]$, with boundary Γ: giving rise to the weak form

$$(u_{tt}, v) + (\nabla u, \nabla v) = 0$$

We perform similar imports and setup as before:

```
from firedrake import *
from irksome import GaussLegendre, Dt, MeshConstant,␣
↪StageDerivativeNystromTimeStepper
butcher_tableau = GaussLegendre(2)
```

However, we need to set up a mesh hierarchy to enable geometric multigrid within Firedrake:

```
N = 4
nref = 3
base = UnitSquareMesh(N, N)
mh = MeshHierarchy(base, nref)
msh = mh[-1]
```

From here, setting up the function space, manufactured solution, etc, are just as for the regular wave equation demo:

```
V = FunctionSpace(msh, "CG", 2)

dt = Constant(2 / (N*2**nref))
t = Constant(0.0)

x, y = SpatialCoordinate(msh)
uinit = sin(pi * x) * cos(pi * y)
u = Function(V)
u.interpolate(uinit)
ut = Function(V)
```

(continues on next page)

(continued from previous page)

```
v = TestFunction(V)

F = inner(Dt(u, 2), v)*dx + inner(grad(u), grad(v))*dx
bc = DirichletBC(V, 0, "on_boundary")
```

And now for the solver parameters. Note that we are solving a block-wise system with all stages coupled together. This performs a monolithic multigrid with a stage-coupled vertex patch smoother:

```
mgparams = {"mat_type": "aij",
            "snes_type": "ksponly",
            "ksp_type": "gmres",
            "ksp_monitor_true_residual": None,
            "pc_type": "mg",
            "mg_levels": {
                "ksp_type": "chebyshev",
                "ksp_max_it": 1,
                "ksp_convergence_test": "skip",
                "pc_type": "python",
                "pc_python_type": "firedrake.ASMStarPC"},
            "mg_coarse": {
                "pc_type": "lu",
                "pc_factor_mat_solver_type": "mumps"}
            }
```

These solver parameters work just fine in the stepper.:

```
stepper = StageDerivativeNystromTimeStepper(
    F, butcher_tableau, t, dt, u, ut, bcs=bc,
    solver_parameters=mgparams)
```

The system energy is an important quantity for the wave equation, and it should be conserved with our choice of time-stepping method:

```
E = 0.5 * (inner(ut, ut) * dx + inner(grad(u), grad(u)) * dx)
print(f"Initial energy: {assemble(E)}")
```

And we can advance the solution in time in typical fashion:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    t.assign(float(t) + float(dt))
    print(f"Time: {float(t)}, Energy: {assemble(E)}")
```

3.16 Solving the telegraph equation with a stage-decoupled preconditioner

This demo solves a slightly more involved equation – the telegraph equation – and uses a stage-segregated preconditioner rather than direct method or monolithic multigrid.

We consider the telegraph equation on $\Omega = [0,1] \times [0,1]$, with boundary Γ: giving rise to the weak form

$$(u_{tt}, v) + (u_t, v) + (\nabla u, \nabla v) = 0$$

We perform similar imports and setup as before:

```
from firedrake import *
from irksome import GaussLegendre, Dt, MeshConstant,␣
↪StageDerivativeNystromTimeStepper
tableau = GaussLegendre(2)
```

We're going to use a stage-segregated preconditioner, and we have access to AMG for the fields. No need for a mesh hierarchy:

```
N = 32
msh = UnitSquareMesh(N, N)
```

From here, setting up the function space, manufactured solution, etc, are just as for the regular wave equation demo:

```
V = FunctionSpace(msh, "CG", 2)

dt = Constant(2 / N)
t = Constant(0.0)

x, y = SpatialCoordinate(msh)
uinit = sin(pi * x) * cos(pi * y)
u = Function(V)
u.interpolate(uinit)
ut = Function(V)

v = TestFunction(V)

F = inner(Dt(u, 2), v)*dx + inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx
bc = DirichletBC(V, 0, "on_boundary")
```

And now for the solver parameters. Note that we are solving a block-wise system with all stages coupled together. We will segregate those stages with the preconditioner from Clines/Howle/Long, and use hypre on the diagonal blocks:

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "gmres",
          "ksp_monitor": None,
          "pc_type": "python",
```

(continues on next page)

(continued from previous page)

```
            "pc_python_type": "irksome.ClinesLD",
            "aux": {
                "pc_type": "fieldsplit",
                "pc_fieldsplit_type": "multiplicative",
                "fieldsplit": {
                  "ksp_type": "preonly",
                  "pc_type": "hypre"
                }
            }}
```

These solver parameters work just fine in the stepper.:

```
stepper = StageDerivativeNystromTimeStepper(
    F, tableau, t, dt, u, ut, bcs=bc,
    solver_parameters=params)
```

The system energy is the same quantity as for the wave equation, but in the telegraph equation it decays exponentially over time instead of being conserved.:

```
E = 0.5 * (inner(ut, ut) * dx + inner(grad(u), grad(u)) * dx)
print(f"Initial energy: {assemble(E)}")
```

And we can advance the solution in time in typical fashion:

```
while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))
    stepper.advance()
    t.assign(float(t) + float(dt))
    print(f"Time: {float(t)}, Energy: {assemble(E)}")
```

and for bounds constraints:

3.17 Solving the Heat Equation with Bounds Constraints

In this demo we solve the simple heat equation with bounds constraints applied uniformly in time and space. Consider the simple heat equation on $\Omega = [0,1] \times [0,1]$, with boundary Γ:

$$\begin{aligned} u_t - \Delta u &= f \\ u &= g \quad \text{on } \Gamma \end{aligned}$$

for some known functions f and g. The solution will be some function $u \in V$, for a suitable function space V.

The weak form is found by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We then have the variational problem of finding $u : [0,T] \to V$ such that .. math:

```
(u_t, v) + (\nabla u, \nabla v) = (f, v)\quad \forall v \in V \textrm{ and }␣
↪t\in [0, T],
```

subject to the boundary condition $u = g$ on Γ. This demo uses particular choices of the functions f and g to be defined below.

The approach to bounds constraints below relies on the geometric properties of the Bernstein basis. In one dimension (on $[0, 1]$), the graph of the polynomial .. math:

```
p(x) = \sum_{i = 0}^n p_i b_i^n(x),
```

where $b_i^n(x)$ are Bernstein basis polynomials, lies in the convex-hull of the points .. math:

```
\left\{\left(\frac{i}{n}, p_i\right)\right\}_{i = 0}^n.
```

In particular, if the coefficients p_i lie in the interval $[m, M]$, then the output of $p(x)$ will also fall within this range. Similar results hold in higher dimensions. This property provides a straightforward approach to uniformly enforced bounds constraints in both space and time.

First, we must import firedrake and certain items from Irksome:

```
from firedrake import *
from irksome import Dt, MeshConstant, RadauIIA, TimeStepper,␣
↪BoundsConstrainedDirichletBC
```

Finally, numpy provides us with the upper bound of infinity:

```
import numpy as np
```

We first define the mesh and the necessary function space. We choose quadratic Bernstein polynomials to support bounds-constraints in space:

```
N = 32

msh = UnitSquareMesh(N, N)
V = FunctionSpace(msh, "Bernstein", 2)
```

In order to enforce bounds constraints in time, we must utilize a collocation method. In this demo, we will time-step using the L-stable, fully implicit, 2-stage RadauIIA Runge-Kutta method. The bounds will be passed as an argument to the `advance` method. We now define the Butcher Tableau and variables to store the time step, current time, and final time:

```
butcher_tableau = RadauIIA(2)

MC = MeshConstant(msh)
dt = MC.Constant(2 / N)
t = MC.Constant(0.0)
Tf = MC.Constant(1.0)
```

We will find an approximate solution at time $t = 1.0$ with and without enforcing a constraint on the lower bound. We will need the following pair of solver parameters:

```
lu_params = {
    "snes_type": "ksponly",
    "ksp_type": "preonly",
    "mat_type": "aij",
    "pc_type": "lu"
}
```

(continues on next page)

(continued from previous page)

```
vi_params = {
    "snes_type": "vinewtonrsls",
    "snes_max_it": 300,
    "snes_atol": 1.e-8,
    "ksp_type": "preonly",
    "mat_type": "aij",
    "pc_type": "lu",
}
```

We now define the right-hand side using the method of manufactured solutions:

```
x, y = SpatialCoordinate(msh)

uexact = 0.5 * exp(-t) * (1 + (tanh((0.1 - sqrt((x - 0.5) ** 2 + (y - 0.5) **␣
↪2)) / 0.015)))

rhs = Dt(uexact) - div(grad(uexact))
```

Note that the exact solution is uniformly positive in space and time. Using a manufactured solution, one usually interpolates or projects the exact solution at time $t = 0$ onto the approximation space to obtain the initial condition. Interpolation does not work with the Bernstein basis, and there is no guarantee that an interpolant or projection would satisfy the bounds constraints. To guarantee that the initial condition satisfies the bounds constraints, we solve a variational inequality:

```
v = TestFunction(V)
u_init = Function(V)

G = inner(u_init - uexact, v) * dx

nlvp = NonlinearVariationalProblem(G, u_init)
nlvs = NonlinearVariationalSolver(nlvp, solver_parameters=vi_params)

lb = Function(V)
ub = Function(V)

ub.assign(np.inf)
lb.assign(0.0)

nlvs.solve(bounds=(lb, ub))

u = Function(V)
u.assign(u_init)

u_c = Function(V)
u_c.assign(u_init)
```

`u` and `u_c` now hold a bounds-constrained approximation to the exact solution at $t = 0$. Note that *ub* = *None* is also supported and gets internally converted to what we have here.

We now construct semidiscrete variational problems for both the constrained and unconstrained

approximations using UFL notation and the `Dt` operator from Irksome:

```
v = TestFunction(V)

F = (inner(Dt(u), v) * dx + inner(grad(u), grad(v)) * dx - inner(rhs, v) * dx)

v_c = TestFunction(V)

F_c = (inner(Dt(u_c), v_c) * dx + inner(grad(u_c), grad(v_c)) * dx -␣
↪inner(rhs, v_c) * dx)
```

We use exact boundary conditions in both cases. When g is the trace of a function defined over the whole domain, Firedrake creates its own version of the boundary condition by either interpolating or projecting that function onto the finite element space and computing the trace of the result. To ensure the internal boundary condition satisfies the bounds constraints, we will pass the bounds to the `TimeStepper` below.

```
bc = DirichletBC(V, uexact, "on_boundary")
```

For the unconstrained approximation, we configure the `TimeStepper` in a familiar way:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc, solver_
↪parameters=lu_params)
```

We will enforce nonnegativity when finding the constrained approximation. We now set up the keyword database to configure an instance of `TimeStepper` for this task. We first specify, using the keyword `stage_type`, that we wish to use a stage-value formulation of the underlying collocation method. The keyword `basis_type` then allows us to change the basis of the collocation polynomial to the Bernstein basis. Having done this, we must specify a solver which is able to handle bounds constraints. In this example we solve a variational inequality using `vinewtonrsls` by passing `vi_params` as `solver_parameters` to the `TimeStepper`.

We set the bounds as follows (reusing those defined in the initial condition):

```
bounds = ('stage', lb, ub)
```

Internally, Firedrake will project the boundary condition expression into the entire space and match degrees of freedom on the boundary. This could introduce bounds violations. To ensure this does not happen, we can use a special kind of boundary condition that projects with bounds contraints.

```
bc = BoundsConstrainedDirichletBC(V, uexact, "on_boundary", (lb, ub), solver_
↪parameters=vi_params)

kwargs_c = {"bounds": bounds,
            "stage_type": "value",
            "basis_type": 'Bernstein',
            "solver_parameters": vi_params
        }

stepper_c = TimeStepper(F_c, butcher_tableau, t, dt, u_c, bcs=bc, **kwargs_c)
```

Note that if one does not set the `basis_type` to Bernstein, the standard basis will be used.

Solving for the Bernstein coefficients of the collocation polynomial we obtain uniform-in-time bounds constraints. If the standard basis is used, the bounds constraints are guaranteed at the Runge-Kutta stages and the discrete times, but not necessarily between them.

When using a stage-value formulation, passing `bounds` to the `TimeStepper` through the `advance` method will enforce the bounds constraints at the discrete stages and time levels (this results in uniformly enforced constraints when using the Bernstein basis).

We now advance both semidiscrete systems in the usual way. We add the bounds as an argument to the `advance` method for the constrained approximation.

In order to monitor our approximate solutions, we check the minimum value of each after every step in time. If an approximate solution violates the lower bound, we append a tuple to indicate the time and minimum value.

```
violations_for_unconstrained_method = []
violations_for_constrained_method = []

timestep = 0
while (float(t) < float(Tf)):

    if (float(t) + float(dt) > float(Tf)):
        dt.assign(float(Tf) - float(t))

    stepper.advance()
    stepper_c.advance()

    t.assign(float(t) + float(dt))
    timestep = timestep + 1

    min_value = min(u.dat.data)
    if min_value < 0:
        violations_for_unconstrained_method.append((float(t), timestep,␣
↪round(min_value, 3)))

    min_value_c = min(u_c.dat.data)
    if min_value_c < 0:
        violations_for_constrained_method.append((float(t), timestep,␣
↪round(min_value_c, 3)))

    print(float(t))
```

Finally, we print the relative L^2 error and the time and severity (if any) of constraint violations:

```
np.set_printoptions(legacy='1.25')

print()
print(f"Relative L^2 norm of the unconstrained solution: {norm(u - uexact) /␣
↪norm(uexact)}")
print(f"Relative L^2 norm of the constrained solution:   {norm(u_c - uexact) /
↪ norm(uexact)}")
print()
```

(continues on next page)

(continued from previous page)

```
print("List of constraint violations in the form (time, time step, minimum␣
↪value) for each approximation:")
print()
print(f"Unconstrained solution: {violations_for_unconstrained_method}")
print()
print(f"Constrained solution: {violations_for_constrained_method}")
```

and for adaptive IRK methods:

3.18 Solving the Heat Equation with Irksome

Let's start with the simple heat equation on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ:

$$\begin{aligned} u_t - \Delta u &= f \\ u &= 0 \quad \text{on } \Gamma \end{aligned}$$

for some known function f. At each time t, the solution to this equation will be some function $u \in V$, for a suitable function space V.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We know have the variational problem of finding $u : [0, T] \to V$ such that

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

This demo implements an example used by Solin with a particular choice of f given below

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import RadauIIA, Dt, MeshConstant, TimeStepper
```

We will create the Butcher tableau for the 3-stage RadauIIA Runge-Kutta method, which has an embedded scheme we can use for adaptivity:

```
butcher_tableau = RadauIIA(3)
ns = butcher_tableau.num_stages
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 100
x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)
V = FunctionSpace(msh, "CG", 1)
```

We define variables to store the time step, current time value, and final time:

```
MC = MeshConstant(msh)
dt = MC.Constant(10.0 / N)
t = MC.Constant(0.0)
Tf = MC.Constant(1.0)
```

This defines the right-hand side using the method of manufactured solutions:

```
x, y = SpatialCoordinate(msh)
S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step:

```
u = Function(V)
u.interpolate(uexact)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome:

```
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx
bc = DirichletBC(V, 0, "on_boundary")
```

Later demos will show how to use Firedrake's sophisticated interface to PETSc for efficient block solvers, but for now, we will solve the system with a direct method (Note: the matrix type needs to be explicitly set to aij if sparse direct factorization were used with a multi-stage method, as we wind up with a mixed problem that Firedrake will assemble into a PETSc MatNest otherwise):

```
luparams = {"mat_type": "aij",
            "ksp_type": "preonly",
            "pc_type": "lu"}
```

Most of Irksome's magic happens in the `TimeStepper`. It transforms our semidiscrete form *F* into a fully discrete form for the stage unknowns and sets up a variational problem to solve for the stages at each time step.

In this demo, we use an adaptive timestepper, selected by sending a dictionary of adaptive parameters, which tells Irksome to also compute an error estimate at each timestep, and use that estimate to adjust the timestep size. The adaptation depends on some given parameters, including a tolerance *tol* for for the error at each step, and *KI* and *KP* that set the so-called integration and performance gain for the estimate.:

```
adapt_params = {"tol":1e-3, "KI":1/15, "KP":0.13}
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      stage_type="deriv", solver_parameters=luparams,
                      adaptive_parameters = adapt_params)
```

This logic is pretty self-explanatory. We use the `TimeStepper`'s `advance` method, which solves the variational problem to compute the Runge-Kutta stage values and then updates the solution.

Here, in contrast to the non-adaptive case, we get an estimate of the error at each step (that we do not use here) and a new adaptive timestep size at each step. We use these to control integrating to a fixed final time, *Tf*. This exposes the *dt_max* data for `TimeStepper`, which puts a hard limit on the timestep size in the adaptive case.:

```
while (float(t) < float(Tf)):
    stepper.dt_max = float(Tf)-float(t)
    (adapt_error, adapt_dt) = stepper.advance()
    print(float(t))
    t.assign(float(t) + float(adapt_dt))
```

Finally, we print out the relative L^2 error:

```
print()
print(norm(u-uexact)/norm(uexact))
```

and for general multistep methods:

3.19 Solving the Heat Equation with a Multistep Method in Irksome

Consider the heat equation on $\Omega = [0, 10] \times [0, 10]$, with boundary Γ:

$$\begin{aligned} u_t - \Delta u &= f \\ u &= 0 \quad \text{on } \Gamma \end{aligned}$$

for some known function f. At each time t, the solution to this equation will be some function $u \in V$, for a suitable function space V.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We now have the variational problem of finding $u : [0, T] \to V$ such that

$$(u_t, v) + (\nabla u, \nabla v) = (f, v)$$

This demo implements an example used by Solin with a particular choice of f given below

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import Dt, TimeStepper, BDF
```

We will use the 3-step Backward Difference Formula:

```
method = BDF(3)
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 100
x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)
V = FunctionSpace(msh, "CG", 1)
```

We define variables to store the time step and current time value:

```
dt = Constant(5.0 / N)
t = Constant(0.0)
```

This defines the right-hand side using the method of manufactured solutions:

```
x, y = SpatialCoordinate(msh)
S = Constant(2.0)
C = Constant(1000.0)
B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
rhs = Dt(uexact) - div(grad(uexact))
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step:

```
u = Function(V)
u.interpolate(uexact)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome:

```
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx
bc = DirichletBC(V, 0, "on_boundary")
```

We'll use a basic solver for this demo:

```
luparams = {"mat_type": "aij",
            "ksp_type": "preonly",
            "pc_type": "lu"}
```

Irksome's `TimeStepper` automates the transformation of our semidiscrete form *F* into a fully discrete form for the next approximate value and sets up a variational problem to solve at each time step.

An s-step multistep method requires starting values $u^0, \ldots, u^{s-1}$ in order to begin the approximation. Irksome provides two ways to set these values. If you wish to set the starting values manually, you can create a `TimeStepper` as shown below, but with no *startup_parameters*. You can then access the list `TimeStepper.us` which contains the starting approximations, assign the desired starting values, and advance *t* by hand. On the other hand, if you wish to use a method

which Irksome supports to obtain these starting values, then Irksome allows this process to be completed automatically.

We'll showcase both methods. The automatic process requires defining a `dict` of keyword arguments used to setup a `TimeStepper` representing a single-step method. This `TimeStepper` is then used to compute the initial approximations needed to start the method. We'll import RadauIIA and use the backward Euler method to obtain our starting values. Formally, the backward Euler method is only first order accurate, and we wish to use it obtain the starting values for the third-order accurate BDF(3) method. A crude way to increase the accuracy of the starting values is to use a smaller timestep for the startup procedure. Here, we use timesteps of size $\Delta t/8$ for the backward Euler method which is accessible through the keyword *num_startup_steps*. Setting *num_startup_steps = 8* will instruct the startup `TimeStepper` to use 8 substeps to compute each starting value. The keyword *stepper_kwargs* allows for easy customization of the startup `TimeStepper`:

```
from irksome import RadauIIA

startup_stepper_kwargs = {'stage_type': 'value',
                          'solver_parameters': luparams}

startup_parameters = {'tableau': RadauIIA(1),
                      'num_startup_steps': 8,
                      'stepper_kwargs': startup_stepper_kwargs}
```

We can then set up the `TimeStepper` as follows:

```
stepper = TimeStepper(F, method, t, dt, u, bcs=bc,
                              solver_parameters=luparams,
                              startup_parameters=startup_parameters)
```

Note that the creation of a `TimeStepper` configured for a multistep method with parameters for the startup procedure will not automatically solve for the required starting values. One must call the `TimeSteppers`'s `startup` method, which will internally construct the single-step `TimeStepper` and solve for the required starting values. It is the user's responsibility to advance *t* to *t + (s-1)*dt*. Unless more refined control is needed, this value is available as `TimeStepper.startup_t`:

```
stepper.startup()
t.assign(stepper.startup_t)
```

If you would rather set the starting values by hand (for instance, in this case the exact solution is available) the process is straightforward. We'll first reset *t* so that everything is consistent, and then set the starting values by interpolating the exact solution.:

```
t.assign(0.0)
stepper.us[0].interpolate(uexact)
t.assign(t + dt)
stepper.us[1].interpolate(uexact)
t.assign(t + dt)
stepper.us[2].interpolate(uexact)
```

This remaining logic is pretty self-explanatory. We use the `TimeStepper`'s `advance` method, which solves the variational problem to compute the next approximate value and updates the

solution:

```
while (float(t) < 1.0):
  if (float(t) + float(dt) > 1.0):
      dt.assign(1.0 - float(t))
  stepper.advance()
  t.assign(float(t) + float(dt))
  print(float(t))
```

Finally, check the relative L^2 error:

```
print()
print(norm(u-uexact)/norm(uexact))
```

Or check out two IMEX-type methods for the monodomain equations:

3.20 Solving monodomain equations with Fitzhugh-Nagumo reaction and an IMEX method

We're solving monodomain (reaction-diffusion) with a particular reaction term. The basic form of the equation is:

$$\chi\left(C_m u_t + I_{ion}(u)\right) = \nabla \cdot \sigma \nabla u$$

where u is the membrane potential, σ is the conductivity tensor, C_m is the specific capacitance of the cell membrane, and χ is the surface area to volume ratio. The term I_{ion} is current due to ionic flows through channels in the cell membranes, and may couple to a complicated reaction network. In our case, we take the relatively simple model due to Fitzhugh and Nagumo. Here, we have a separate concentration variable c satisfying the reaction equation:

$$c_t = \epsilon(u + \beta - \gamma c)$$

for certain positive parameters β and γ, and the current takes the form of:

$$I_{ion}(u, c) = \frac{1}{\epsilon}\left(u - \frac{u^3}{3} - c\right)$$

so that we have an overall system of two equations. One of them is linear but stiff/diffusive, and the other is nonstiff but nonlinear. This combination makes the system a good candidate for additive partitioning/IMEX-type methods.

We start with standard Firedrake/Irksome imports:

```
import copy

from firedrake import (And, Constant, VTKFile, Function, FunctionSpace,
                       RectangleMesh, SpatialCoordinate, TestFunctions,
                       as_matrix, conditional, dx, grad, inner, split)
from irksome import Dt, MeshConstant, RadauIIA, TimeStepper
```

And we set up the mesh and function space. Note this demo uses serendipity elements, but could just as easily use Lagrange on quads or triangles.:

```
mesh = RectangleMesh(20, 20, 70, 70, quadrilateral=True)
polyOrder = 2

V = FunctionSpace(mesh, "S", polyOrder)
Z = V * V

x, y = SpatialCoordinate(mesh)
MC = MeshConstant(mesh)
dt = MC.Constant(0.05)
t = MC.Constant(0.0)
```

Specify the physical constants and initial conditions:

```
eps = Constant(0.1)
beta = Constant(1.0)
gamma = Constant(0.5)

chi = Constant(1.0)
capacitance = Constant(1.0)

sigma1 = sigma2 = 1.0
sigma = as_matrix([[sigma1, 0.0], [0.0, sigma2]])


initial_potential = conditional(x < 3.5, Constant(2.0), Constant(-1.28791))
initial_cell = conditional(And(And(31 <= x, x < 39), And(0 <= y, y < 35)),
                           Constant(2.0), Constant(-0.5758))


uu = Function(Z)
vu, vc = TestFunctions(Z)
uu.sub(0).interpolate(initial_potential)
uu.sub(1).interpolate(initial_cell)

(u, c) = split(uu)
```

This sets up the Butcher tableau. All of our IMEX-type methods are based on a RadauIIA method for the implicit part. We use a two-stage method.:

```
butcher_tableau = RadauIIA(2)
ns = butcher_tableau.num_stages
```

To access an IMEX method, we need to separately specify the implicit and explicit parts of the operator. The part to be handled implicitly is taken to contain the time derivatives as well:

```
F1 = (inner(chi * capacitance * Dt(u), vu)*dx
      + inner(grad(u), sigma * grad(vu))*dx
      + inner(Dt(c), vc)*dx - inner(eps * u, vc)*dx
      - inner(beta * eps, vc)*dx + inner(gamma * eps * c, vc)*dx)
```

This is the part to be additively partitioned and handled explicitly:

```
F2 = inner((chi/eps) * (-u + (u**3 / 3) + c), vu)*dx
```

If we wanted to use a fully implicit method, we would just take F = F1 + F2. Now, set up solver parameters. For this problem, the Rana preconditioner applied with a multiplicative field split works very well.:

```
params = {"snes_type": "ksponly",
          "ksp_rtol": 1.e-8,
          "ksp_monitor": None,
          "mat_type": "matfree",
          "ksp_type": "fgmres",
          "pc_type": "python",
          "pc_python_type": "irksome.RanaLD",
          "aux": {
              "pc_type": "fieldsplit",
              "pc_fieldsplit_type": "multiplicative"}}
```

Each diagonal block to be solved is itself a block-coupled system, with a diffusive block and a mass matrix on the diagonal. Hence, we further fieldsplit each diagonal block, using algebraic multigrid for the diffusive block and incomplete Cholesky for the mass matrix.:

```
per_stage = {
    "ksp_type": "preonly",
    "pc_type": "fieldsplit",
    "pc_fieldsplit_type": "additive",
    "fieldsplit_0": {
        "ksp_type": "preonly",
        "pc_type": "gamg",
    },
    "fieldsplit_1": {
        "ksp_type": "preonly",
        "pc_type": "icc",
    }}
```

This bit of mess specifies how the overall system is split up. Each stage corresponds to a pair of fields (potential and concentration):

```
for s in range(ns):
    params["aux"][f"pc_fieldsplit_{s}_fields"] = f"{2*s},{2*s+1}"
    params["aux"][f"fieldsplit_{s}"] = per_stage
```

The partitioned IMEX methods also provide an "iterator" that is used both to start the method (filling in the stage values between the initial condition and first time step taken) and can also be used after a time step to improve the accuracy/stability of the solution. If the iterator "works", applying it a large number of times will tend to produce the solution to a fully implicit RadauIIA method. In practice, we can sometimes get away with applying the iterator to a looser tolerance, but we otherwise use the same method as for the propagator.:

```
itparams = copy.deepcopy(params)
itparams["ksp_rtol"] = 1.e-4
```

Now, we access the IMEX method via the *TimeStepper* as with other methods. Note that we

specify somewhat different kwargs, needing to specify the implicit and explicit parts separately as well as separate solver options for propagator and iterator.:

```
stepper = TimeStepper(F1, butcher_tableau, t, dt, uu,
                      stage_type="imex",
                      prop_solver_parameters=params,
                      it_solver_parameters=itparams,
                      Fexp=F2,
                      num_its_initial=5,
                      num_its_per_step=3)

uFinal, cFinal = uu.subfunctions
outfile1 = VTKFile("FHN_results/FHN_2d_u.pvd")
outfile2 = VTKFile("FHN_results/FHN_2d_c.pvd")
outfile1.write(uFinal, time=0)
outfile2.write(cFinal, time=0)

for j in range(12):
    print(f"{float(t)}")
    stepper.advance()
    t.assign(float(t) + float(dt))

    if (j % 5 == 0):
        outfile1.write(uFinal, time=j * float(dt))
        outfile2.write(cFinal, time=j * float(dt))

nsteps, nprop, nit, nnonlinprop, nlinprop, nnonlinit, nlinit = stepper.solver_
↪stats()
print(f"Time steps taken: {nsteps}")
print(f"  {nprop} propagator steps")
print(f"  {nit} iterator steps")
print(f"  {nnonlinprop} nonlinear steps in propagators")
print(f"  {nlinprop} linear steps in propagators")
print(f"  {nnonlinit} nonlinear steps in iterators")
print(f"  {nlinit} linear steps in iterators")
```

3.21 Solving monodomain equations with Fitzhugh-Nagumo reaction and a DIRK-IMEX method

We're solving monodomain (reaction-diffusion) with a particular reaction term. The basic form of the equation is:

$$\chi \left(C_m u_t + I_{ion}(u) \right) = \nabla \cdot \sigma \nabla u$$

where u is the membrane potential, σ is the conductivity tensor, C_m is the specific capacitance of the cell membrane, and χ is the surface area to volume ratio. The term I_{ion} is current due to ionic flows through channels in the cell membranes, and may couple to a complicated reaction network. In our case, we take the relatively simple model due to Fitzhugh and Nagumo. Here, we have a separate concentration variable c satisfying the reaction equation:

$$c_t = \epsilon(u + \beta - \gamma c)$$

for certain positive parameters β and γ, and the current takes the form of:

$$I_{ion}(u, c) = \frac{1}{\epsilon}\left(u - \frac{u^3}{3} - c\right)$$

so that we have an overall system of two equations. One of them is linear but stiff/diffusive, and the other is nonstiff but nonlinear. This combination makes the system a good candidate for IMEX-type methods.

We start with standard Firedrake/Irksome imports:

```
import copy

from firedrake import (And, Constant, VTKFile, Function, FunctionSpace,
                       RectangleMesh, SpatialCoordinate, TestFunctions,
                       as_matrix, conditional, dx, grad, inner, split)
from irksome import Dt, MeshConstant, TimeStepper, ARS_DIRK_IMEX
from irksome.labeling import explicit
```

And we set up the mesh and function space.:

```
mesh = RectangleMesh(20, 20, 70, 70, quadrilateral=True)
polyOrder = 2

V = FunctionSpace(mesh, "CG", 2)
Z = V * V

x, y = SpatialCoordinate(mesh)
MC = MeshConstant(mesh)
dt = MC.Constant(0.05)
t = MC.Constant(0.0)
```

Specify the physical constants and initial conditions:

```
eps = Constant(0.1)
beta = Constant(1.0)
gamma = Constant(0.5)

chi = Constant(1.0)
capacitance = Constant(1.0)

sigma1 = sigma2 = 1.0
sigma = as_matrix([[sigma1, 0.0], [0.0, sigma2]])


initial_potential = conditional(x < 3.5, Constant(2.0), Constant(-1.28791))
initial_cell = conditional(And(And(31 <= x, x < 39), And(0 <= y, y < 35)),
                           Constant(2.0), Constant(-0.5758))


uu = Function(Z)
vu, vc = TestFunctions(Z)
uu.sub(0).interpolate(initial_potential)
```

(continues on next page)

(continued from previous page)

```
uu.sub(1).interpolate(initial_cell)

(u, c) = split(uu)
```

This sets up the Butcher tableau. Here, we use the DIRK-IMEX methods proposed by Ascher, Ruuth, and Spiteri in their 1997 Applied Numerical Mathematics paper. For this case, We use a four-stage method.:

```
butcher_tableau = ARS_DIRK_IMEX(4, 4, 3)
ns = butcher_tableau.num_stages
```

To access an IMEX method, we need to separately specify the implicit and explicit parts of the operator. The part to be handled implicitly is taken to contain the time derivatives as well:

```
F1 = (inner(chi * capacitance * Dt(u), vu)*dx
      + inner(grad(u), sigma * grad(vu))*dx
      + inner(Dt(c), vc)*dx - inner(eps * u, vc)*dx
      - inner(beta * eps, vc)*dx + inner(gamma * eps * c, vc)*dx)
```

This is the part to be handled explicitly.:

```
F2 = inner((chi/eps) * (-u + (u**3 / 3) + c), vu)*dx
```

If we wanted to use a fully implicit method, we would just take F = F1 + F2. Instead, we use a label:

```
F = F1 + explicit(F2)
```

Now, set up solver parameters. Since we're using a DIRK-IMEX scheme, we can specify only parameters for each stage. We use an additive Schwarz (fieldsplit) method that applies AMG to the potential block and incomplete Cholesky to the cell block independently for each stage:

```
params = {"snes_type": "ksponly",
          "ksp_monitor": None,
          "mat_type": "aij",
          "ksp_type": "fgmres",
          "pc_type": "fieldsplit",
          "pc_fieldsplit_type": "additive",
          "fieldsplit_0": {
              "ksp_type": "preonly",
              "pc_type": "gamg",
          },
          "fieldsplit_1": {
              "ksp_type": "preonly",
              "pc_type": "icc",
          }}
```

The DIRK-IMEX schemes also require a mass-matrix solver. Here, we just use an incomplete Cholesky preconditioner for CG on the coupled system, which works fine.:

```
mass_params = {"snes_type": "ksponly",
               "ksp_rtol": 1.e-8,
               "ksp_monitor": None,
               "mat_type": "aij",
               "ksp_type": "cg",
               "pc_type": "icc",
              }
```

Now, we access the IMEX method via the *TimeStepper* as with other methods. Note that we specify somewhat different kwargs, needing to specify the implicit and explicit parts separately as well as separate solver options for the implicit and mass solvers.:

```
stepper = TimeStepper(F1, butcher_tableau, t, dt, uu,
                      stage_type="dirkimex",
                      solver_parameters=params,
                      mass_parameters=mass_params,
                      Fexp=F2)

uFinal, cFinal = uu.subfunctions
outfile1 = VTKFile("FHN_results/FHN_2d_u.pvd")
outfile2 = VTKFile("FHN_results/FHN_2d_c.pvd")
outfile1.write(uFinal, time=0)
outfile2.write(cFinal, time=0)

for j in range(12):
    print(f"{float(t)}")
    stepper.advance()
    t.assign(float(t) + float(dt))

    if (j % 5 == 0):
        outfile1.write(uFinal, time=j * float(dt))
        outfile2.write(cFinal, time=j * float(dt))
```

We can print out some solver statistics here. We expect one implicit solve per stage per timestep, and that's what we see with the four-stage method. For this Butcher Tableau, we can avoid computing the final explicit stage (since it's coefficient in the next stage reconstruction is zero), so we see the same number of mass solves.:

```
nsteps, n_nonlin, n_lin, n_nonlin_mass, n_lin_mass = stepper.solver_stats()
print(f"Time steps taken: {nsteps}")
print(f"  {n_nonlin} nonlinear steps in implicit stage solves (should be
↪{nsteps*ns})")
print(f"  {n_lin} linear steps in implicit stage solves")
print(f"  {n_nonlin_mass} nonlinear steps in mass solves (should be
↪{nsteps*ns})")
print(f"  {n_lin_mass} linear steps in mass solves")
```

3.22 Advanced demos

There's also an example solving a Sobolev-type equation with symplectic RK methods:

3.22.1 Solving the Benjamin-Bona-Mahony equation

This demo solves the Benjamin-Bona-Mahony equation:

$$u_t - u_{txx} + u_x + uu_x = 0$$

typically posed on $\mathbb{R}$ or a bounded interval with periodic boundaries.

It is interesting because it is nonlinear (uu_x) and also a Sobolev-type equation, with spatial derivatives acting on a time derivative. We can obtain a weak form in the standard way:

$$(u_t, v) + (u_{tx}, v_x) + (u_x, v) + (uu_x, v) = 0$$

BBM is known to have a Hamiltonian structure, and there are three canonical polynomial invariants:

$$I_1 = \int u\,dx$$
$$I_2 = \int u^2 + (u_x)^2\,dx$$
$$I_3 = \int \tfrac{1}{2}u^2 + \tfrac{1}{6}u^3\,dx$$

We are mainly interested in accuracy and in conserving these quantities reasonably well.

Firedrake imports:

```
from firedrake import *
from irksome import Dt, GaussLegendre, TimeStepper
```

This function seems to be left out of UFL, but solitary wave solutions for BBM need it:

```
def sech(x):
    return 2 / (exp(x) + exp(-x))
```

Set up problem parameters, etc:

```
N = 1000
L = 100
h = L / N
msh = PeriodicIntervalMesh(N, L)

t = Constant(0)
dt = Constant(10*h)

x, = SpatialCoordinate(msh)
```

Here is the true solution, which is right-moving solitary wave (but not a soliton):

```
c = Constant(0.5)

center = 30.0
delta = -c * center

uexact = 3 * c**2 / (1-c**2) * sech(0.5 * (c * x - c * t / (1 - c ** 2) +␣
↪delta))**2
```

Create the approximating space and project true solution:

```
V = FunctionSpace(msh, "CG", 1)
u = project(uexact, V)
v = TestFunction(V)
```

The symplectic Gauss-Legendre methods are of interest here:

```
butcher_tableau = GaussLegendre(2)

F = (inner(Dt(u), v) * dx
     + inner((Dt(u)).dx(0), v.dx(0)) * dx
     + inner(u.dx(0), v) * dx
     + inner(u * u.dx(0), v) * dx)
```

For a 1d problem, we don't worry much about efficient solvers.:

```
params = {"mat_type": "aij",
          "ksp_type": "preonly",
          "pc_type": "lu"}

stepper = TimeStepper(F, butcher_tableau, t, dt, u,
                      solver_parameters=params)
```

UFL for the mathematical invariants and containers to track them over time:

```
I1 = u * dx
I2 = (u**2 + (u.dx(0))**2) * dx
I3 = (u**2 / 2 + u**3 / 6) * dx
functionals = (I1, I2, I3)
invariants = [tuple(map(assemble, functionals))]
```

Time-stepping loop, keeping track of I values:

```
tfinal = 18.0
while (float(t) < tfinal):
    if float(t) + float(dt) > tfinal:
        dt.assign(tfinal - float(t))
    stepper.advance()

    invariants.append(tuple(map(assemble, functionals)))

    print('%.15f %.15f %.15f %.15f' % (float(t), *invariants[-1]))
```

(continues on next page)

(continued from previous page)

```
    t.assign(float(t) + float(dt))

print(errornorm(uexact, u) / norm(uexact))
```

and with a Galerkin-in-Time approach, in standard form or with auxiliary variables for alternate conservation properties:

3.22.2 Solving the Benjamin-Bona-Mahony equation with Galerkin-in-Time

This demo solves the Benjamin-Bona-Mahony equation:

$$u_t - u_{txx} + u_x + uu_x = 0$$

typically posed on $\mathbb{R}$ or a bounded interval with periodic boundaries.

It is interesting because it is nonlinear (uu_x) and also a Sobolev-type equation, with spatial derivatives acting on a time derivative. We can obtain a weak form in the standard way:

$$(u_t, v) + (u_{tx}, v_x) + (u_x, v) + (uu_x, v) = 0$$

BBM is known to have a Hamiltonian structure, and there are three canonical polynomial invariants:

$$I_1 = \int u \,\mathrm{d}x$$
$$I_2 = \int u^2 + (u_x)^2 \,\mathrm{d}x$$
$$I_3 = \int \frac{u^2}{2} + \frac{u^3}{6} \,\mathrm{d}x$$

We are mainly interested in accuracy and in conserving these quantities reasonably well.

Firedrake imports:

```
from firedrake import *
from irksome import Dt, TimeStepper, ContinuousPetrovGalerkinScheme
```

This function seems to be left out of UFL, but solitary wave solutions for BBM need it:

```
def sech(x):
    return 2 / (exp(x) + exp(-x))
```

Set up problem parameters, etc:

```
N = 1000
L = 100
h = L / N
msh = PeriodicIntervalMesh(N, L)

t = Constant(0)
dt = Constant(10*h)

x, = SpatialCoordinate(msh)
```

Here is the true solution, which is right-moving solitary wave (but not a soliton):

```
c = Constant(0.5)

center = 30.0
delta = -c * center

uexact = 3 * c**2 / (1-c**2) * sech(0.5 * (c * x - c * t / (1 - c ** 2) +␣
↪delta))**2
```

Create the approximating space and project true solution:

```
V = FunctionSpace(msh, "CG", 1)
u = project(uexact, V)
v = TestFunction(V)

F = (inner(Dt(u), v) * dx
     + inner((Dt(u)).dx(0), v.dx(0)) * dx
     + inner(u.dx(0), v) * dx
     + inner(u * u.dx(0), v) * dx)
```

For a 1d problem, we don't worry much about efficient solvers.:

```
params = {"mat_type": "aij",
          "ksp_type": "preonly",
          "pc_type": "lu"}
```

The Galerkin-in-Time approach should have symplectic properties:

```
scheme = ContinuousPetrovGalerkinScheme(2)
stepper = TimeStepper(F, scheme, t, dt, u, solver_parameters=params)
```

UFL for the mathematical invariants and containers to track them over time:

```
I1 = u * dx
I2 = (u**2 + (u.dx(0))**2) * dx
I3 = (u**2 / 2 + u**3 / 6) * dx
functionals = (I1, I2, I3)
invariants = [tuple(map(assemble, functionals))]
```

Time-stepping loop, keeping track of I values:

```
tfinal = 18.0
while (float(t) < tfinal):
    if float(t) + float(dt) > tfinal:
        dt.assign(tfinal - float(t))
    stepper.advance()

    invariants.append(tuple(map(assemble, functionals)))

    print('%.15f %.15f %.15f %.15f' % (float(t), *invariants[-1]))
    t.assign(float(t) + float(dt))
```

(continues on next page)

(continued from previous page)

```
print(errornorm(uexact, u) / norm(uexact))
```

3.22.3 Hamiltonian-structure-preserving implementation of the Benjamin-Bona-Mahoney equation

This demo solves the Benjamin-Bona-Mahony equation:

$$u_t - u_{txx} + u_x + uu_x = 0$$

posed on a bounded interval with periodic boundaries.

BBM is known to have a Hamiltonian structure, and there are several canonical polynomial invariants:

$$I_1 = \int u \,\mathrm{d}x$$
$$I_2 = \int u^2 + (u_x)^2 \,\mathrm{d}x$$
$$I_3 = \int \frac{u^2}{2} + \frac{u^3}{6} \,\mathrm{d}x$$

The BBM invariants are the total momentum I_1, the H^1-energy norm I_2, and the Hamiltonian I_3. The Hamiltonian variational formulation reads

$$(\partial_t u + \partial_x \tilde{w}_H, v)_{H^1} = 0$$
$$(\tilde{w}_H, v_H)_{H^1} = \langle \frac{\delta I_3}{\delta u}, v_H \rangle$$

For all test functions v, v_H in a suitable function space. The numerical scheme in this demo introduces the H^1-Riesz representative of the Fréchet derivative of the Hamiltonian $\frac{\delta I_3}{\delta u}$ as the auxiliary variable $\tilde{w}_H$.

Standard Gauss-Legendre and continuous Petrov-Galerkin (cPG) methods conserve the first two invariants exactly (up to roundoff and solver tolerances). They do quite well, but are inexact for the cubic one. Here, we consider the reformulation in Andrews and Farrell, "Enforcing conservation laws and dissipation inequalities numerically via auxiliary variables" (arXiv:2407.11904, to appear in SIAM J. Scientific Computing) that preserves the third invariant at the expense of the second. This method has an auxiliary variable in the system and requires a continuously differentiable spatial discretization (1D Hermite elements in this case). The time discretization puts the main unknown in a continuous space and the auxiliary variable in a discontinuous one. See equation (7.17) of Boris Andrews' thesis for the particular formulation.

Firedrake, Irksome, and other imports:

```
from firedrake import (Constant, Function, FunctionSpace,
    PeriodicIntervalMesh, SpatialCoordinate, TestFunction, TrialFunction,
    assemble, derivative, dx, errornorm, exp, grad, inner,
    interpolate, norm, plot, project, replace, solve, split
)
```

(continues on next page)

(continued from previous page)

```
from irksome import Dt, TimeStepper, ContinuousPetrovGalerkinScheme

import matplotlib.pyplot as plt
import numpy
```

Next, we define the domain and the exact solution

```
N = 8000
L = 100
h = L / N
msh = PeriodicIntervalMesh(N, L)
x, = SpatialCoordinate(msh)

t = Constant(0)
inv_dt = N // (10 * L)
tfinal = 18
Nt = tfinal * inv_dt
dt = Constant(tfinal / Nt)

c = Constant(0.5)
center = Constant(40.0)
delta = -c * center

def sech(x):
    return 2 / (exp(x) + exp(-x))

uexact = 3 * c**2 / (1-c**2) \
    * sech(0.5 * (c * x - c * t / (1 - c ** 2) + delta))**2
```

This sets up the function space for the unknown u and auxiliary variable $\tilde{w}_H$:

```
space_deg = 3
time_deg = 1

V = FunctionSpace(msh, "Hermite", space_deg)
Z = V * V
```

We next define the BBM invariants. Again, the discrete formulation preserves I_1 and I_3 up to solver tolerances and roundoff errors, but I_2 is preserved up to a bounded oscillation

```
def h1inner(u, v):
    return inner(u, v) + inner(grad(u), grad(v))

def I1(u):
    return u * dx

def I2(u):
    return h1inner(u, u) * dx

def I3(u):
```

(continues on next page)

(continued from previous page)

```
    return (u**2 / 2 + u**3 / 6) * dx
```

We project the initial condition on u, but we also need a consistent initial condition for the auxiliary variable. We need to find $\tilde{w}_H \in V$ such that

$$(\tilde{w}_H, v)_{H^1} = \langle \frac{\delta I_3}{\delta u}, v \rangle \text{ for all } v \in V$$

```
uwH = Function(Z)
u0, wH0 = uwH.subfunctions

v = TestFunction(V)
w = TrialFunction(V)
a = h1inner(w, v) * dx
dHdu = derivative(I3(u0), u0, v)

solve(a == h1inner(uexact, v)*dx, u0)
solve(a == dHdu, wH0)
```

Visualize the initial condition:

```
fig, axes = plt.subplots(1)
plot(Function(FunctionSpace(msh, "CG", 1)).interpolate(u0), axes=axes)
axes.set_title("Initial condition")
axes.set_xlabel("x")
axes.set_ylabel("u")
plt.savefig("bbm_init.png")
```

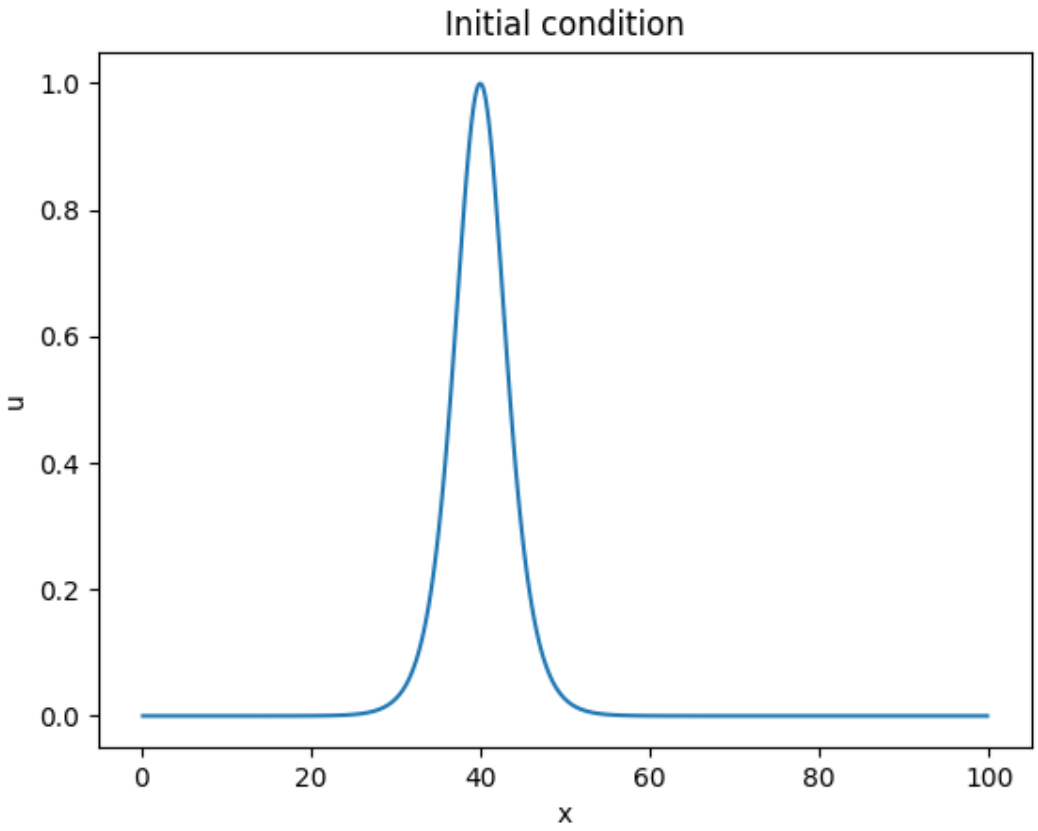
Set up the semidiscrete form as usual:

```
u, wH = split(uwH)
v, vH = split(TestFunction(Z))

F = h1inner(Dt(u) + wH.dx(0), v) * dx
F += h1inner(wH, vH) * dx - derivative(I3(u), u, vH)
```

Next, we set up the cPG scheme. We specify *quadrature_degree="auto"* to override the default quadrature of degree *2*time_deg-1*. This automatically estimates the quadrature degree for each individual term in *F*. The degree estimation algorithm will only be exact for polynomial nonlinearities, such as the cubic term in *I3*. For complicated nonlinearities, the estimated degree might be too large and result in very slow compilation and runtime. The quadrature degree might be optionally capped via *max_quadrature_degree* keyword argument.

```
scheme = ContinuousPetrovGalerkinScheme(time_deg, quadrature_degree="auto",
                                        max_quadrature_degree=3*time_deg-1)
```

This sets up the cPG time stepper. There are two fields in the unknown, we indicate that the second one is an auxiliary and hence to be discretized in the DG test space instead by passing the *aux_indices* keyword:




```
stepper = TimeStepper(F, scheme, t, dt, uwH, aux_indices=[1])
```

UFL expressions for the invariants, which we are going to track as we go through time steps:

```
times = [float(t)]
functionals = (I1(u), I2(u), I3(u))
invariants = [tuple(map(assemble, functionals))]
```

Do the time-stepping:

```
for _ in range(Nt):
    stepper.advance()

    invariants.append(tuple(map(assemble, functionals)))

    i1, i2, i3 = invariants[-1]
    t.assign(float(t) + float(dt))
    times.append(float(t))

    print(f'{float(t):.15f}, {i1:.15f}, {i2:.15f}, {i3:.15f}')
```

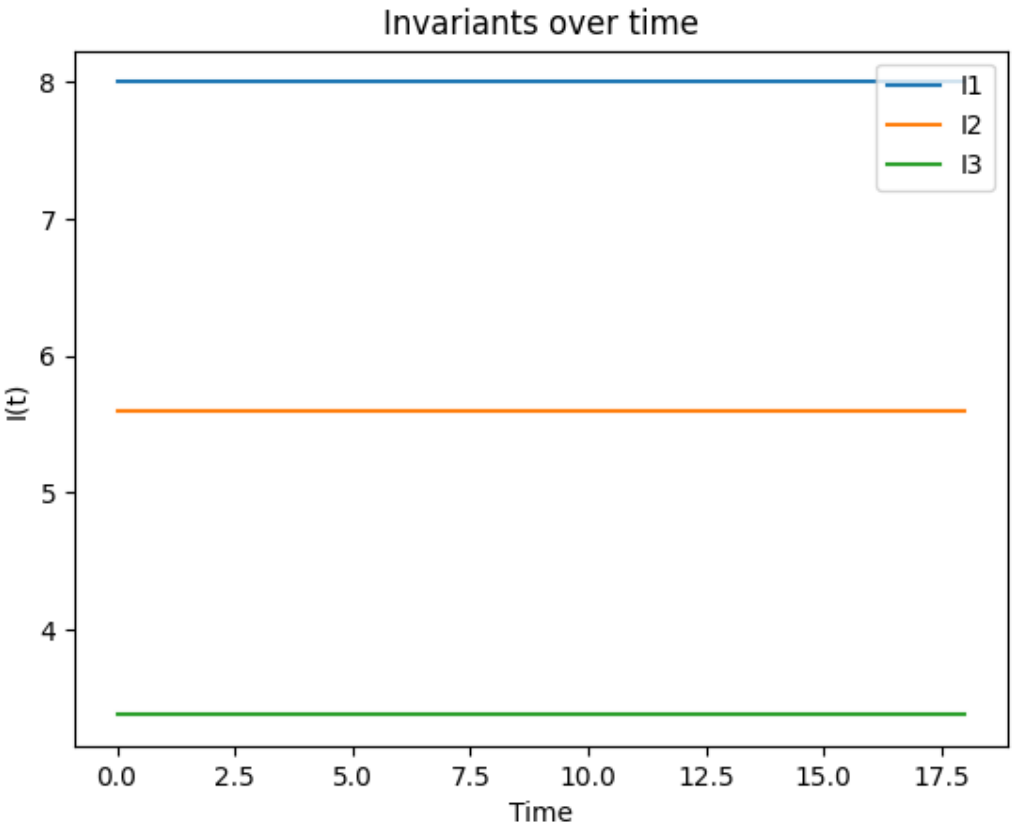
Visualize invariant preservation:

```
axes.clear()
```

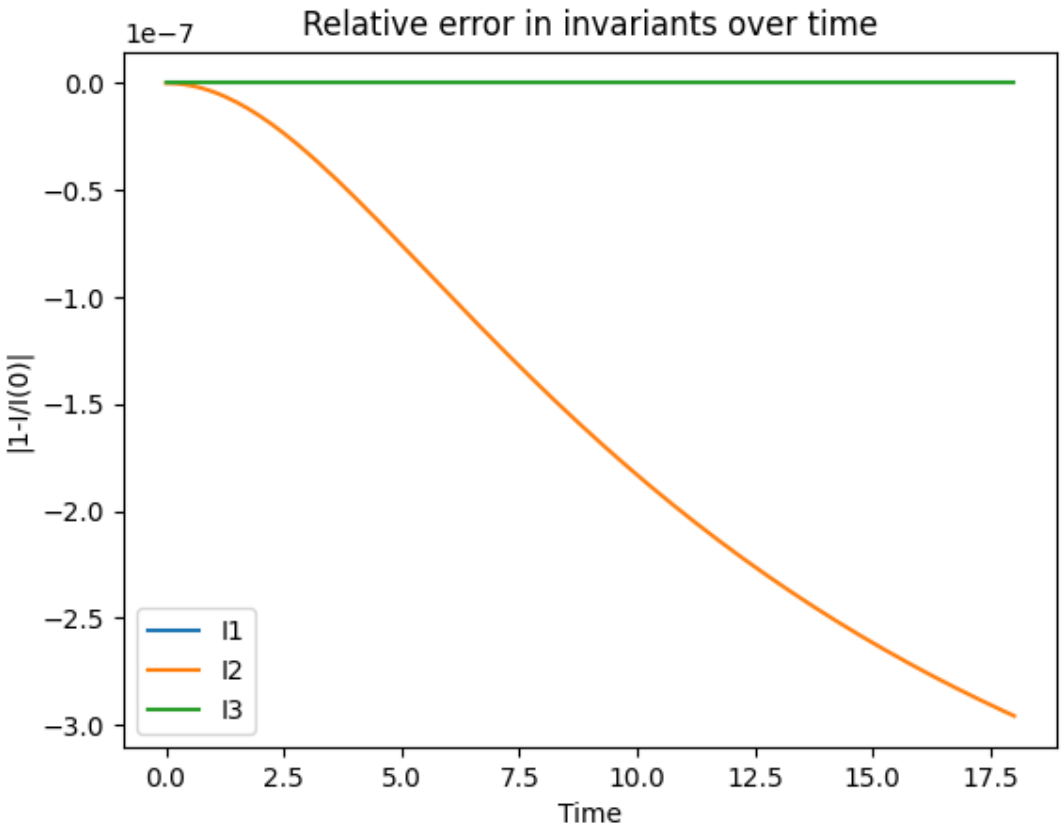
(continues on next page)

(continued from previous page)

```
invariants = numpy.array(invariants)

lbls = ("I1", "I2", "I3")

for i in (0, 1, 2):
    plt.plot(times, invariants[:, i], label=lbls[i])
axes.set_title("Invariants over time")
axes.set_xlabel("Time")
axes.set_ylabel("I(t)")
axes.legend()
plt.savefig("invariants.png")
axes.clear()

for i in (0, 1, 2):
    plt.plot(times, 1.0 - invariants[:, i]/invariants[0, i], label=lbls[i])
axes.set_title("Relative error in invariants over time")
axes.set_xlabel("Time")
axes.set_ylabel("|1-I/I(0)|")
axes.legend()
plt.savefig("invariant_errors.png")
```



Visualize the solution at final time step:




```
axes.clear()
plot(Function(FunctionSpace(msh, "CG", 1)).interpolate(u0), axes=axes)
axes.set_title(f"Solution at time {tfinal}")
axes.set_xlabel("x")
axes.set_ylabel("u")
plt.savefig("bbm_final.png")
```

Other structure-preserving methods have similar demos, such as for the dissipation law for the heat equation:

3.22.4 Preserving Structure for the Heat Equation

Here we'll consider the unforced heat equation on $\Omega = [0,1] \times [0,1]$, with boundary Γ:

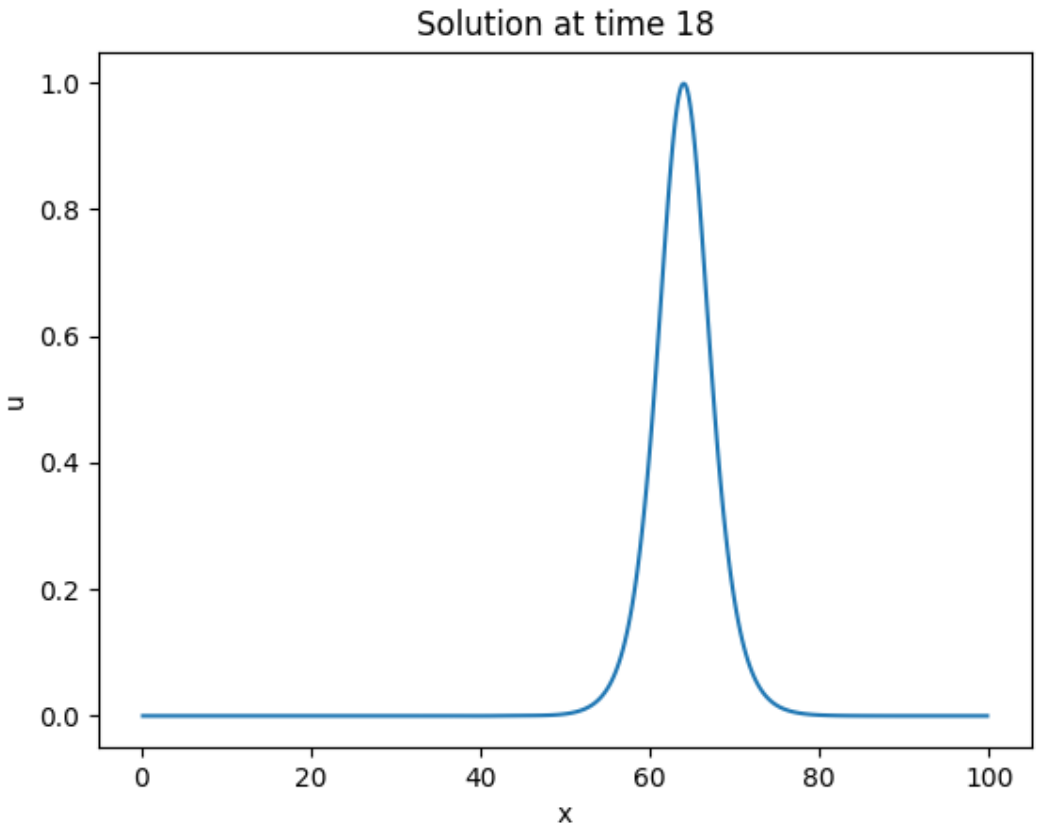
$$\begin{aligned} u_t - \Delta u &= 0 \quad \text{in } \Omega \\ u(x,0) &= u_0(x) \quad \text{in } \Omega \\ u(x,t) &= 0 \quad \text{on } \Gamma \end{aligned}$$

for some known function u_0.

We transform this into weak form by multiplying by an arbitrary test function $v \in V$ and integrating over Ω. We now have the variational problem of finding $u : [0,T] \to V$ such that

$$(u_t, v) + (\nabla u, \nabla v) = 0$$

with $u(x,0) = \Pi u_0(x)$ for some projection or interpolation operator, Π. Here, we focus on how some IRK discretizations preserve structure, given by an energy law. Choosing $v = u$ and



doing some calculus, the above variational form becomes

$$\frac{\partial}{\partial t}\frac{1}{2}(u,u) = -(\nabla u, \nabla u)$$

Using Crank-Nicolson as the integrator, it's possible to show a discrete form of this law holds, with

$$\frac{(u^{n+1},u^{n+1}) - (u^n,u^n)}{2} = -(\nabla u^{n+1/2}, \nabla u^{n+1/2})$$

Here, we'll ensure that this happens.

As usual, we need to import firedrake:

```
from firedrake import *
```

We will also need to import certain items from irksome:

```
from irksome import GaussLegendre, Dt, TimeStepper
```

We will create the Butcher tableau for the lowest-order Gauss-Legendre Runge-Kutta method, which is equivalent to Crank-Nicolson in this case:

```
butcher_tableau = GaussLegendre(1)
ns = butcher_tableau.num_stages
```

Now we define the mesh and piecewise linear approximating space in standard Firedrake fashion:

```
N = 100

msh = UnitSquareMesh(N, N)
V = FunctionSpace(msh, "CG", 1)
```

We define variables to store the time step and current time value:

```
dt = Constant(0.005)
t = Constant(0.0)
```

We use a manufactured solution that gives a zero right-hand side:

```
x, y = SpatialCoordinate(msh)
uexact = sin(pi*x)*sin(2*pi*y)*exp(-5*pi**2*t)
```

We define the initial condition for the fully discrete problem, which will get overwritten at each time step:

```
u = Function(V)
u.interpolate(uexact)
```

Now, we will define the semidiscrete variational problem using standard UFL notation, augmented by the `Dt` operator from Irksome:

```
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx
bc = DirichletBC(V, 0, "on_boundary")
```

Other demos show how to use Firedrake's sophisticated interface to PETSc for efficient block solvers, so we will solve the system with a direct method:

```
luparams = {"mat_type": "aij",
            "ksp_type": "preonly",
            "pc_type": "lu"}
```

We use the midpoint of each interval as a sample point:

```
sample_point = [0.5]
```

This gets passed into Irksome's `TimeStepper` as a keyword argument.:

```
stepper = TimeStepper(F, butcher_tableau, t, dt, u, bcs=bc,
                      solver_parameters=luparams,
                      sample_points=sample_point)
```

From here, we use the `TimeStepper`'s `advance` method to solve the variational problem to compute the Runge-Kutta stage values and update the solution. To verify the structure preservation, we compute the norm-squared of the solution at each timestep and the norm-squared of the solution gradient at the midpoint of the timestep after each timestep, accessible through the variable `stepper.sample_values[0]`. From the discrete energy law above, one half of the finite differencing of the solution norm-squared should be equal to, but opposite in sign of the gradient norm squared at the midpoints.:

```
norm_form = inner(u,u)*dx
solution_norms = [assemble(norm_form)]
deriv_norms = []
deriv_form = inner(grad(stepper.sample_values[0]),grad(stepper.sample_
↪values[0]))*dx

while (float(t) < 0.2):
    if (float(t) + float(dt) > 0.2):
        dt.assign(0.2 - float(t))
    stepper.advance()
    t.assign(float(t) + float(dt))
    solution_norms.append(assemble(norm_form))
    deriv_norms.append(assemble(deriv_form))
    print(f"At time {float(t):.3f}, LHS is {(solution_norms[-1]-solution_
↪norms[-2])/(2*float(dt)):.4e}, RHS is {deriv_norms[-1]:.4e}, difference is
↪{(solution_norms[-1]-solution_norms[-2])/(2*float(dt)) + deriv_norms[-1]:.
↪4e}")
```

Finally, if you feel you must bypass the `TimeStepper` abstraction, we have some examples how to interact with Irksome at a slightly lower level:

3.22.5 Not using the TimeStepper interface for the heat equation

Invariably, somebody will have a (possibly) compelling reason or at least urgent desire to avoid the top-level interface. This demo shows how one can do just that. It will be sparsely documented except for the critical bits and should only be read after perusing the more basic demos.

We're solving the same problem that is done in the heat equation demos.

Imports:

```
from firedrake import *

from irksome import GaussLegendre, getForm, Dt, MeshConstant
from irksome.tools import get_stage_space
```

Note that we imported *getForm* rather than *TimeStepper*. That's the lower-level function inside Irksome that manipulates UFL and boundary conditions.

Continuing:

```
butcher_tableau = GaussLegendre(1)
N = 64

x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0

msh = RectangleMesh(N, N, x1, y1)
V = FunctionSpace(msh, "CG", 1)
x, y = SpatialCoordinate(msh)
```

(continues on next page)

(continued from previous page)

```
MC = MeshConstant(msh)
dt = MC.Constant(10 / N)
t = MC.Constant(0.0)

S = Constant(2.0)
C = Constant(1000.0)

B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5
uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))

rhs = Dt(uexact) - div(grad(uexact))

u = Function(V)
u.interpolate(uexact)

v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx

bc = DirichletBC(V, 0, "on_boundary")
```

Get the function space for the stage-coupled problem and a function to hold the stages we're computing:

```
Vbig = get_stage_space(V, butcher_tableau.num_stages)
k = Function(Vbig)
```

Get the variational form and bcs for the stage-coupled variational problem:

```
Fnew, bcnew = getForm(F, butcher_tableau, t, dt, u, k, bcs=bc)
```

This returns several things:

- `Fnew` is the UFL variational form for the fully discrete method.
- `bcnew` is a list of new `DirichletBC` that need to be enforced on the variational problem for the stages

Solver parameters are just blunt-force LU. Other options are surely possible:

```
luparams = {"mat_type": "aij",
            "snes_type": "ksponly",
            "ksp_type": "preonly",
            "pc_type": "lu"}
```

We can set up a new nonlinear variational problem and create a solver for it in standard Firedrake fashion:

```
prob = NonlinearVariationalProblem(Fnew, k, bcs=bcnew)
solver = NonlinearVariationalSolver(prob, solver_parameters=luparams)
```

We'll need to split the stage variable so that we can update the solution after solving for the stages at each time step:

```
ks = k.subfunctions
```

And here is our time-stepping loop. Note that unlike in the higher-level interface examples, we have to manually update the solution:

```
while (float(t) < 1.0):
    if float(t) + float(dt) > 1.0:
        dt.assign(1.0 - float(t))
    solver.solve()

    for i in range(butcher_tableau.num_stages):
        u += float(dt) * butcher_tableau.b[i] * ks[i]

    t.assign(float(t) + float(dt))
    print(float(t))

print()
print(errornorm(uexact, u)/norm(uexact))
```

3.22.6 Time-dependent BCs and the low-level interface

This demo shows how to update inhomogeneous and time-dependent boundary conditions with the low-level interface. We are using a different manufactured solution than before, which is obvious from reading through the code.

Imports:

```
from firedrake import *

from irksome import GaussLegendre, getForm, Dt, MeshConstant
from irksome.tools import get_stage_space

butcher_tableau = GaussLegendre(2)
N = 64

msh = UnitSquareMesh(N, N)

MC = MeshConstant(msh)
dt = MC.Constant(10 / N)
t = MC.Constant(0.0)

V = FunctionSpace(msh, "CG", 1)
x, y = SpatialCoordinate(msh)

uexact = exp(-t) * cos(pi * x) * sin(pi * y)
rhs = Dt(uexact) - div(grad(uexact))

u = Function(V)
u.interpolate(uexact)
```

(continues on next page)

(continued from previous page)

```
v = TestFunction(V)
F = inner(Dt(u), v)*dx + inner(grad(u), grad(v))*dx - inner(rhs, v)*dx

bc = DirichletBC(V, uexact, "on_boundary")
```

Get the function space for the stage-coupled problem and a function to hold the stages we're computing:

```
Vbig = get_stage_space(V, butcher_tableau.num_stages)
k = Function(Vbig)
```

Get the variational form and bcs for the stage-coupled variational problem:

```
Fnew, bcnew = getForm(F, butcher_tableau, t, dt, u, k, bcs=bc)
```

Recall that *getForm* produces:

- `Fnew` is the UFL variational form for the fully discrete method.
- `bcnew` is a list of new `DirichletBC` that need to be enforced on the variational problem for the stages

We just use basic solver parameters and set up the variational problem and solver:

```
luparams = {"mat_type": "aij",
            "snes_type": "ksponly",
            "ksp_type": "preonly",
            "pc_type": "lu"}

prob = NonlinearVariationalProblem(Fnew, k, bcs=bcnew)
solver = NonlinearVariationalSolver(prob, solver_parameters=luparams)

ks = k.subfunctions
```

Now, our time-stepping loop shows how to manually update the per-stage boundary conditions at each time step:

```
while (float(t) < 1.0):
    if float(t) + float(dt) > 1.0:
        dt.assign(1.0 - float(t))

    solver.solve()

    for i in range(butcher_tableau.num_stages):
        u += float(dt) * butcher_tableau.b[i] * ks[i]

    t.assign(float(t) + float(dt))
    print(float(t))

print()
print(errornorm(uexact, u)/norm(uexact))
```

3.22.7 Mixed problems with the low-level interface

Updating the solution at each time step is more complicated when we have problems on mixed function spaces. This demo peels back the `TimeStepper` abstraction in the mixed heat equation demo. In this case, the Dirichlet boundary conditions are weakly enforced. However, mixed problems do not change the way strongly-enforced BC are handled, just how the solution is updated.

Imports:

```
from irksome.tools import get_stage_space
from firedrake import *
from irksome import LobattoIIIC, Dt, getForm, MeshConstant

butcher_tableau = LobattoIIIC(2)
```

Build the mesh and approximating spaces:

```
N = 32
x0 = 0.0
x1 = 10.0
y0 = 0.0
y1 = 10.0
msh = RectangleMesh(N, N, x1, y1)

V = FunctionSpace(msh, "RT", 2)
W = FunctionSpace(msh, "DG", 1)
Z = V * W

MC = MeshConstant(msh)
dt = MC.Constant(10.0 / N)
t = MC.Constant(0.0)

x, y = SpatialCoordinate(msh)

S = Constant(2.0)
C = Constant(1000.0)

B = (x-Constant(x0))*(x-Constant(x1))*(y-Constant(y0))*(y-Constant(y1))/C
R = (x * x + y * y) ** 0.5

uexact = B * atan(t)*(pi / 2.0 - atan(S * (R - t)))
sigexact = -grad(uexact)

rhs = Dt(uexact) + div(sigexact)

sigu = project(as_vector([0, 0, uexact]), Z)
sigma, u = split(sigu)

v, w = TestFunctions(Z)

F = (inner(Dt(u), w) * dx + inner(div(sigma), w) * dx - inner(rhs, w) * dx
```

(continues on next page)

(continued from previous page)

```
    + inner(sigma, v) * dx - inner(u, div(v)) * dx)
```

Get the function space for the stage-coupled problem:

```
Vbig = get_stage_space(Z, butcher_tableau.num_stages)
k = Function(Vbig)
```

Get the form and new boundary conditions (which are dropped since we have weak Dirichlet)):

```
Fnew, _ = getForm(F, butcher_tableau, t, dt, sigu, k)
```

We set up the variational problem and solver using a sparse direct method:

```
params = {"mat_type": "aij",
          "snes_type": "ksponly",
          "ksp_type": "preonly",
          "pc_type": "lu"}

prob = NonlinearVariationalProblem(Fnew, k)
solver = NonlinearVariationalSolver(prob, solver_parameters=params)
```

Advancing the solution in time is a bit more complicated now:

```
num_fields = len(Z)
b = butcher_tableau.b

while (float(t) < 1.0):
    if (float(t) + float(dt) > 1.0):
        dt.assign(1.0 - float(t))

    solver.solve()

    for s in range(butcher_tableau.num_stages):
        for i in range(num_fields):
            sigu.dat.data[i][:] += float(dt) * b[s] * k.dat.data[num_fields *␣
↪s + i][:]

    print(float(t))
    t.assign(float(t) + float(dt))
```

Finally, we check the accuracy of the solution:

```
sigma, u = sigu.subfunctions
print("U error      : ", errornorm(uexact, u) / norm(uexact))
print("Sig error    : ", errornorm(sigexact, sigma) / norm(sigexact))
print("Div Sig error: ",
      errornorm(sigexact, sigma, norm_type='Hdiv')
      / norm(sigexact, norm_type='Hdiv'))
```

CHAPTER

FOUR

API DOCUMENTATION

There is also an alphabetical index, and a search engine.

4.1 irksome package

4.1.1 Subpackages

irksome.backends package

Submodules

irksome.backends.dolfinx module

DOLFINx backend for Irksome

class irksome.backends.dolfinx.**Constant**(*value*)

Bases: ScalarValue

Initialise.

assign(*value*)

class irksome.backends.dolfinx.**DirichletBC**(**args: Any*, ***kwargs: Any*)

Bases: DirichletBC

Create an Irksome compatible DirichletBC from an existing DOLFINx bc.

> **Note**
>
> This is not a user-facing class, but rather an internal class used for BC reconstruction in time-stepping problems. Use: *irksome.backends.dolfinx.dirichletbc*.

Parameters

- **g** – The boundary condition expression
- **dofs** – An array of degree-of-freedom indices in V
- **V** – The space to construct the BC on.

pack()

reconstruct(*V*, *g*)

class irksome.backends.dolfinx.**EquationBC**(**args*, ***kwargs*)

Bases: object

class irksome.backends.dolfinx.**EquationBCSplit**(**args*, ***kwargs*)

Bases: object

class irksome.backends.dolfinx.**FloatConstantFunction**(**args: Any*, ***kwargs: Any*)

Bases: Function

assign(*value*)

irksome.backends.dolfinx.**Function**(*V: FunctionSpace | MixedFunctionSpace*, *name=None*)

Create a function in the backend language.

class irksome.backends.dolfinx.**LinearProblem**(**args: Any*, ***kwargs: Any*)

Bases: LinearProblem

Overloaded linear problem that pack BCs before solving

solve(*bounds=None*)

class irksome.backends.dolfinx.**ListTensor**(**expressions*)

Bases: ListTensor

A list tensor that exposes subfunctions for DOLFINx functions

Initialise.

function_space()

property subfunctions

class irksome.backends.dolfinx.**MeshConstant**(*msh*)

Bases: object

Constant(*val=0.0*) → Coefficient

class irksome.backends.dolfinx.**NonlinearProblem**(**args: Any*, ***kwargs: Any*)

Bases: NonlinearProblem

Overloaded nonlinear problem that pack BCs before solving.

Does each Newton iteration or line search step by overriding the SNES function and Jacobian assembly routines to pack BCs before assembly.

property snes: SNES

solve(*bounds=None*)

irksome.backends.dolfinx.**TestFunction**(*function_space*)

Create a test function in the backend language.

This is required as ufl.TestFunctions returns a tuple of test functions, which we need to convert to a tensor for consistency with the Firedrake backend.

irksome.backends.dolfinx.**TrialFunction**(*function_space*)

Create a trial function in the backend language.

This is required as `ufl.TrialFunctions` returns a tuple of trial functions, which we need to convert to a tensor for consistency with the Firedrake backend.

irksome.backends.dolfinx.**assemble**(*expr: Expr | float*)

Assemble a UFL expression in the backend language.

irksome.backends.dolfinx.**bc2space**(*bc*, *V*)

irksome.backends.dolfinx.**create_bounds_constrained_bc**(*V*, *g*, *sub_domain*, *bounds*, *solver_parameters=None*)

irksome.backends.dolfinx.**create_variational_problem**(*F*, *u*, *bcs=None*, *aP=None*, ***kwargs*)

Create a variational problem.

irksome.backends.dolfinx.**create_variational_solver**(*problem: dolfinx.fem.petsc.LinearProblem | dolfinx.fem.petsc.NonlinearProblem*, ***kwargs*)

Create a variational solver that uses PETSc SNES or KSP.

irksome.backends.dolfinx.**dirichletbc**(*value: ufl.core.expr.Expr*, *dofs: npt.NDArray[np.int32]*, *V: dolfinx.fem.FunctionSpace | None = None*) → *DirichletBC*

Overloaded DirichletBC so that we can reconstruct BCs with UFL expressions.

> **Note**
>
> This class is user-facing.

Parameters

- **value** – A UFL expression representing the boundary condition.
- **dofs** – An array of degree-of-freedom indices in *V* where the BC should be applied.
- **V** – The function space on which the BC applies. It can be a subspace of a mixed/blocked space.

irksome.backends.dolfinx.**extract_bcs**(*bcs: Any*) → tuple[Any]

Extract boundary conditions

irksome.backends.dolfinx.**extract_dtype**(*expr: Expr*) → DTypeLike

Extract the dtype from an expression.

Looks for any constants or coefficients and returning their dtype. This is necessary for determining which DOLFINx DirichletBC constructor to use when packing UFL expressions into DOLFINx Expressions for use in BC reconstruction.

irksome.backends.dolfinx.**extract_function**(*expr*) → tuple[bool, dolfinx.fem.Function | None]

Recursively extract a Function from nested UFL expressions.

Returns

A tuple (is_real, func) where is_real=True means func is on RealElement (a constant)

> **Note**
>
> Returns (False, None) for RealElement functions so they get handled as scalars by extract_scalar_value, preserving any multipliers.

irksome.backends.dolfinx.**extract_linear_combination**(*expr: Expr*) → list[tuple[float, dolfinx.fem.Function]]

Extract (weight, function) pairs from a UFL linear combination.

Analyzes expressions like: 0.5*u + 0.3*v + 0.2*w Returns: [(0.5, u), (0.3, v), (0.2, w)]

Parameters

expr – UFL expression (Sum, Product, or single Function)

Returns

List of (weight, function) tuples

irksome.backends.dolfinx.**extract_scalar_value**(*scalar_expr*)

Extract float from a scalar UFL expression.

irksome.backends.dolfinx.**extract_term**(*term*)

Extract (weight, function) from a single term.

irksome.backends.dolfinx.**function_iadd**(*func: dolfinx.fem.Function*, *expr: Expr*) → None

Add a UFL expression to a DOLFINx Function in-place (func += expr).

Extracts the linear combination structure and adds terms directly using PETSc vector operations. Works with mixed spaces and subfunction views.

Parameters

- **func** – DOLFINx Function or subfunction view to update in-place
- **expr** – UFL expression (linear combination of Functions)

```
function_iadd(u, 0.5 * v + 0.3 * w)  # equivalent to u += 0.5*v + 0.3*w
```

irksome.backends.dolfinx.**function_iadd_method**(*self*, *other*)

In-place addition for DOLFINx functions (self += other).

This method is monkey-patched onto `dolfinx.fem.Function` to enable Firedrake-style arithmetic operations.

Parameters

- **self** – The function to be modified

- **other** – The expression to add. Can be a UFL expression, Function, Constant, or scalar.

Returns
self (for chaining)

irksome.backends.dolfinx.**function_space_length**(*self*)

irksome.backends.dolfinx.**function_subfunctions**(*self*)

Get subfunctions for a DOLFINx function, which may be in a mixed space.

This function is called when updating *u0* in time-stepping problems.

irksome.backends.dolfinx.**getNullspace**(*V*, *Vbig*, *num_stages*, *nullspace*)

Computes the nullspace for a multi-stage method.

Parameters

- **V** – The `ufl.FunctionSpace` on which the original time-dependent PDE is posed.
- **Vbig** – The multi-stage `ufl.MixedFunctionSpace` for the stage problem
- **num_stages** – The number of stages in the RK method
- **nullspace** – The nullspace for the original problem.

On output, we produce a PETSc nullspace defining the nullspace for the multistage problem.

irksome.backends.dolfinx.**get_function_space**(*u: Coefficient | Argument*) → FunctionSpace

irksome.backends.dolfinx.**get_mesh_constant**(*MC:* MeshConstant | *None*) → Expr

irksome.backends.dolfinx.**get_stage_space**(*V: FunctionSpace*, *num_stages: int*) → FunctionSpace

irksome.backends.dolfinx.**get_stages**(*V: dolfinx.fem.FunctionSpace*, *num_stages: int*) → *ListTensor*

Given a function space for a single time-step, get a duplicate of this space, repeated *num_stages* times.

Parameters

- **V** – Space for single step
- **num_stages** – Number of stages

Returns
A coefficient in the new function space

irksome.backends.dolfinx.**invalidate_jacobian**(*solver: dolfinx.fem.petsc.LinearProblem*)

Invalidate the Jacobian matrix in the backend language.

irksome.backends.dolfinx.**mixed_space_length**(*self*)

irksome.backends.dolfinx.**norm**(*v: Expr*, *norm_type: str = 'L2'*, *mesh: Mesh | None = None*) → float

Compute the norm of a function in the backend language.

irksome.backends.dolfinx.**stage2spaces4bc**(*bc*, *V*, *Vbig*, *i*)

used to figure out how to apply Dirichlet BC to each stage

irksome.backends.firedrake module

Firedrake backend for Irksome

class irksome.backends.firedrake.**BoundsConstrainedDirichletBC**(*V*, *g*, *sub_domain*, *bounds*, *solver_parameters=None*)

Bases: DirichletBC

property function_arg

The value of this boundary condition.

reconstruct(*V=None*, *g=None*, *sub_domain=None*)

class irksome.backends.firedrake.**MeshConstant**(*msh: Mesh*)

Bases: object

Constant(*val=0.0*) → Coefficient

irksome.backends.firedrake.**bc2space**(*bc*, *V*)

irksome.backends.firedrake.**create_bounds_constrained_bc**(*V*, *g*, *sub_domain*, *bounds*, *solver_parameters=None*)

irksome.backends.firedrake.**create_variational_problem**(*F*, *u*, *bcs=None*, *J=None*, *Jp=None*, ***kwargs*)

irksome.backends.firedrake.**create_variational_solver**(*problem*, ***kwargs*)

irksome.backends.firedrake.**extract_bcs**(*bcs: Any*) → tuple[Any]

Return an iterable of boundary conditions on the residual form

irksome.backends.firedrake.**getNullspace**(*V*, *Vbig*, *num_stages*, *nullspace*)

Computes the nullspace for a multi-stage method.

Parameters

- **V** – The firedrake.FunctionSpace on which the original time-dependent PDE is posed.
- **Vbig** – The multi-stage firedrake.FunctionSpace for the stage problem
- **num_stages** – The number of stages in the RK method
- **nullspace** – The nullspace for the original problem.

On output, we produce a firedrake.MixedVectorSpaceBasis defining the nullspace for the multistage problem.

irksome.backends.firedrake.**get_function_space**(*u: Coefficient*) → FunctionSpace

irksome.backends.firedrake.**get_mesh_constant**(*MC:* MeshConstant | *None*)

irksome.backends.firedrake.**get_stage_space**(*V: FunctionSpace*, *num_stages: int*) → FunctionSpace

irksome.backends.firedrake.**get_stages**(*V: FunctionSpace*, *num_stages: int*) → Function

Given a function space for a single time-step, get a duplicate of this space, repeated *num_stages* times.

Parameters

- **V** – Space for single step
- **num_stages** – Number of stages

Returns

A coefficient in the new function space

irksome.backends.firedrake.**invalidate_jacobian**(*solver*)

irksome.backends.firedrake.**stage2spaces4bc**(*bc*, *V*, *Vbig*, *i*)

used to figure out how to apply Dirichlet BC to each stage

Module contents

irksome.ufl package

Submodules

irksome.ufl.deriv module

irksome.ufl.deriv.**Dt**(*f*, *order=1*)

Short-hand function to produce a *TimeDerivative* of a given order.

exception irksome.ufl.deriv.**IrksomeImportOrderException**

Bases: Exception

class irksome.ufl.deriv.**TimeDerivative**(*f*)

Bases: Derivative

UFL node representing a time derivative of some quantity/field. Note: Currently form compilers do not understand how to process these nodes. Instead, Irksome pre-processes forms containing *TimeDerivative* nodes.

Initalise.

property ufl_free_indices

property ufl_index_dimensions

property ufl_shape

class irksome.ufl.deriv.**TimeDerivativeRuleDispatcher**(*t=None*, *timedep_coeffs=None*, ***kwargs*)

Bases: DAGTraverser

Mapping rules to splat out time derivatives so that replacement should work on more complex problems.

Initialise.

process(*o*)

process(*o:* TimeDerivative)

process(*o: BaseForm*)

process(*o: Expr*)

Process node by type.

Args:

o:
UFL expression to start DAG traversal from.

****kwargs:**
Keyword arguments for the process singledispatchmethod.

Returns:
Processed Expr.

time_derivative(*o*)

class irksome.ufl.deriv.**TimeDerivativeRuleset**(*t=None*, *timedep_coeffs=None*)

Bases: GenericDerivativeRuleset

Apply AD rules to time derivative expressions.

Initialise.

constant(*o*)

process(*o*)

process(*o: ConstantValue*)

process(*o: SpatialCoordinate*)

process(*o: Coefficient*)

process(*o: Argument*)

process(*o:* TimeDerivative, *f*)

process(*o: Variable*, **operands*)

process(*o: ReferenceValue*, **operands*)

process(*o: ReferenceGrad*, **operands*)

process(*o: Indexed*, **operands*)

process(*o: Grad*, **operands*)

process(*o: Div*, **operands*)

process(*o: Derivative*, **operands*)

process(*o: Curl*, **operands*)

process(*o: Conj*, **operands*)

Process o.

Args:
o: *Expr* to be processed.

Returns:
Processed object.

terminal(*o*)

terminal_modifier(*o*, **operands*)

time_derivative(*o*, *f*)

irksome.ufl.deriv.**apply_time_derivatives**(*expression*, *t=None*, *timedep_coeffs=None*)

irksome.ufl.deriv.**check_irksome_import_order**()

Check that irksome has been imported early enough.

Due to the inadequacies of the UFL type system, it is not possible to define a new UFL type once any ufl MultiFunction has been used.

This restriction can be removed once all MultiFunctions have been transitioned to ufl.corealg.dag_traverser.DAGTraverser.

irksome.ufl.deriv.**expand_time_derivatives**(*expression*, *t=None*, *timedep_coeffs=None*)

irksome.ufl.estimate_degrees module

class irksome.ufl.estimate_degrees.**TimeDegreeEstimator**(*degree_mapping=None*, ***kwargs*)

Bases: DAGTraverser

Time degree estimator.

This algorithm is exact for a few operators and heuristic for many.

Initialise.

add_degrees(*v*, **ops*)

conditional(*v*, *c*, **ops*)

form(*o*)

formsum(*o*)

integral(*o*)

interpolate(*o*)

math_function(*v*, *a*)

Apply to math_function.

Using the heuristic: degree(sin(const)) == 0 degree(sin(a)) == degree(a)+2 which can be wildly inaccurate but at least gives a somewhat high integration degree.

max_degree(*v*, **ops*)

minmax(*v*, **ops*)

non_numeric(*v*, **args*)

not_handled(*v*, **ops*)

power(*v*, *a*, *b*)

Apply to power.

If b is a positive integer: degree(a**b) == degree(a)*b otherwise use the heuristic: degree(a**b) == degree(a) + 2.

process(*o*)

process(*o: FormSum*)

process(*o: Interpolate*)

process(*o: Form*)

process(*o: Integral*)

process(*o: SpatialCoordinate*)

process(*o: ConstantValue*)

process(*o: Coefficient*)

process(*o: Cofunction*)

process(*o: Argument*)

process(*o:* TimeDerivative, *degree*)

process(*o: IndexSum*, *degree*, **ops*)

process(*o: ComponentTensor*, *degree*, **ops*)

process(*o: Variable*, *degree*, **ops*)

process(*o: ReferenceValue*, *degree*, **ops*)

process(*o: ReferenceGrad*, *degree*, **ops*)

process(*o: Indexed*, *degree*, **ops*)

process(*o: Grad*, *degree*, **ops*)

process(*o: Div*, *degree*, **ops*)

process(*o: Derivative*, *degree*, **ops*)

process(*o: Curl*, *degree*, **ops*)

process(*o: Conj*, *degree*, **ops*)

process(*o: Abs*, *degree*, **ops*)

process(*v: Inverse*, **ops*)

process(*v: Determinant*, **ops*)

process(*v: Transposed*, **ops*)

process(*v: Trace*, **ops*)

process(*v: Sym*, **ops*)

process(*v: Skew*, **ops*)

process(*v: Cofactor*, **ops*)

process(*v: ExprMapping*, **ops*)

process(*v: ExprList*, **ops*)

process(*v: ListTensor*, **ops*)

process(*v: Sum*, **ops*)

process(*v: Cross*, **ops*)
process(*v: Outer*, **ops*)
process(*v: Dot*, **ops*)
process(*v: Inner*, **ops*)
process(*v: Product*, **ops*)
process(*v: Division*, **ops*)
process(*v: Power*, *a*, *b*)
process(*v: MathFunction*, *a*)
process(*v: Conditional*, *c*, **ops*)
process(*v: MaxValue*, **ops*)
process(*v: MinValue*, **ops*)
process(*v: MultiIndex*, **args*)
process(*v: Condition*, **args*)
process(*v: Label*, **args*)

Process node by type.

Args:

o:
UFL expression to start DAG traversal from.

****kwargs:**
Keyword arguments for the `process` singledispatchmethod.

Returns:
Processed `Expr`.

terminal(*o*)

terminal_modifier(*o*, *degree*, **ops*)

time_derivative(*o*, *degree*)

irksome.ufl.estimate_degrees.**estimate_time_degree**(*expression*, *test_degree*, *trial_degree*, *t=None*, *timedep_coeffs=None*)

irksome.ufl.estimate_degrees.**get_degree_mapping**(*expression*, *test_degree*, *trial_degree*, *t=None*, *timedep_coeffs=None*)

Map time-dependent terminals to their polynomial degree.

Parameters

- **expression** – a `ufl.BaseForm` or `ufl.Expr`.
- **test_degree** – the temporal polynomial degree of the test space.
- **trial_degree** – the temporal polynomial degree of the trial space.
- **t** – the time variable as a `Constant` or `Function` in the Real space.
- **timedep_coeffs** – a list of `Function` that depend on time.

Returns
a *dict* mapping time-dependent terminals to their degree in time.

irksome.ufl.lag module

irksome.ufl.lag.**lag**(*expr*)

Mark a sub-expression to be evaluated only at the start of the timestep during the implicit solve.

irksome.ufl.manipulation module

Manipulation of expressions containing *TimeDerivative* terms.

These can be used to do some basic checking of the suitability of a Form for use in Irksome (via *check_integrals*), and splitting out terms in the Form that contain a time derivative from those that don't (via *split_time_derivative_terms*).

class irksome.ufl.manipulation.**SplitTimeForm**(*time: BaseForm*, *remainder: BaseForm*)

Bases: NamedTuple

A container for a form split into time terms and a remainder.

Create new instance of SplitTimeForm(time, remainder)

remainder: BaseForm

Alias for field number 1

time: BaseForm

Alias for field number 0

irksome.ufl.manipulation.**check_integrals**(*integrals: Sequence[Integral]*, *t: Expr = None*, *timedep_coeffs: Sequence[Coefficient] = ()*, *expect_time_derivative: bool = True*)

Check a list of integrals for linearity in the time derivative.

Parameters

- **integrals** – list of integrals.
- **timedep_coeffs** – The time-dependent coefficients.
- **expect_time_derivative** – Are we expecting to see a time derivative?

Raises

ValueError – if we are expecting a time derivative and don't see one, or time derivatives are applied nonlinearly, to more than one coefficient, or more than first order.

irksome.ufl.manipulation.**has_nonlinear_time_derivative**(*F*, *u0*)

True iff F contains a TimeDerivative of an expression that is nonlinear in u0 – i.e. Dt(g(u0)) for some nonlinear g. These cases lose mass conservation when chain-ruled through the stage-derivative form, and require the conservative two-evaluation discretisation.

For each Dt(f) in the form, the Gateaux derivative of f with respect to u0 is taken in a trial direction. If the derivative still depends on u0, f is nonlinear in u0. This delegates

the classification of linear operators (Grad, Div, Indexed, restrictions, ListTensor, ComponentTensor, ...) to UFL's own derivative machinery rather than maintaining a parallel exemption list inside Irksome.

> **Warning**
>
> The detection is syntactic: it checks whether `u0` appears under `Dt` after differentiation. If a user creates an intermediate `Function` whose values were interpolated from an expression in u0 and then writes `Dt(that_intermediate)`, the syntactic dependence on u0 is lost and this function will declare the form safe. The resulting discretisation is *not* mass-conservative. Always wrap the symbolic expression directly in `Dt` (as `Dt(theta(u))`, not `Dt(theta_function)`).

`irksome.ufl.manipulation.`**`remove_time_derivatives`**(*F: Form*)

Helper function to strip all time derivatives from a Form

`irksome.ufl.manipulation.`**`split_time_derivative_terms`**(*form: BaseForm*, *t: Expr = None*, *timedep_coeffs: Sequence[Coefficient] = ()*) → *SplitTimeForm*

Split terms from a `Form`.

This splits a form (a sum of integrals) into those integrals which do contain a *`TimeDerivative`* acting on *timedep_coeffs* and those that don't.

Parameters

- **`form`** – The form to split.
- **`t`** – The time variable.
- **`timedep_coeffs`** – The time-dependent coefficients.

Returns

a *`SplitTimeForm`* tuple.

Raises

`ValueError` – if the form does not apply anything other than first-order time derivatives to a single coefficient.

Module contents

4.1.2 Submodules

4.1.3 irksome.backend module

`class` `irksome.backend.`**`Backend`**(**args*, ***kwargs*)

Bases: `Protocol`

`class Constant`

Bases: `object`

MeshLess constant class

ConstantOrZero(*MC:* MeshConstant | *None* = *None*) → Expr

Create a constant with backend class if MeshConstant is not supplied. Create UFL zero if *x* is sufficiently small

class DirichletBC

Bases: `object`

class EquationBC

Bases: `object`

class EquationBCSplit

Bases: `object`

class Function

Bases: `object`

class MeshConstant(*msh: Mesh*)

Bases: `object`

Initialize a mesh constant over a domain

Constant(*val: float = 0.0*)

Generate a constant in the backend language with a specific value

TestFunction(*part: int* | *None* = *None*) → Argument

Return a test-function that can be used by forms in the backend.

TrialFunction(*part: int* | *None* = *None*) → Argument

Return a trial-function that can be used by forms in the backend.

assemble() → Any

Assemble a UFL expression in the backend language.

create_bounds_constrained_bc(*g*, *sub_domain*, *bounds*, *solver_parameters=None*) → *DirichletBC*

create_variational_problem(*u: Coefficient*, *bcs:* DirichletBC | *Sequence* | *None* = *None*, ***kwargs*) → Any

Create a variational problem in the backend language.

create_variational_solver(*solver_parameters: dict* | *None* = *None*, ***kwargs*)

Create a variational solver in the backend language.

derivative(*u: Coefficient*, *du: Argument* | *None* = *None*, *coefficient_derivatives: dict* | *None* = *None*) → Form

Compute the derivative of a form with respect to a coefficient in the backend language.

extract_bcs() → tuple[Any]

Extract boundary conditions

getNullspace(*Vbig*, *num_stages*, *nullspace*)

get_function_space(*V: Coefficient*) → FunctionSpace

Get a function space from the backend

get_mesh_constant() → Expr

Get a backend class to construct a mesh constant from

get_stage_spaces(*num_stages: int*) → FunctionSpace

Create a stage space with M number of components.

get_stages(*V: FunctionSpace*, *num_stages: int*) → Coefficient

Given a function space for a single time-step, get a duplicate of this space, repeated *num_stages* times.

Parameters

- **V** – Space for single step
- **num_stages** – Number of stages

Returns

A coefficient in the new function space

invalidate_jacobian()

Invalidate the Jacobian matrix in the backend language.

norm(*norm_type: str = 'L2'*, *mesh: Mesh | None = None*) → float

Compute the norm of a function in the backend language.

irksome.backend.**get_backend**(*backend: str | ModuleType*) → *Backend*

Get backend class from backend name.

Parameters

backend – Name of the backend to get

Returns

Backend class

4.1.4 irksome.base_time_stepper module

class irksome.base_time_stepper.**BaseTimeStepper**(*F*, *t*, *dt*, *u0*, *bcs=None*, *appctx=None*, *nullspace=None*, *backend: str = 'firedrake'*)

Bases: object

Base class for various time steppers. This is mainly to give code reuse stashing objects that are common to all the time steppers. It's a developer-level class.

abstractmethod advance()

abstractmethod solver_stats()

class irksome.base_time_stepper.**StageCoupledTimeStepper**(*F*, *t*, *dt*, *u0*, *num_stages*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *splitting=None*, *bc_type=None*, *scheme_F=None*, *scheme_J=None*, *scheme_Jp=None*, *bounds=None*, *sample_points=None*, *backend='firedrake'*, ***kwargs*)

Bases: *BaseTimeStepper*

This developer-level class provides common features used by various methods requiring stage-coupled variational problems to compute the stages (e.g. fully implicit RK, Galerkin-in-time)

Parameters

- **F** – A `ufl.Form` instance describing the semi-discrete problem `F(t, u; v) == 0`, where `u` is the unknown `Function` and `v` is the `TestFunction`. To specify a linear problem, `F` must be of the form `a(t; w, v) - L(t; v)`, where `w` is a `TrialFunction`.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **num_stages** – The number of stages to solve for. It could be the number of RK stages or relate to the polynomial degree (Galerkin)
- **bcs** – An iterable of `DirichletBC` or `EquationBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method. Support for *EquationBC* is limited to the stage derivative formulation with DAE style BCs.
- **J** – A `ufl.Form` instance with the semi-discrete Jacobian.
- **Jp** – A `ufl.Form` instance to precondition the semi-discrete linearization.
- **solver_parameters** – An optional `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.

- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **nullspace** – A `VectorSpaceBasis` or `MixedVectorSpaceBasis` specifying a nullspace over the space of `u0`.
- **splitting** – An optional kwarg (not used by all superclasses)
- **bc_type** – An optional kwarg (not used by all superclasses)
- **butcher_tableau** – A `ButcherTableau` instance giving the Runge-Kutta method to be used for time marching.
- **bounds** – An optional kwarg used in certain bounds-constrained methods.
- **sample_points** – An optional kwarg used to evaluate collocation methods at additional points in time.

advance()

Advances the system from time *t* to time *t* + *dt*. Note: overwrites the value *u0*.

build_poly()

When provided with a list of *sample_points* (intended to be in the interval [0,1]), this builds a symbolic expression for the values of the RK collocation polynomial at the corresponding points on the interval [t_n, t_{n+1}]. These are stored in the list *self.sample_values* as functions in the same FunctionSpace as *self.u0*. The resulting expressions can then be assigned to a Function on that same FunctionSpace.

get_bilinear_form(*form*, *stages*, *tableau=None*)

abstractmethod get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

get_stage_bounds(*bounds=None*)

get_stages() → Coefficient

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

abstractmethod tabulate_poly(*sample_points*)

4.1.5 irksome.bcs module

irksome.bcs.**BCStageData**(*bc*, *gcur*, *u0*, *stages*, *i*, *backend='firedrake'*)

irksome.bcs.**BoundsConstrainedDirichletBC**(*V*, *g*, *sub_domain*, *bounds*, *solver_parameters=None*, *backend='firedrake'*)

A DirichletBC with bounds-constrained data.

irksome.bcs.**EmbeddedBCData**(*bc*, *butcher_tableau*, *t*, *dt*, *u0*, *stages*, *backend='firedrake'*)

4.1.6 irksome.constant module

irksome.constant.**ConstantOrZero**(*x: float | complex*, *MC:* MeshConstant | *None = None*, *backend: str = 'firedrake'*) → Expr

irksome.constant.**MeshConstant**(*msh*, *backend: str = 'firedrake'*)

4.1.7 irksome.dirk_stepper module

class irksome.dirk_stepper.**DIRKTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *backend='firedrake'*, ***kwargs*)

Bases: `object`

Front-end class for advancing a time-dependent PDE via a diagonally-implicit Runge-Kutta method formulated in terms of stage derivatives.

advance()

get_bilinear_form(*form*, *u0*, *stages*, *tableau=None*)

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

update_bc_constants(*i*, *c*)

irksome.dirk_stepper.**getFormDIRK**(*F*, *ks*, *butch*, *t*, *dt*, *u0*, *bcs=None*, *kgac=None*, *backend='firedrake'*)

4.1.8 irksome.discontinuous_galerkin_stepper module

class irksome.discontinuous_galerkin_stepper.**DiscontinuousGalerkinTimeStepper**(*F*, *scheme*, *t*, *dt*, *u0*, *bcs=None*, *basis_type=Non* *quad-* *rat-* *ure=None*, *backend='firedra* ***kwargs*)

Bases: *StageCoupledTimeStepper*

Front-end class for advancing a time-dependent PDE via a Discontinuous Galerkin in time method

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **scheme** – a `DiscontinuousGalerkinScheme` instance describing the order, basis type, default quadrature scheme, and time derivative integration type.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step size.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of *DirichletBC* containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the method.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **nullspace** – An optional nullspace object.

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*, *basis_type=None*, *order=None*, *quadrature=None*, *deriv_type=None*)

set_initial_guess()

Set a constant-in-time initial guess.

tabulate_poly(*sample_points*)

irksome.discontinuous_galerkin_stepper.**getElement**(*basis_type*, *order*)

irksome.discontinuous_galerkin_stepper.**getFormDiscGalerkin**(*F*, *L*, *Qdefault*, *t*, *dt*, *u0*, *stages*, *bcs=None*, *deriv_type='strong'*, *max_quadrature_degree=None*, *backend='firedrake'*)

Given a time-dependent variational form, trial and test spaces, and a quadrature rule, produce UFL for the Discontinuous Galerkin-in-Time method.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **L** – A `FIAT.FiniteElement` for the test and trial functions in time
- **Qdefault** – A `FIAT.QuadratureRule` for the time integration
- **t** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.

- **dt** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step. The user may adjust this value between time steps.
- **u0** – a `Function` referring to the state of the PDE system at time *t*
- **stages** – a `Function` representing the stages to be solved for.
- **bcs** – optionally, a *DirichletBC* object (or iterable thereof) containing (possibly time-dependent) boundary conditions imposed on the system.
- **deriv_type** – A string indicating how to integrate terms with time derivatives. Valid values are: - *"weak"*: Time derivatives act on the test function (integrating by parts once). - *"strong"*: Time derivatives act on the unknown (integrating by parts twice).
- **max_quadrature_degree** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If `None`, then the estimated quadrature degree will always be used.

On output, we return a tuple consisting of two parts:

- Fnew, the `Form` corresponding to the DG-in-Time discretized problem
- *bcnew*, a list of *DirichletBC* objects to be posed on the Galerkin-in-time solution,

irksome.discontinuous_galerkin_stepper.**getTermDiscGalerkin**(*F*, *L*, *Q*, *t*, *dt*, *u0*, *stages*, *test*, *deriv_type='strong'*, *backend='firedrake'*)

irksome.discontinuous_galerkin_stepper.**get_element_and_quadrature**(*scheme*)

4.1.9 irksome.explicit_stepper module

class irksome.explicit_stepper.**ExplicitTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *solver_parameters=None*, ***kwargs*)

Bases: *DIRKTimeStepper*

4.1.10 irksome.galerkin_stepper module

class irksome.galerkin_stepper.**ContinuousPetrovGalerkinTimeStepper**(*F*, *scheme*, *t*, *dt*, *u0*, *bcs=None*, *bc_type=None*, *aux_indices=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

Front-end class for advancing a time-dependent PDE via a Galerkin in time method

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **scheme** – `ContinuousPetrovGalerkinScheme` encoding the order, basis type, and default quadrature rule of the method.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the method.
- **bc_type** – How to manipulate the strongly-enforced boundary conditions to derive the stage boundary conditions. Should be a string, either "DAE", which implements BCs as constraints in the style of a differential-algebraic equation, or "ODE", which evaluates the temporal degrees of freedom of the boundary data. Currently there is no support for *EquationBC*.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **nullspace** – An optional nullspace object.
- **aux_indices** – a list of field indices to be discretized in the test space rather than trial space.

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*, *basis_type=None*, *order=None*, *quadrature=None*, *aux_indices=None*)

set_initial_guess()

Set a constant-in-time initial guess.

set_update_expressions()

Set up symbolic expressions for the update.

tabulate_poly(*sample_points*)

irksome.galerkin_stepper.**getElements**(*basis_type*, *order*)

irksome.galerkin_stepper.**getFormGalerkin**(*F*, *L_trial*, *L_test*, *Qdefault*, *t*, *dt*, *u0*, *stages*, *bcs=None*, *bc_type=None*, *aux_indices=None*, *max_quadrature_degree=None*, *backend: str = 'firedrake'*)

Given a time-dependent variational form, trial and test spaces, and a quadrature rule, produce UFL for the Galerkin-in-Time method.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **L_trial** – A `FIAT.FiniteElement` for the trial functions in time
- **L_test** – A `FIAT.FinteElement` for the test functions in time
- **Qdefault** – A `FIAT.QuadratureRule` for the time integration. This rule will be used for all terms in the semidiscrete variational form that aren't specifically tagged with another quadrature rule.
- **t** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step. The user may adjust this value between time steps.
- **u0** – a `Function` referring to the state of the PDE system at time *t*
- **stages** – a `Function` representing the stages to be solved for.
- **bcs** – optionally, a `DirichletBC` object (or iterable thereof) containing (possibly time-dependent) boundary conditions imposed on the system.
- **bc_type** – How to manipulate the strongly-enforced boundary conditions to derive the stage boundary conditions. Should be a string, either "DAE", which implements BCs as constraints in the style of a differential-algebraic equation, or "ODE", which evaluates the temporal degrees of freedom of the boundary data. Currently there is no support for `EquationBC`.
- **aux_indices** – a list of field indices to be discretized in the test space rather than trial space.
- **max_quadrature_degree** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If `None`, then the estimated quadrature degree will always be used.

Returns

a 2-tuple of - *Fnew*, the `Form` corresponding to the Galerkin-in-Time discretized problem - *bcnew*, a list of `DirichletBC` objects to be posed

on the Galerkin-in-time solution

irksome.galerkin_stepper.**getTermGalerkin**(*F*, *L_trial*, *L_test*, *Q*, *t*, *dt*, *u0*, *stages*, *test*, *aux_indices*, *backend='firedrake'*)

irksome.galerkin_stepper.**getTrialElement**(*basis_type*, *order*)

irksome.galerkin_stepper.**get_elements_and_quadrature**(*scheme*)

4.1.11 irksome.imex module

class irksome.imex.**DIRKIMEXMethod**(*F*, *F_explicit*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *solver_parameters=None*, *mass_parameters=None*, *appctx=None*, *nullspace=None*, *backend='firedrake'*, ***kwargs*)

Bases: object

Front-end class for advancing a time-dependent PDE via a diagonally-implicit Runge-Kutta IMEX method formulated in terms of stage derivatives. This implementation assumes a weak form written as F + F_explicit = 0, where both F and F_explicit are UFL Forms, with terms in F to be handled implicitly and those in F_explicit to be handled explicitly

advance()

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

class irksome.imex.**RadauIIAIMEXMethod**(*F*, *Fexp*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *it_solver_parameters=None*, *prop_solver_parameters=None*, *splitting=<function AI>*, *appctx=None*, *nullspace=None*, *num_its_initial=0*, *num_its_per_step=0*, *backend='firedrake'*, ***kwargs*)

Bases: object

Class for advancing a time-dependent PDE via a polynomial IMEX/RadauIIA method. This requires one to split the PDE into an implicit and explicit part. The class sets up two methods – *advance* and *iterate*. The former is used to move the solution forward in time, while the latter is used both to start the method (filling up the initial stage values) and can be used at each time step to increase the accuracy/stability. In the limit as the iterator is applied many times per time step, one expects convergence to the solution that would have been obtained from fully-implicit RadauIIA method.

Parameters

- **F** – A `ufl.Form` instance describing the implicit part of the semi-discrete problem `F(t, u; v) == 0`, where `u` is the unknown `Function` and `v` is the `TestFunction`. To specify a linear problem, `F` must be of the form `a(t; w, v) - L(t; v)`, where `w` is a `TrialFunction`.
- **Fexp** – A `ufl.Form` instance describing the part of the PDE that is explicitly split off.
- **butcher_tableau** – A `ButcherTableau` instance giving the Runge-Kutta method to be used for time marching. Only RadauIIA is allowed here (but it can be any number of stages).
- **t** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time.

- **dt** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method.
- **it_solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with the iterator.
- **prop_solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with the propagator.
- **splitting** – A callable used to factor the Butcher matrix, currently, only AI is supported.
- **appctx** – An optional `dict` containing application context.
- **nullspace** – An optional null space object.

advance()

get_form_and_bcs(*stages*, *tableau=None*, *F=None*)

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

iterate()

Called 1 or more times to set up the initial state of the system before time-stepping. Can also be called after each call to *advance*

propagate()

Moves the solution forward in time, to be followed by 0 or more calls to *iterate*.

solver_stats()

irksome.imex.**getFormExplicit**(*Fexp*, *butch*, *u0*, *UU*, *t*, *dt*, *splitting=None*, *backend='firedrake'*)

Processes the explicitly split-off part for a RadauIIA-IMEX method. Returns the forms for both the iterator and propagator, which really just differ by which constants are in them.

irksome.imex.**getFormsDIRKIMEX**(*F*, *Fexp*, *ks*, *khats*, *butch*, *t*, *dt*, *u0*, *bcs=None*, *backend='firedrake'*)

irksome.imex.**riia_explicit_coeffs**(*k*)

Computes the coefficients needed for the explicit part of a RadauIIA-IMEX method.

4.1.12 irksome.integrated_lagrange module

class irksome.integrated_lagrange.**IntegratedLagrange**(*element*)

Bases: `CiarletElement`

1D element with derivate DOFs on quadrature points.

class irksome.integrated_lagrange.**IntegratedLagrangeDualSet**(*ref_el*, *points*)

Bases: DualSet

The dual basis for 1D Integrated Lagrange elements.

4.1.13 irksome.labeling module

class irksome.labeling.**TimeQuadratureLabel**(**args*, *scheme='default'*)

Bases: Label

If the constructor gets one argument, it's an integer for the order of the quadrature rule. If there are two arguments, assume they are the points and weights.

Parameters

label
The name of the label.

value
The value for the label to take. Can be any type (subject to the validator). Defaults to True.

validator
Function to check the validity of any value later passed to the label. Defaults to None.

default_value

label

validator

value

class irksome.labeling.**TimeQuadratureRule**(*x*, *w*)

Bases: object

get_points()

get_weights()

irksome.labeling.**apply_time_quadrature_labels**(*form*, *degree_estimator*, *scheme=None*, *max_quadrature_degree=None*)

Estimates the polynomial degree in time for each integral in the given form and labels each term with a quadrature rule to be used for time integration.

Parameters

- **form** – a BaseForm or a partially labelled LabelledForm.
- **degree_estimator** – a TimeDegreeEstimator instance.
- **scheme** – a string with the quadrature scheme.
- **max_quadrature_degree** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If None, then the estimated quadrature degree will always be used.

Returns

a `LabelledForm` labelled by `TimeQuaradratureRule` instances.

`irksome.labeling.`**`as_form`**(*form*)

Extracts the `Form` from a `LabelledForm`.

`irksome.labeling.`**`has_quad_labels`**(*term*)

`irksome.labeling.`**`split_explicit`**(*F*)

`irksome.labeling.`**`split_quadrature`**(*F*, *degree_estimator=None*, *Qdefault=None*, *max_quadrature_degree=None*)

Splits a `LabelledForm` into the terms to be integrated in time by the different *`TimeQuadratureRule`* objects used as labels.

Parameters

- **`F`** – a `Form` or a `LabelledForm`.
- **`degree_estimator`** – a `TimeDegreeEstimator` instance.
- **`Qdefault`** – the *`TimeQuadratureRule`* to be applied on unlabelled terms. Alternatively, a string indicating the quadrature scheme, in which case the degree is automatically estimated for each unlabelled term.
- **`max_quadrature_degree`** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If `None`, then the estimated quadrature degree will always be used.

Returns

a *dict* mapping unique *`TimeQuadratureRule`* objects to the `Form` to be integrated in time.

4.1.14 irksome.multistep module

class `irksome.multistep.`**`MultistepTimeStepper`**(*F*, *method*, *t*, *dt*, *u0*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *bounds=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *startup_parameters=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *`BaseTimeStepper`*

front-end class for advancing time-dependent PDE via a general multistep method

Parameters

- **`F`** – A `ufl.Form` instance describing the semi-discrete problem F(t, u; v) == 0, where *u* is the unknown `Function and` `v` is the :class:TestFunction`.
- **`method`** – A `MultistepMethod` corresponding to the desired multistep method.

- **t** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` containing the strongly-enforced boundary conditions.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **startup_parameters** – An optional `dict` used to construct a single-step TimeStepper to be used to find the required starting values.

advance()

get_bilinear_form(*form*, *u0*)

get_form_and_bcs(*F*, *t*, *dt*, *u0*, *a*, *b*, *bcs=None*)

solver_stats()

startup()

4.1.15 irksome.nystrom_dirk_stepper module

class irksome.nystrom_dirk_stepper.**DIRKNystromTimeStepper**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *bc_type=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: `object`

Front-end class for advancing a second-order time-dependent PDE via a diagonally-implicit Runge-Kutta-Nystrom method formulated in terms of stage derivatives.

advance()

get_bilinear_form(*form*, *u0*, *stages*, *tableau=None*)

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

update_bc_constants(*i*, *c*)

class irksome.nystrom_dirk_stepper.**ExplicitNystromTimeStepper**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *bc_type=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *DIRKNystromTimeStepper*

Front-end class for advancing a second-order time-dependent PDE via an explicit Runge-Kutta-Nystrom method formulated in terms of stage derivatives.

irksome.nystrom_dirk_stepper.**getFormDIRKNystrom**(*F*, *ks*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *bc_type=None*, *kgac=None*, *backend='firedrake'*)

4.1.16 irksome.nystrom_stepper module

class irksome.nystrom_stepper.**ClassicNystrom4Tableau**

Bases: *NystromTableau*

class irksome.nystrom_stepper.**NystromTableau**(*A*, *b*, *c*, *Abar*, *bbar*, *order*)

Bases: object

property is_diagonally_implicit

property is_explicit

property is_fully_implicit

property is_implicit

property num_stages

class irksome.nystrom_stepper.**StageDerivativeNystromTimeStepper**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *bc_type='DAE'*, *backend='firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

tabulate_poly(*sample_points*)

irksome.nystrom_stepper.**butcher_to_nystrom**(*butch*)

irksome.nystrom_stepper.**getFormNystrom**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *stages*, *bcs=None*, *bc_type=None*, *backend='firedrake'*)

4.1.17 irksome.pc module

class irksome.pc.**ClinesBase**

Bases: *NystromAuxiliaryOperatorPC*

Base class for methods out of Clines/Howle/Long.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update
- apply
- applyTranspose

class irksome.pc.**ClinesLD**

Bases: *ClinesBase*

Implements Clines-type preconditioner using Atilde = LD where A=LDU.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update
- apply
- applyTranspose

getAtildes(*A*, *Abar*)

Derived classes produce a typically structured approximation to A and Abar.

class irksome.pc.**IRKAuxiliaryOperatorPC**

Bases: AuxiliaryOperatorPC

Base class that inherits from Firedrake's AuxiliaryOperatorPC class and provides the preconditioning bilinear form associated with an auxiliary Form and/or approximate Butcher matrix (which are provided by subclasses).

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update

- apply
- applyTranspose

form(*pc*, *test*, *trial*)

Implements the interface for AuxiliaryOperatorPC.

getAtilde(*A*)

Derived classes produce a typically structured approximation to A.

getNewForm(*pc*, *u0*, *test*)

Derived classes can optionally provide an auxiliary Form.

class irksome.pc.**NystromAuxiliaryOperatorPC**

Bases: AuxiliaryOperatorPC

Base class that inherits from Firedrake's AuxiliaryOperatorPC class and provides the preconditioning bilinear form associated with an auxiliary Form and/or approximate Nystrom matrices (which are provided by subclasses).

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update
- apply
- applyTranspose

form(*pc*, *test*, *trial*)

Implements the interface for AuxiliaryOperatorPC.

getAtildes(*A*, *Abar*)

Derived classes produce a typically structured approximation to A and Abar.

getNewForm(*pc*, *u0*, *ut0*, *test*)

Derived classes can optionally provide an auxiliary Form.

class irksome.pc.**RanaBase**

Bases: *IRKAuxiliaryOperatorPC*

Base class for methods out of Rana, Howle, Long, Meek, & Milestone.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update
- apply
- applyTranspose

class irksome.pc.**RanaDU**

Bases: *RanaBase*

Implements Rana-type preconditioner using Atilde = DU where A=LDU.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`
- `apply`
- `applyTranspose`

getAtilde(*A*)

Derived classes produce a typically structured approximation to A.

class irksome.pc.**RanaLD**

Bases: *RanaBase*

Implements Rana-type preconditioner using Atilde = LD where A=LDU.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`
- `apply`
- `applyTranspose`

getAtilde(*A*)

Derived classes produce a typically structured approximation to A.

irksome.pc.**ldu**(*A*)

4.1.18 irksome.scheme module

class irksome.scheme.**ContinuousPetrovGalerkinScheme**(*order*, *basis_type=None*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*)

Bases: *GalerkinScheme*

Class for describing cPG-in-time methods

class irksome.scheme.**DiscontinuousGalerkinCollocationScheme**(*order*, *deriv_type='strong'*, *quadrature_degree=None*, *quadrature_scheme='radau'*, *max_quadrature_degree=None*)

Bases: *DiscontinuousGalerkinScheme*

Class for describing collocation DG-in-time methods

as_collocation_scheme()

Return a true collocation scheme with collocated quadrature

class irksome.scheme.**DiscontinuousGalerkinScheme**(*order*, *basis_type=None*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*, *deriv_type='strong'*)

Bases: *GalerkinScheme*

Class for describing DG-in-time methods

Parameters

- **order** – An integer indicating the order of the method
- **basis_type** – A string (or tuple of strings) indicating the finite element family (either `'Lagrange'` or `'Bernstein'`) or the Lagrange variant (either `'equispaced'`, `'spectral'`, `'chebyshev'`, or `'integral'`) for the test/trial spaces. Defaults to equispaced Lagrange elements.
- **quadrature_degree** – An integer or string indicating the degree of the quadrature to be used in time. Defaults to the sum of the degrees of the trial and test spaces. The string value `'auto'` enables automatic degree estimation for each term in the form.
- **quadrature_scheme** – A string indicating the quadrature scheme to be used in time. Defaults to Gauss-Legendre.
- **max_quadrature_degree** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If `None`, then the estimated quadrature degree will always be used.
- **deriv_type** – A string indicating how to integrate terms with time derivatives. Valid values are: - *'weak'*: Time derivatives act on the test function (integrating by parts once). - *'strong'*: Time derivatives act on the unknown (integrating by parts twice).

class irksome.scheme.**GalerkinCollocationScheme**(*order*, *stage_type='deriv'*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*)

Bases: *ContinuousPetrovGalerkinScheme*

Class for describing collocation cPG-in-time methods

as_collocation_scheme()

Return a true collocation scheme with collocated quadrature

class irksome.scheme.**GalerkinScheme**(*order*, *basis_type=None*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*)

Bases: `object`

Base class for describing Galerkin-in-time methods in lieu of a Butcher tableau.

Parameters

- **order** – An integer indicating the order of the method
- **basis_type** – A string (or tuple of strings) indicating the finite element family (either `'Lagrange'` or `'Bernstein'`) or the Lagrange variant (either `'equispaced'`, `'spectral'`, `'chebyshev'`, or `'integral'`) for the test/trial spaces. Defaults to equispaced Lagrange elements.
- **quadrature_degree** – An integer or string indicating the degree of the quadrature to be used in time. Defaults to the sum of the degrees of the trial and test spaces. The string value `'auto'` enables automatic degree estimation for each term in the form.
- **quadrature_scheme** – A string indicating the quadrature scheme to be used in time. Defaults to Gauss-Legendre.
- **max_quadrature_degree** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If `None`, then the estimated quadrature degree will always be used.

irksome.scheme.**collocation_quadrature_degree**(*trial_degree*, *scheme*)

Return the quadrature degree for a collocation scheme.

irksome.scheme.**create_time_quadrature**(*degree*, *scheme=None*)

Return a `FIAT.QuadratureRule` on the unit interval.

Parameters

- **degree** – The degree of polynomial that the rule should integrate exactly.
- **scheme** – The quadrature scheme. Can be either *'default'* for Gauss-Legendre, *'Lobatto'* for Gauss-Lobatto-Legendre, or *'Radau'* for Gauss-Radau.

4.1.19 irksome.stage_derivative module

class irksome.stage_derivative.**AdaptiveTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *appctx=None*, *solver_parameters=None*, *bc_type='DAE'*, *splitting=<function AI>*, *nullspace=None*, *tol=0.001*, *dtmin=1e-15*, *dtmax=1.0*, *KI=0.06666666666666667*, *KP=0.13*, *max_reject=10*, *onscale_factor=1.2*, *safety_factor=0.9*, *gamma0_params=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *StageDerivativeTimeStepper*

Front-end class for advancing a time-dependent PDE via an adaptive Runge-Kutta method.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **butcher_tableau** – A `ButcherTableau` instance giving the Runge-Kutta method to be used for time marching.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step size.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **tol** – The temporal truncation error tolerance
- **dtmin** – Minimal acceptable time step. An exception is raised if the step size drops below this threshhold.
- **dtmax** – Maximal acceptable time step, imposed as a hard cap; this can be adjusted externally once the time-stepper is instantiated, by modifying *stepper.dt_max*
- **KI** – Integration gain for step-size controller. Should be less than 1/p, where p is the expected order of the scheme. Larger values lead to faster (attempted) increases in time-step size when steps are accepted. See Gustafsson, Lundh, and Soderlind, BIT 1988.
- **KP** – Proportional gain for step-size controller. Controls dependence on ratio of (error estimate)/(step size) in determining new time-step size when steps are accepted. See Gustafsson, Lundh, and Soderlind, BIT 1988.
- **max_reject** – Maximum number of rejected timesteps in a row that does not lead to a failure
- **onscale_factor** – Allowable tolerance in determining initial timestep to be “on scale”
- **safety_factor** – Safety factor used when shrinking timestep if a proposed step is rejected
- **gamma0_params** – Solver parameters for mass matrix solve when using an embedded scheme with explicit first stage
- **bcs** – An iterable of `DirichletBC` or `EquationBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **nullspace** – An optional nullspace object.

advance()

Attempts to advances the system from time *t* to time *t* + *dt*. If the error threshhold is exceeded, will adaptively decrease the time step until the step is accepted. Also predicts new time step once the step is accepted. Note: overwrites the value *u0*.

class irksome.stage_derivative.**StageDerivativeTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *solver_parameters=None*, *splitting=<function AI>*, *appctx=None*, *bc_type='DAE'*, *aux_indices=None*, *sample_points=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

Front-end class for advancing a time-dependent PDE via a Runge-Kutta method formulated in terms of stage derivatives.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **butcher_tableau** – A `ButcherTableau` instance giving the Runge-Kutta method to be used for time marching.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time step size.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` or `EquationBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method.
- **bc_type** – How to manipulate the strongly-enforced boundary conditions to derive the stage boundary conditions. Should be a string, either "DAE", which implements BCs as constraints in the style of a differential-algebraic equation, or "ODE", which takes the time derivative of the boundary data and evaluates this for the stage values. Support for `EquationBC` in *bcs* is limited to DAE style BCs.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **splitting** – An callable used to factor the Butcher matrix
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **nullspace** – An optional nullspace object.

- **sample_points** – An optional kwarg used to evaluate collocation methods at additional points in time.

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

tabulate_poly(*sample_points*)

irksome.stage_derivative.**getForm**(*F*, *butch*, *t*, *dt*, *u0*, *stages*, *bcs=None*, *bc_type=None*, *splitting=<function AI>*, *aux_indices=None*, *backend: str = 'firedrake'*)

Given a time-dependent variational form and a `ButcherTableau`, produce UFL for the s-stage RK method.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **butch** – the `ButcherTableau` for the RK method being used to advance in time.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step. The user may adjust this value between time steps.
- **u0** – a `Function` referring to the state of the PDE system at time *t*
- **stages** – a `Function` representing the stages to be solved for.
- **bcs** – optionally, a `DirichletBC` or `EquationBC` object (or iterable thereof) containing (possibly time-dependent) boundary conditions imposed on the system.
- **bc_type** – How to manipulate the strongly-enforced boundary conditions to derive the stage boundary conditions. Should be a string, either "DAE", which implements BCs as constraints in the style of a differential-algebraic equation, or "ODE", which takes the time derivative of the boundary data and evaluates this for the stage values. Support for *EquationBC* in *bcs* is limited to DAE style BCs.
- **splitting** – a callable that maps the (floating point) Butcher matrix to a pair of matrices *A1, A2* such that *butch.A = A1 A2*. This is used to vary between the classical RK formulation and Butcher's reformulation that leads to a denser mass matrix with block-diagonal stiffness. Some choices of function will assume that *butch.A* is invertible.
- **aux_indices** – a list of field indices to be discretized as `TimeDerivative`, analogous to `ContinouosPetrovGalerkinTimeStepper`.

Returns

a 2-tuple of - *Fnew*, the `Form` - *bcnew*, a list of `DirichletBC` or `EquationBC`

objects to be posed on the stages

4.1.20 irksome.stage_value module

class irksome.stage_value.**StageValueTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *solver_parameters=None*, *update_solver_parameters=None*, *splitting=<function AI>*, *basis_type=None*, *appctx=None*, *bounds=None*, *use_collocation_update=False*, *sample_points=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

get_update_solver(*update_solver_parameters*)

Build a conservative variational update solve for u_new.

For a mass term `inner(Dt(g(u)), v) * dx` the update head is

inner(g(u_new) - g(u_0), v) * dx

evaluated at the stage-solve test function `v`. For `g = identity` it reduces to `inner(u_new - u_0, v) * dx`, so the discrete update equation is unchanged in the linear case. The remaining (non-time-derivative) part of the form is contributed by the standard RK quadrature `sum_i b_i * F_remainder(stage_i)`.

`update_solver_parameters` does not inherit from `solver_parameters`. The update solve is a different problem from the stage solve – it is posed on `V` rather than `V^s = V x ... x V`, and its Jacobian is a (nonlinear) weighted mass matrix rather than the stage operator. Stage- tuned options such as fieldsplit indices, `snes_type='ksponly'`, lagged Jacobians, or custom multigrid transfers generally do not apply. If `update_solver_parameters` is None, Firedrake's default solver parameters are used (typically a sparse direct solve). Pass an explicit dict to override.

set_initial_guess()

Set a constant-in-time initial guess

tabulate_poly(*sample_points*)

irksome.stage_value.**getFormStage**(*F*, *butch*, *t*, *dt*, *u0*, *stages*, *bcs=None*, *splitting=<function AI>*, *vandermonde=None*, *aux_indices=None*, *backend: str = 'firedrake'*)

Given a time-dependent variational form and a `ButcherTableau`, produce UFL for the s-stage RK method.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **butch** – the `ButcherTableau` for the RK method being used to advance in time.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.

- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – a `Function` referring to the state of the PDE system at time *t*
- **stages** – a `Function` representing the stages to be solved for. It lives in a `FunctionSpace` corresponding to the s-way tensor product of the space on which the semidiscrete form lives.
- **bcs** – optionally, a `DirichletBC` object (or iterable thereof) containing (possibly time-dependent) boundary conditions imposed on the system.
- **splitting** – a callable that maps the (floating point) Butcher matrix a to a pair of matrices *A1, A2* such that *butch.A = A1 A2*. This is used to vary between the classical RK formulation and Butcher's reformulation that leads to a denser mass matrix with block-diagonal stiffness. Only *AI* and *IA* are currently supported.
- **vandermonde** – a numpy array encoding a change of basis to the Lagrange polynomials associated with the collocation nodes from some other (e.g. Bernstein or Chebyshev) basis. This allows us to solve for the coefficients in some basis rather than the values at particular stages, which can be useful for satisfying bounds constraints. If none is provided, we assume it is the identity, working in the Lagrange basis.
- **aux_indices** – a list of field indices, currently ignored.
- **sample_points** – An optional kwarg used to evaluate collocation methods at additional points in time.

Returns

a 2-tuple of - *Fnew*, the `Form` - *bcnew*, a list of `DirichletBC` objects to be posed

on the stages

`irksome.stage_value.`**to_value**(*u0*, *stages*, *vandermonde*)

convert from Bernstein to Lagrange representation

the Bernstein coefficients are [u0; ZZ], and the Lagrange are [u0; UU] since the value at the left-endpoint is unchanged. Since u0 is not part of the unknown vector of stages, we disassemble the Vandermonde matrix (first row is [1, 0, ...]).

4.1.21 irksome.stepper module

`irksome.stepper.`**TimeStepper**(*F*, *method*, *t*, *dt*, *u0*, ***kwargs*)

Helper function to dispatch between various back-end classes
for doing time stepping. Returns an instance of the appropriate class.

Parameters

- **F** – A `ufl.Form` instance describing the semi-discrete problem `F(t, u; v) == 0`, where `u` is the unknown `Function` and `v` is the `TestFunction`.

To specify a linear problem, `F` must be of the form `a(t; w, v) - L(t; v)`, where `w` is a `TrialFunction`.

- **method** – A `ButcherTableau` instance (for RK methods) or a `GalerkinScheme` instance (for CPG or DG) methods to be used in time marching.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` or :class: *EquationBC* containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method.
- **J** – A `ufl.Form` instance with the Jacobian of the semi-discrete problem.
- **Jp** – A `ufl.Form` instance to precondition the semi-discrete linearization.
- **scheme_J** – A `ButcherTableau` or `GalerkinScheme` to discretize the Jacobian.
- **scheme_Jp** – A `ButcherTableau` or `GalerkinScheme` to discretize the preconditioner.
- **constant_jacobian** – A boolean flag indicating whether the Jacobian does not change between time steps. If `dt` is updated, the Jacobian may be flagged for an update via `invalidate_jacobian`.
- **nullspace** – A `VectorSpaceBasis` or `MixedVectorSpaceBasis` specifying a nullspace over the space of `u0`.
- **stage_type** – Whether to formulate in terms of a stage derivatives or stage values. Support for `EquationBC` in `bcs` is limited to the stage derivative formulation.
- **splitting** – A callable used to factor the Butcher matrix
- **bc_type** – For stage derivative formulation, how to manipulate the strongly-enforced boundary conditions. Support for `EquationBC` in `bcs` is limited to DAE style BCs.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **update_solver_parameters** – A `dict` of parameters for inverting the mass matrix at each step (only used if stage_type is “value”)
- **adaptive_parameters** – A `dict` of parameters for use with adaptive time stepping (only used if stage_type is “deriv”)
- **use_collocation_update** – An optional kwarg indicating whether to use the terminal value of the collocation polynomial as the solution up-

date. This is needed to bypass the mass matrix inversion when enforcing bounds constraints with an RK method that is not stiffly accurate. Currently, only constant-in-time boundary conditions are supported.

- **aux_indices** – Only valid for continuous Petrov Galerkin time scheme. It specifies that some of the variables in *u0* are to be treated as auxiliary, that is, discretized in the lower-order DG test space.
- **sample_points** – An optional kwarg used to evaluate collocation methods at additional points in time.

Startup_parameters
An optional `dict` containing parameters used to automatically find starting values for multistep methods.

`irksome.stepper.`**imex_separation**(*F*, *Fexp_kwarg*, *label*)

4.1.22 irksome.tools module

`irksome.tools.`**AI**(*A*)

`irksome.tools.`**IA**(*A*)

`irksome.tools.`**dot**(*A*, *B*)

`irksome.tools.`**extract_timedep_arguments**(*F*, *u0*)
Return both arguments if `F` is a bilinear form, otherwise return the unique argument and `u0`.

`irksome.tools.`**fields_to_components**(*V*, *fields*)
Returns the scalar component indices corresponding to the possibly tensor-valued subspaces of a mixed function space.

Parameters

- **V** – a `FunctionSpace`.
- **fields** – a list of integers defining subspaces of V.

Returns
a list of integers with the scalar components corresponding to

the subfields.

`irksome.tools.`**flatten_dats**(*dats*)

`irksome.tools.`**get_lagrange_permutation**(*L*)
Given a univariate Lagrange element, return the points ordered from left to right and the permutation of the dofs required to obtain this re-ordering.

`irksome.tools.`**get_stage_space**(*V*, *num_stages*, *backend='firedrake'*)

`irksome.tools.`**get_sub**(*u: FunctionSpace | Coefficient*, *indices: tuple[int, ...]*) → FunctionSpace | Coefficient
Recursively access the subfunction of a mixed function space or the space itself, given the indices of the subspace.

Args:

u: A coefficient in a mixed function space indices: A tuple of integers giving the indices of the subspace to access. For example

if u is in a mixed space (V1, V2, V3), then get_sub(u, (0, 2)) will return the third subfunction of u in V1, i.e. *u.sub(0).sub(2)*.

irksome.tools.**is_ode**(*f*, *u*)

Given a form defined over a function *u*, checks if (each bit of) u appears under a time derivative.

irksome.tools.**replace**(*e*, *mapping*)

A wrapper for ufl.replace that allows numpy arrays and skips substitution into sub-expressions wrapped by `lag`.

irksome.tools.**reshape**(*expr*, *shape*)

irksome.tools.**split_stages**(*V*, *stages*)

Reconstruct the stages as a list of Function(V)

4.1.23 Module contents

class irksome.**ARS_DIRK_IMEX**(*ns_imp*, *ns_exp*, *order*)

Bases: *DIRK_IMEX*

Class to generate IMEX tableaux based on Ascher, Ruuth, and Spiteri (ARS). It has members

Parameters

- **ns_imp** – number of implicit stages
- **ns_exp** – number of explicit stages
- **order** – the (integer) former order of accuracy of the method

class irksome.**AdamsBashforth**(*order*)

Bases: *MultistepTableau*

class irksome.**AdamsMoulton**(*order*)

Bases: *MultistepTableau*

class irksome.**Alexander**

Bases: ButcherTableau

Third-order, diagonally implicit, 3-stage, L-stable scheme from Diagonally Implicit Runge-Kutta Methods for Stiff O.D.E.'s, R. Alexander, SINUM 14(6): 1006-1021, 1977.

class irksome.**BDF**(*order*)

Bases: *MultistepTableau*

class irksome.**BackwardEuler**

Bases: *RadauIIA*

The rock-solid first-order implicit method.

irksome.**BoundsConstrainedDirichletBC**(*V*, *g*, *sub_domain*, *bounds*, *solver_parameters=None*, *backend='firedrake'*)

A DirichletBC with bounds-constrained data.

class irksome.**ClassicNystrom4Tableau**

Bases: *NystromTableau*

class irksome.**ClinesBase**

Bases: *NystromAuxiliaryOperatorPC*

Base class for methods out of Clines/Howle/Long.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update
- apply
- applyTranspose

class irksome.**ClinesLD**

Bases: *ClinesBase*

Implements Clines-type preconditioner using Atilde = LD where A=LDU.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- initialize
- update
- apply
- applyTranspose

getAtildes(*A*, *Abar*)

Derived classes produce a typically structured approximation to A and Abar.

class irksome.**ContinuousPetrovGalerkinScheme**(*order*, *basis_type=None*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*)

Bases: *GalerkinScheme*

Class for describing cPG-in-time methods

class irksome.**ContinuousPetrovGalerkinTimeStepper**(*F*, *scheme*, *t*, *dt*, *u0*, *bcs=None*, *bc_type=None*, *aux_indices=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

Front-end class for advancing a time-dependent PDE via a Galerkin in time method

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **scheme** – *ContinuousPetrovGalerkinScheme* encoding the order, basis type, and default quadrature rule of the method.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the method.
- **bc_type** – How to manipulate the strongly-enforced boundary conditions to derive the stage boundary conditions. Should be a string, either "DAE", which implements BCs as constraints in the style of a differential-algebraic equation, or "ODE", which evaluates the temporal degrees of freedom of the boundary data. Currently there is no support for *EquationBC*.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **nullspace** – An optional nullspace object.
- **aux_indices** – a list of field indices to be discretized in the test space rather than trial space.

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*, *basis_type=None*, *order=None*, *quadrature=None*, *aux_indices=None*)

set_initial_guess()

Set a constant-in-time initial guess.

set_update_expressions()

Set up symbolic expressions for the update.

tabulate_poly(*sample_points*)

class irksome.**DIRKIMEXMethod**(*F*, *F_explicit*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *solver_parameters=None*, *mass_parameters=None*, *appctx=None*, *nullspace=None*, *backend='firedrake'*, ***kwargs*)

Bases: `object`

Front-end class for advancing a time-dependent PDE via a diagonally-implicit Runge-Kutta IMEX method formulated in terms of stage derivatives. This implementation assumes a weak form written as F + F_explicit = 0, where both F and F_explicit are UFL

Forms, with terms in F to be handled implicitly and those in F_explicit to be handled explicitly

advance()

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

class irksome.**DIRKNystromTimeStepper**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *bc_type=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: object

Front-end class for advancing a second-order time-dependent PDE via a diagonally-implicit Runge-Kutta-Nystrom method formulated in terms of stage derivatives.

advance()

get_bilinear_form(*form*, *u0*, *stages*, *tableau=None*)

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

update_bc_constants(*i*, *c*)

class irksome.**DIRKTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *backend='firedrake'*, ***kwargs*)

Bases: object

Front-end class for advancing a time-dependent PDE via a diagonally-implicit Runge-Kutta method formulated in terms of stage derivatives.

advance()

get_bilinear_form(*form*, *u0*, *stages*, *tableau=None*)

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

solver_stats()

update_bc_constants(*i*, *c*)

class irksome.**DIRK_IMEX**(*A: ndarray*, *b: ndarray*, *c: ndarray*, *A_hat: ndarray*, *b_hat: ndarray*, *c_hat: ndarray*, *order: int = None*)

Bases: ButcherTableau

Top-level class representing a pair of Butcher tableau encoding an implicit-explicit additive Runge-Kutta method. Since the explicit Butcher matrix is strictly lower triangular, only the lower-left (ns - 1)x(ns - 1) block is given. However, the full b_hat and c_hat are given. It has members

Parameters

- **A** – a 2d array containing the implicit Butcher matrix
- **b** – a 1d array giving weights assigned to each implicit stage when computing the solution at time n+1.
- **c** – a 1d array containing weights at which time-dependent implicit terms are evaluated.
- **A_hat** – a 2d array containing the explicit Butcher matrix (lower-left block only)
- **b_hat** – a 1d array giving weights assigned to each explicit stage when computing the solution at time n+1.
- **c_hat** – a 1d array containing weights at which time-dependent explicit terms are evaluated.
- **order** – the (integer) formal order of accuracy of the method

class irksome.**DiscontinuousGalerkinCollocationScheme**(*order*, *deriv_type='strong'*, *quadrature_degree=None*, *quadrature_scheme='radau'*, *max_quadrature_degree=None*)

Bases: *DiscontinuousGalerkinScheme*

Class for describing collocation DG-in-time methods

as_collocation_scheme()

Return a true collocation scheme with collocated quadrature

class irksome.**DiscontinuousGalerkinScheme**(*order*, *basis_type=None*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*, *deriv_type='strong'*)

Bases: *GalerkinScheme*

Class for describing DG-in-time methods

Parameters

- **order** – An integer indicating the order of the method
- **basis_type** – A string (or tuple of strings) indicating the finite element family (either 'Lagrange' or 'Bernstein') or the Lagrange variant (either 'equispaced', 'spectral', 'chebyshev', or 'integral') for the test/trial spaces. Defaults to equispaced Lagrange elements.

- **quadrature_degree** – An integer or string indicating the degree of the quadrature to be used in time. Defaults to the sum of the degrees of the trial and test spaces. The string value `'auto'` enables automatic degree estimation for each term in the form.
- **quadrature_scheme** – A string indicating the quadrature scheme to be used in time. Defaults to Gauss-Legendre.
- **max_quadrature_degree** – An integer indicating the maximum quadrature degree allowed in the automatic degree estimation. If `None`, then the estimated quadrature degree will always be used.
- **deriv_type** – A string indicating how to integrate terms with time derivatives. Valid values are: - *'weak'*: Time derivatives act on the test function (integrating by parts once). - *'strong'*: Time derivatives act on the unknown (integrating by parts twice).

class irksome.**DiscontinuousGalerkinTimeStepper**(*F*, *scheme*, *t*, *dt*, *u0*, *bcs=None*, *basis_type=None*, *quadrature=None*, *backend='firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

Front-end class for advancing a time-dependent PDE via a Discontinuous Galerkin in time method

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **scheme** – a *DiscontinuousGalerkinScheme* instance describing the order, basis type, default quadrature scheme, and time derivative integration type.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step size.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of *DirichletBC* containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the method.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **nullspace** – An optional nullspace object.

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*, *basis_type=None*, *order=None*, *quadrature=None*, *deriv_type=None*)

set_initial_guess()

Set a constant-in-time initial guess.

tabulate_poly(*sample_points*)

irksome.**Dt**(*f*, *order=1*)

Short-hand function to produce a `TimeDerivative` of a given order.

class irksome.**ExplicitNystromTimeStepper**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *solver_parameters=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *bc_type=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *DIRKNystromTimeStepper*

Front-end class for advancing a second-order time-dependent PDE via an explicit Runge-Kutta-Nystrom method formulated in terms of stage derivatives.

class irksome.**GalerkinCollocationScheme**(*order*, *stage_type='deriv'*, *quadrature_degree=None*, *quadrature_scheme=None*, *max_quadrature_degree=None*)

Bases: *ContinuousPetrovGalerkinScheme*

Class for describing collocation cPG-in-time methods

as_collocation_scheme()

Return a true collocation scheme with collocated quadrature

class irksome.**GaussLegendre**(*num_stages*)

Bases: `CollocationButcherTableau`

Collocation method based on the Gauss-Legendre points. The order of accuracy is 2 * *num_stages*. GL methods are A-stable, B-stable, and symplectic.

Parameters

num_stages – The number of stages (1 or greater)

class irksome.**IRKAuxiliaryOperatorPC**

Bases: `AuxiliaryOperatorPC`

Base class that inherits from Firedrake's AuxiliaryOperatorPC class and provides the preconditioning bilinear form associated with an auxiliary Form and/or approximate Butcher matrix (which are provided by subclasses).

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`
- `apply`
- `applyTranspose`

form(*pc*, *test*, *trial*)

Implements the interface for AuxiliaryOperatorPC.

getAtilde(*A*)

Derived classes produce a typically structured approximation to A.

getNewForm(*pc*, *u0*, *test*)

Derived classes can optionally provide an auxiliary Form.

class irksome.**LobattoIIIA**(*num_stages*)

Bases: CollocationButcherTableau

Collocation method based on the Gauss-Lobatto points. The order of accuracy is 2 * *num_stages* - 2. LobattoIIIA methods are A-stable but not B- or L-stable.

Parameters

num_stages – The number of stages (2 or greater)

class irksome.**LobattoIIIC**(*num_stages*)

Bases: ButcherTableau

Discontinuous collocation method based on the Lobatto points. The order of accuracy is 2 * *num_stages* - 2. LobattoIIIC methods are A-, L-, algebraically, and B- stable.

Parameters

num_stages – The number of stages (2 or greater)

irksome.**MeshConstant**(*msh*, *backend: str = 'firedrake'*)

class irksome.**MultistepTableau**(*a*, *b*, *order=None*)

Bases: object

Top-level class representing a tableau encoding

a multistep method. It has members

Parameters

- **a** – a 1d array containing the coefficients of the previous steps
- **b** – a 1d array containing the weights of the right hand side of the method

property is_explicit

property is_implicit

property num_prev_steps

Return the number of previous steps the method requires.

property num_total_steps

Return the number of steps the method has.

class irksome.**MultistepTimeStepper**(*F*, *method*, *t*, *dt*, *u0*, *bcs=None*, *J=None*, *Jp=None*, *solver_parameters=None*, *bounds=None*, *appctx=None*, *nullspace=None*, *transpose_nullspace=None*, *near_nullspace=None*, *startup_parameters=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *BaseTimeStepper*

front-end class for advancing time-dependent PDE via a general multistep method

Parameters

- **F** – A `ufl.Form` instance describing the semi-discrete problem F(t, u; v) == 0, where *u* is the unknown `Function and `v` is the :class:TestFunction`.
- **method** – A `MultistepMethod` corresponding to the desired multistep method.
- **t** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` containing the strongly-enforced boundary conditions.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **appctx** – An optional `dict` containing application context. This gets included with particular things that Irksome will pass into the nonlinear solver so that, say, user-defined preconditioners have access to it.
- **startup_parameters** – An optional `dict` used to construct a single-step TimeStepper to be used to find the required starting values.

advance()

get_bilinear_form(*form*, *u0*)

get_form_and_bcs(*F*, *t*, *dt*, *u0*, *a*, *b*, *bcs=None*)

solver_stats()

startup()

class irksome.**NystromAuxiliaryOperatorPC**

Bases: `AuxiliaryOperatorPC`

Base class that inherits from Firedrake's AuxiliaryOperatorPC class and provides the preconditioning bilinear form associated with an auxiliary Form and/or approximate Nystrom matrices (which are provided by subclasses).

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`

- `apply`
- `applyTranspose`

form(*pc*, *test*, *trial*)

Implements the interface for AuxiliaryOperatorPC.

getAtildes(*A*, *Abar*)

Derived classes produce a typically structured approximation to A and Abar.

getNewForm(*pc*, *u0*, *ut0*, *test*)

Derived classes can optionally provide an auxiliary Form.

class irksome.**PEPRK**(*ns*, *order*, *peporder*)

Bases: `ButcherTableau`

class irksome.**PareschiRusso**(*x*)

Bases: `ButcherTableau`

Second order, diagonally implicit, 2-stage. A-stable if x >= 1/4 and L-stable iff x = 1 plus/minus 1/sqrt(2).

class irksome.**QinZhang**

Bases: *PareschiRusso*

Symplectic Pareschi-Russo DIRK

class irksome.**RadauIIA**(*num_stages*, *variant='embed_Radau5'*)

Bases: `CollocationButcherTableau`

Collocation method based on the Gauss-Radau points. The order of accuracy is 2 * *num_stages* - 1. RadauIIA methods are algebraically (hence B-) stable.

Parameters

- **num_stages** – The number of stages (2 or greater)
- **variant** – Indicate whether to use the Radau5 style of embedded scheme (with *variant = "embed_Radau5"*) or the simple collocation type (with *variant = "embed_colloc"*)

class irksome.**RadauIIAIMEXMethod**(*F*, *Fexp*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *it_solver_parameters=None*, *prop_solver_parameters=None*, *splitting=<function AI>*, *appctx=None*, *nullspace=None*, *num_its_initial=0*, *num_its_per_step=0*, *backend='firedrake'*, ***kwargs*)

Bases: `object`

Class for advancing a time-dependent PDE via a polynomial IMEX/RadauIIA method. This requires one to split the PDE into an implicit and explicit part. The class sets up two methods – *advance* and *iterate*. The former is used to move the solution forward in time, while the latter is used both to start the method (filling up the initial stage values) and can be used at each time step to increase the accuracy/stability. In the limit as the iterator is applied many times per time step, one expects convergence to the solution that would have been obtained from fully-implicit RadauIIA method.

Parameters

- **F** – A `ufl.Form` instance describing the implicit part of the semi-discrete problem `F(t, u; v) == 0`, where `u` is the unknown `Function` and `v` is the `TestFunction`. To specify a linear problem, `F` must be of the form `a(t; w, v) - L(t; v)`, where `w` is a `TrialFunction`.
- **Fexp** – A `ufl.Form` instance describing the part of the PDE that is explicitly split off.
- **butcher_tableau** – A `ButcherTableau` instance giving the Runge-Kutta method to be used for time marching. Only RadauIIA is allowed here (but it can be any number of stages).
- **t** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as `u0`. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A `Function` containing the current state of the problem to be solved.
- **bcs** – An iterable of `DirichletBC` containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method.
- **it_solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with the iterator.
- **prop_solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with the propagator.
- **splitting** – A callable used to factor the Butcher matrix, currently, only AI is supported.
- **appctx** – An optional `dict` containing application context.
- **nullspace** – An optional null space object.

advance()

get_form_and_bcs(*stages*, *tableau=None*, *F=None*)

invalidate_jacobian()

Forces the matrix to be reassembled next time it is required.

iterate()

Called 1 or more times to set up the initial state of the system before time-stepping. Can also be called after each call to *advance*

propagate()

Moves the solution forward in time, to be followed by 0 or more calls to *iterate*.

solver_stats()

class irksome.**RanaBase**

Bases: *IRKAuxiliaryOperatorPC*

Base class for methods out of Rana, Howle, Long, Meek, & Milestone.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`
- `apply`
- `applyTranspose`

class irksome.**RanaDU**

Bases: *RanaBase*

Implements Rana-type preconditioner using Atilde = DU where A=LDU.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`
- `apply`
- `applyTranspose`

getAtilde(*A*)

Derived classes produce a typically structured approximation to A.

class irksome.**RanaLD**

Bases: *RanaBase*

Implements Rana-type preconditioner using Atilde = LD where A=LDU.

Create a PC context suitable for PETSc.

Matrix free preconditioners should inherit from this class and implement:

- `initialize`
- `update`
- `apply`
- `applyTranspose`

getAtilde(*A*)

Derived classes produce a typically structured approximation to A.

class irksome.**SSPButcherTableau**(*order*, *ns*)

Bases: `ButcherTableau`

Class used to generate tableau for strong stability preserving (SSP) schemes. It has members

Parameters

- **ns** – number of stages
- **order** – the (integer) formal order of accuracy of the method

class irksome.**SSPK_DIRK_IMEX**(*ssp_order*, *ns_imp*, *ns_exp*, *order*)

Bases: *DIRK_IMEX*

Class to generate IMEX tableaux based on Pareschi and Russo. It has members

Parameters

- **ssp_order** – order of ssp scheme
- **ns_imp** – number of implicit stages
- **ns_exp** – number of explicit stages
- **order** – the (integer) formal order of accuracy of the method

class irksome.**StageDerivativeNystromTimeStepper**(*F*, *tableau*, *t*, *dt*, *u0*, *ut0*, *bcs=None*, *bc_type='DAE'*, *backend='firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

tabulate_poly(*sample_points*)

class irksome.**StageValueTimeStepper**(*F*, *butcher_tableau*, *t*, *dt*, *u0*, *bcs=None*, *solver_parameters=None*, *update_solver_parameters=None*, *splitting=<function AI>*, *basis_type=None*, *appctx=None*, *bounds=None*, *use_collocation_update=False*, *sample_points=None*, *backend: str = 'firedrake'*, ***kwargs*)

Bases: *StageCoupledTimeStepper*

get_form_and_bcs(*stages*, *F=None*, *bcs=None*, *tableau=None*)

get_update_solver(*update_solver_parameters*)

Build a conservative variational update solve for u_new.

For a mass term `inner(Dt(g(u)), v) * dx` the update head is

inner(g(u_new) - g(u_0), v) * dx

evaluated at the stage-solve test function `v`. For `g = identity` it reduces to `inner(u_new - u_0, v) * dx`, so the discrete update equation is unchanged in the linear case. The remaining (non-time-derivative) part of the form is contributed by the standard RK quadrature `sum_i b_i * F_remainder(stage_i)`.

`update_solver_parameters` does not inherit from `solver_parameters`. The update solve is a different problem from the stage solve – it is posed on `V` rather than `V^s = V x ... x V`, and its Jacobian is a (nonlinear) weighted mass matrix rather than the stage operator. Stage- tuned options such as fieldsplit indices, `snes_type='ksponly'`, lagged Jacobians, or custom multigrid transfers generally do not apply. If `update_solver_parameters` is None, Firedrake's default solver parameters are used (typically a sparse direct solve). Pass an explicit dict to override.

set_initial_guess()

Set a constant-in-time initial guess

tabulate_poly(*sample_points*)

class irksome.**TimeQuadratureLabel**(**args*, *scheme='default'*)

Bases: Label

If the constructor gets one argument, it's an integer for the order of the quadrature rule. If there are two arguments, assume they are the points and weights.

Parameters

label
The name of the label.

value
The value for the label to take. Can be any type (subject to the validator). Defaults to True.

validator
Function to check the validity of any value later passed to the label. Defaults to None.

default_value

label

validator

value

irksome.**TimeStepper**(*F*, *method*, *t*, *dt*, *u0*, ***kwargs*)

Helper function to dispatch between various back-end classes
for doing time stepping. Returns an instance of the appropriate class.

Parameters

- **F** – A ufl.Form instance describing the semi-discrete problem F(t, u; v) == 0, where u is the unknown Function and v is the TestFunction. To specify a linear problem, F must be of the form a(t; w, v) - L(t; v), where w is a TrialFunction.
- **method** – A ButcherTableau instance (for RK methods) or a GalerkinScheme instance (for CPG or DG) methods to be used in time marching.
- **t** – a Constant or Function on the Real space over the same mesh as u0. This serves as a variable referring to the current time.
- **dt** – a Constant or Function on the Real space over the same mesh as u0. This serves as a variable referring to the current time step size. The user may adjust this value between time steps.
- **u0** – A Function containing the current state of the problem to be solved.
- **bcs** – An iterable of DirichletBC or :class: *EquationBC* containing the strongly-enforced boundary conditions. Irksome will manipulate these to obtain boundary conditions for each stage of the RK method.

- **J** – A `ufl.Form` instance with the Jacobian of the semi-discrete problem.
- **Jp** – A `ufl.Form` instance to precondition the semi-discrete linearization.
- **scheme_J** – A `ButcherTableau` or `GalerkinScheme` to discretize the Jacobian.
- **scheme_Jp** – A `ButcherTableau` or `GalerkinScheme` to discretize the preconditioner.
- **constant_jacobian** – A boolean flag indicating whether the Jacobian does not change between time steps. If `dt` is updated, the Jacobian may be flagged for an update via `invalidate_jacobian`.
- **nullspace** – A `VectorSpaceBasis` or `MixedVectorSpaceBasis` specifying a nullspace over the space of `u0`.
- **stage_type** – Whether to formulate in terms of a stage derivatives or stage values. Support for `EquationBC` in `bcs` is limited to the stage derivative formulation.
- **splitting** – A callable used to factor the Butcher matrix
- **bc_type** – For stage derivative formulation, how to manipulate the strongly-enforced boundary conditions. Support for `EquationBC` in `bcs` is limited to DAE style BCs.
- **solver_parameters** – A `dict` of solver parameters that will be used in solving the algebraic problem associated with each time step.
- **update_solver_parameters** – A `dict` of parameters for inverting the mass matrix at each step (only used if stage_type is "value")
- **adaptive_parameters** – A `dict` of parameters for use with adaptive time stepping (only used if stage_type is "deriv")
- **use_collocation_update** – An optional kwarg indicating whether to use the terminal value of the collocation polynomial as the solution update. This is needed to bypass the mass matrix inversion when enforcing bounds constraints with an RK method that is not stiffly accurate. Currently, only constant-in-time boundary conditions are supported.
- **aux_indices** – Only valid for continuous Petrov Galerkin time scheme. It specifies that some of the variables in *u0* are to be treated as auxiliary, that is, discretized in the lower-order DG test space.
- **sample_points** – An optional kwarg used to evaluate collocation methods at additional points in time.

Startup_parameters
An optional `dict` containing parameters used to automatically find starting values for multistep methods.

class irksome.**WSODIRK**(*ns*, *order*, *ws_order*)

Bases: `ButcherTableau`

irksome.**create_time_quadrature**(*degree*, *scheme=None*)

Return a `FIAT.QuadratureRule` on the unit interval.

Parameters

- **degree** – The degree of polynomial that the rule should integrate exactly.
- **scheme** – The quadrature scheme. Can be either *'default'* for Gauss-Legendre, *'Lobatto'* for Gauss-Lobatto-Legendre, or *'Radau'* for Gauss-Radau.

irksome.**expand_time_derivatives**(*expression*, *t=None*, *timedep_coeffs=None*)

irksome.**getForm**(*F*, *butch*, *t*, *dt*, *u0*, *stages*, *bcs=None*, *bc_type=None*, *splitting=<function AI>*, *aux_indices=None*, *backend: str = 'firedrake'*)

Given a time-dependent variational form and a `ButcherTableau`, produce UFL for the s-stage RK method.

Parameters

- **F** – a `ufl.Form` instance describing the semi-discrete problem.
- **butch** – the `ButcherTableau` for the RK method being used to advance in time.
- **t** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time.
- **dt** – a `Constant` or `Function` on the Real space over the same mesh as *u0*. This serves as a variable referring to the current time step. The user may adjust this value between time steps.
- **u0** – a `Function` referring to the state of the PDE system at time *t*
- **stages** – a `Function` representing the stages to be solved for.
- **bcs** – optionally, a `DirichletBC` or `EquationBC` object (or iterable thereof) containing (possibly time-dependent) boundary conditions imposed on the system.
- **bc_type** – How to manipulate the strongly-enforced boundary conditions to derive the stage boundary conditions. Should be a string, either "DAE", which implements BCs as constraints in the style of a differential-algebraic equation, or "ODE", which takes the time derivative of the boundary data and evaluates this for the stage values. Support for *EquationBC* in *bcs* is limited to DAE style BCs.
- **splitting** – a callable that maps the (floating point) Butcher matrix to a pair of matrices *A1, A2* such that *butch.A = A1 A2*. This is used to vary between the classical RK formulation and Butcher's reformulation that leads to a denser mass matrix with block-diagonal stiffness. Some choices of function will assume that *butch.A* is invertible.
- **aux_indices** – a list of field indices to be discretized as `TimeDerivative`, analogous to `ContinouosPetrovGalerkinTimeStepper`.

Returns

a 2-tuple of - *Fnew*, the `Form` - *bcnew*, a list of `DirichletBC` or `EquationBC`

objects to be posed on the stages

irksome.**lag**(*expr*)

Mark a sub-expression to be evaluated only at the start of the timestep during the implicit solve.

PYTHON MODULE INDEX

i

INDEX

A

B

C

D

E

F

G

H

I

L

M

N

P

Q

R

S

T

U

V

W